TrackFormer 1.2

Training and reproducibility

A single-document account of the released model: data processing, neural architecture, learning objectives, optimizer updates, validation, checkpointing and public reproduction.

Release identity	Document scope
Public 1.2 / development 1.2.73	Selected report checkpoint: completed epoch 4
21,452,595 trainable parameters	Western Pacific pressure-field and typhoon forecasts
Nine input analyses / twenty outputs	Historical inputs -48 to 0 h; predictions +6 to +120 h
Original source + exact frozen corpus	Readable method followed by complete source/record appendices

Evidence boundary. This manual consolidates the available released training record. It does not claim a new full training run, bitwise reproduction across hardware, or recovery of missing original metadata. The original raw IBTrACS snapshot ID, complete runtime lockfile, full process argv and earlier warm-start training history were not recorded/recovered.

Published 30 October 2026. This is a standalone technical manual, separate from the 11-page science-fair report. Training binaries remain downloadable assets; this PDF includes their exact identities and instructions, not 2.64 GB of weather arrays.

Contents

Contents 2

1. What the released model learns 5

The causal boundary . . . 5

One training example . . . 5

2. Source data and exact public corpus 6

Historical preparation versus the released corpus . . . 6

3. Partitions, eligibility and normalization 7

Train-only scaling . . . 7

4. Tensor construction and masks 8

Geography and interpolation . . . 8

5. Architecture, initialization and rollout 9

Inherited backbone . . . 9

Multiscale environmental attention . . . 9

Core initialization . . . 9

Each six-hour step . . . 9

6. The exact objective that is optimized 10

7. Sampling, batches and optimizer updates 11

Epoch sampler . . . 11

Gradient accumulation and truncated backpropagation . . . 11

8. Curriculum, validation and saved history 12

What was and was not checked again . . . 12

9. Checkpoint durability and resume behavior 13

10. Reproduce using the public package 14

Checkpoint-compatible batch reconstruction . . . 14

Original direct entry point . . . 14

11. Export, inference, evaluation and reuse 15

Permanent evidence links . . . 15

Appendix A. Exact artifact identity ledger 16

manifest.json . . . 16

track_windows_v13.npz . . . 16

basin_all_int8.npz . . . 16

basin_slp_atlas_float16.npy . . . 16
pressure_hpa.npy . . . 16
plan.json . . . 16
geography.npz . . . 16
v168_epoch_003.pt . . . 16
v173_epoch_004.pt . . . 16
Frozen dataset manifest . . . 17

Appendix B. Complete source and record archive 18

B01. training/trackformer_1_2/source/v173/DESIGN.md . . . 19
B02. training/trackformer_1_2/source/v173/baseline_model.py . . . 20
B03. training/trackformer_1_2/source/v173/model.py . . . 23
B04. training/trackformer_1_2/source/v173/test_model.py . . . 24
B05. training/trackformer_1_2/source/v173/v165_base.py . . . 26
B06. training/trackformer_1_2/source/v173/v173_pressure_train.py . . . 35
B07. training/trackformer_1_2/EXECUTION_CHECKS.json . . . 38
B08. training/trackformer_1_2/LICENSE . . . 39
B09. training/trackformer_1_2/PUBLICATION.md . . . 40
B10. training/trackformer_1_2/README.md . . . 41
B11. training/trackformer_1_2/SOURCE_MANIFEST.json . . . 43
B12. training/trackformer_1_2/THIRD_PARTY_NOTICES.md . . . 46
B13. training/trackformer_1_2/assets.json . . . 47
B14. training/trackformer_1_2/checkpoint-history.json . . . 48
B15. training/trackformer_1_2/config.json . . . 49
B16. training/trackformer_1_2/download_assets.py . . . 50
B17. training/trackformer_1_2/requirements.txt . . . 51
B18. training/trackformer_1_2/test_training_cpu.py . . . 52
B19. training/trackformer_1_2/train.py . . . 53
B20. CITATION.cff . . . 54
B21. CONTRIBUTING.md . . . 55
B22. LICENSE . . . 56
B23. OPENNESS.md . . . 57
B24. derived-records/selected-checkpoint-training.json . . . 58
B25. docs/trackformer_1_2_training_data.md . . . 59
B26. evaluation/training_data/trackformer_1_2_provenance.json . . . 61
B27. models/trackformer_1_2_field/README.md . . . 63
B28. models/trackformer_1_2_field/WIND_ESTIMATION.md . . . 66
B29. models/trackformer_1_2_field/manifest.json . . . 67
B30. models/trackformer_1_2_field/plot_pressure.py . . . 72
B31. models/trackformer_1_2_field/predict.py . . . 74
B32. release_tools/export_field_1_2.py . . . 77
B33. training/trackformer_1_2/source/data_preparation/requirements.txt . . . 78
B34. training/trackformer_1_2/source/data_preparation/v164_dataset_contract.md . . . 79
B35. training/trackformer_1_2/source/data_preparation/v164_download_pressure_geography.py . . . 80
B36. training/trackformer_1_2/source/data_preparation/v164_global_regional_train.py . . . 83

B37. training/trackformer_1_2/source/data_preparation/v164_prepare_data.py 94
B38. training/trackformer_1_2/source/data_preparation/v164_prepare_mac.py 99
B39. training/trackformer_1_2/source/data_preparation/v164_reuse_impl.py 100
B40. training/trackformer_1_2/source/data_preparation/v164_structure_research.md 104

1. What the released model learns

The released model evolves an eight-channel basin weather state together with a moving, reconstructed pressure core. It advances in six-hour steps. A pressure-based association reads the cyclone position from that evolving core; central pressure is sampled from the same field at the associated position. An auxiliary maximum-wind scalar is supervised separately. There is no independent displayed route head or independent central-pressure head.

This is the pressure-field model identified internally as **v173-moving-core-multiscale-attention**. Older route-only experiments and later 1.3 rainfall candidates are not this release. Released 1.2 has no humidity, satellite-image or rainfall input/output head and no native R34/R50/R64 radius-prediction head. Optional wind-radius diagnostics are pressure-derived assumptions, not newly trained capabilities.

The causal boundary

The model receives only weather history ending at issue time, static geography, issue-time storm position, recent observed motion, current intensity and validity masks. Future weather and best-track values are read as supervised targets only. They do not enter model.initial() or the autoregressive state update. Positive-lead agency forecasts are not inputs. Retrospective reanalysis may be finalized later than the historical issue; a causal timestamp contract is not proof of real-time historical operational availability.

One training example

```
INPUT: nine six-hour analyses at -48, -42, ..., -6, 0 h
       static geography + issue-time storm state + masks
MODEL: initialize -> predict +6 -> +12 -> ... -> +120 h
LABEL: later fields / centres / pressure / wind, used by loss only
UPDATE: backward every four steps -> accumulate microbatches -> AdamW
```

A clean forecast uses one trajectory. The published 50-member examples repeat the same frozen weights with distinct, seeded, small perturbations to historical inputs; 50 members are not 50 separately trained networks.

2. Source data and exact public corpus

Component	Source and transformation
Storm windows	NOAA/IBTrACS-derived track_windows_v13.npz: storm/time identifiers, position, intensity features, targets and masks. Raw snapshot ID not recorded.
Basin weather	NOAA PSL NCEP/NCAR Reanalysis 1: MSLP, HGT500 and U/V at 850, 500, 200 hPa. Six-hour fields, 2.5 degree grid. Seven non-MSLP channels quantized in basin_all_int8.npz; MSLP in a float16 atlas.
Regional pressure	ARCO-ERA5 mean sea-level pressure. Exactly 1,000 issue-centred, fixed 121 x 121 patches on a 0.25 degree grid, with 29 history/target times each.
Geography	ARCO-ERA5 land fraction and surface geopotential divided by 9.80665 for elevation. Latitude/longitude encodings are additional static channels.

The exact six derived archive files and original manifest are public. Their identity is authoritative for this release; re-downloading a current upstream archive may produce different bytes. The basin archive covers 1980-01-01 to 2026-03-17 18 UTC and the derived track archive reaches 2026-07-13. Those cache endpoints are not the fitting cutoff.

Exact asset	Bytes	Role
manifest.json	814	data
track_windows_v13.npz	42,755,049	data
basin_all_int8.npz	274,115,577	data
basin_slp_atlas_float16.npy	111,394,928	data
pressure_hpa.npy	1,698,356,128	data
plan.json	295,005	data
geography.npz	2,547,017	data
v168_epoch_003.pt	256,826,555	checkpoints
v173_epoch_004.pt	257,969,971	checkpoints

All nine manifest/data/checkpoint assets total **2,644,261,044 bytes**. A separate writable basin mmap is reconstructed by the loader; it is a cache, not an extra independent dataset. Full SHA-256 values are reproduced in Appendix A and the complete assets.json in Appendix B.

Historical preparation versus the released corpus

The archived preparation directory contains both the earlier full-field pilot and the later native-pressure/geography downloader. The earlier small ERA5 pilot is explicitly labelled as such and is not the six-file non-pilot corpus. The final loader combines existing NCEP/track archives with the 1,000-window native-pressure package. Using the exact public corpus avoids pretending a missing raw-source build snapshot was recovered. The complete historical utilities are included in this PDF with their original labels.

3. Partitions, eligibility and normalization

Partition	Years	Windows / storms	Native subset
Fit	2000-2021	13,949 / 611	800
Validation	2022-2023	1,041 / 46	100
Original test	2024-2025	1,195 / 56	100

Partition membership uses each storm's first year in the frozen archive. Entire storms remain in one partition. Eligible windows must have an exact basin timestamp, eight earlier six-hour frames and twenty later frames; centre latitude must be finite and strictly between 0 and 60 degrees N, longitude between 100 and 180 degrees E. Windows crossing a partition-year boundary between -48 and +120 h are excluded. Patch membership is cross-checked against exact storm ID, issue timestamp and track archive row.

Native windows are subsets of the eligible windows, not additional storms. A missing native patch is represented by zeros with detail_available false; it is not fabricated from a finer observed grid. Route labels retain missing-future masks. Wind labels are finite and in [0,250) knots, pressure labels finite and in (800,1100) hPa; invalid values are masked.

Train-only scaling

The loader reconstructs physical basin channels, then recomputes population mean and standard deviation using basin frames from 2000-2021 only. It accumulates sums and squares in float64 and floors each standard deviation at 1e-4. Each physical field is normalized as (value - mean) / standard_deviation. This is not normalization fitted on validation/test or on the later cache years.

Channel / unit	Mean	Standard deviation
mslp / hPa	1012.653435072	8.265883618
hgt500 / m	5702.172555051	233.831473571
uwnd850 / m/s	0.655687825	7.441798737
vwnd850 / m/s	0.077107236	5.293417692
uwnd500 / m/s	6.437907876	13.162218958
vwnd500 / m/s	-0.221134819	7.578773744
uwnd200 / m/s	14.724588065	22.626341661
vwnd200 / m/s	-0.201353298	11.056788740

4. Tensor construction and masks

Input key	Shape for one example	Construction
global_history	9 x 8 x 25 x 33	Normalized basin fields at -48 through 0 h.
regional_history	9 x 1 x 121 x 121	Normalized native MSLP, or masked unavailable zeros.
global_static	4 x 25 x 33	Latitude / 90; longitude / 180 - 1; land fraction; elevation / 8000.
regional_static	4 x 121 x 121	Same geographic features on the fixed issue-centred patch.
detail_available	1	Whether the native-pressure archive has this exact window.
center; motion	2 each	Issue lat/lon and prior six-hour east/north displacement in km. Missing prior motion is zero.
issue_intensity; issue_mask	2 each	Current maximum wind and central pressure with independent validity masks.

The returned target dictionary holds twenty future normalized basin fields, regional pressure fields, future centres, wind and pressure, and the corresponding masks. Global masks are populated for the complete checked basin archive. Regional masks are zero when detail is absent. Field outputs carry geometric coverage masks. Input and target dictionaries remain separate; initial() rejects undeclared extra keys.

Geography and interpolation

The basin grid runs north-to-south from 60 to 0 degrees N and 100 to 180 degrees E, 25 x 33 cells. Regional patches use fixed issue-centred offsets from +15 to -15 latitude and -15 to +15 longitude at 0.25 degrees. The moving core is a different grid: 65 x 65 cells at approximately 20 km spacing, spanning +/-640 km. Grid sampling is bilinear with explicit coordinate conventions and FP32 geometry. Display interpolation does not create additional observed resolution.

5. Architecture, initialization and rollout

Inherited backbone

The basin evolution network reads 8 state + 32 recurrent + 4 geography channels (44 total), with width stages 72, 144, 288 and 432 and block depths 2, 2, 4, 4. Residual blocks use depthwise 5 x 5 convolutions, GroupNorm, pointwise expansion, GELU and learned residual scale. Downsampling, bilinear upsampling and skip additions provide spatial context. The moving-core evolution network uses 45 inputs (1 core + 8 environment + 32 memory + 4 static), three outputs and base width 64, with the same depth pattern. Both produce 32-channel features for recurrent memory.

Basin history memory encodes 72 historical weather channels; core memory encodes nine pressure frames. Gated recurrent updates use reset/update gates and a tanh proposal. The baseline's unused fixed fine decoder and separate baseline wind head are removed in the moving-core subclass. The full v173 model contains 21,452,595 trainable parameters.

Multiscale environmental attention

A 1 x 1 projection maps 44 channels to 64. Average pooling with sizes/strides 3 and 5 produces two token scales; two Transformer encoder blocks use 64-dimensional embeddings, four heads, feed-forward width 128, the layer's default ReLU activation, zero dropout and norm_first=True. The convolutional backbone separately uses GELU. A four-head cross-attention returns context to every basin memory cell. A final 64-to-32 linear projection is initialized to zero. Thus the transferred warm-start forecast initially receives zero context correction; the context branch learns during v173 fitting.

Core initialization

Nine historical pressure fields are sampled onto the issue-centred moving grid, using native detail if available and basin pressure otherwise. At issue time only, an explicit Gaussian pressure correction (sigma 100 km, tapered toward the core edge) matches the valid observed current central pressure at the centre cell. This is an observation-conditioned reconstruction, not resolved observed eyewall geometry. That scalar correction is not repeatedly reinserted at every future lead.

Each six-hour step

First add the attention correction to basin memory. Predict basin tendencies and a learned velocity correction bounded by 15 m/s via tanh; combine it with the learned weighted 850/500/200 hPa steering winds. Backward departure sampling transports the state; normalized basin tendency is bounded by 0.2 via tanh. Recurrent basin memory is transported and updated.

Next sample local basin/memory features. A motion network combines them with previous motion and intensity-availability masks. Proposed displacement combines 0.7 times flow over six hours, 0.3 times previous displacement and a bounded 100 km learned correction. Translate the moving frame, transport the pressure anomaly relative to its motion, add a learned core tendency and taper it into the new environment. Core-flow corrections are bounded by 5 m/s. The factor 21.6 converts m/s over six hours into km.

Within 300 km of the moving-grid origin, pressure and a distance prior define a softmax association. Its expected geographic position is the reported centre. Bilinear sampling of the physical core pressure at that same position gives central pressure. The fixed regional reconstruction is basin pressure plus the sampled tapered core anomaly; it is not independently predicted truth. The auxiliary maximum-wind scalar updates with the motion network's third output and is clipped below at zero. Twenty autoregressive steps yield five days. There is no future-truth teacher forcing in the state rollout.

6. The exact objective that is optimized

All losses below come from baseline_model.objective() plus model.objective(). Although v165_base.py contains older objectives, those older standalone objective165 terms are not called by the v173 trainer. SmoothL1 uses PyTorch's default beta=1. Weighted reduction is sum(value x mask) / max(sum(expanded mask),1), preserving masked missing labels.

```
L = L_environment + L_regional
    + 0.20 L_route + 0.50 L_pressure + 0.05 L_wind
    + 0.25 L_pressure_change + 0.10 L_motion_change
```

Term	Definition / scaling
Environment	SmoothL1 of all normalized future basin channels, weighted by cos(latitude) and global target mask.
Regional	SmoothL1 of normalized reconstructed MSLP outside 150 km of the label centre, with native-target and geometric coverage masks. Add 0.2 times absolute first-difference error along each spatial axis, requiring both neighbouring cells valid.
Route	SmoothL1 of east/north position errors divided by 100 km against zero; use label-centre latitude for longitude-to-km conversion and storm validity mask.
Pressure	SmoothL1(predicted_pressure / 20, target_pressure / 20), masked. Pressures are physical hPa.
Wind	SmoothL1(predicted_vmax / 30, target_vmax / 30), masked. Auxiliary scalar in knots.
Pressure change	SmoothL1 of predicted and target six-hour pressure tendencies divided by 10 hPa; both current and previous pressure labels must be valid.
Motion change	SmoothL1 of predicted and observed six-hour displacement divided by 100 km; current and previous storm masks must be valid.

For the first forecast lead, previous pressure/centre comes from the issue-time input with its masks. Later previous target values are used only to define target changes. They do not replace the predicted recurrent state. The inner 150 km exclusion prevents coarse reanalysis absolute-pressure targets from training away the observation-conditioned deep core; central pressure is separately supervised. This exclusion also means that inner spatial structure is weakly constrained and must not be called spatially validated.

7. Sampling, batches and optimizer updates

Epoch sampler

Every eligible TRAIN window appears once in the initial list. Native windows are additionally replayed toward an approximately 25% native draw fraction, with round-robin selection over native storms and a random native window within each selected storm. For N=13,949 and native=800, extra=ceil((0.25N-800)/0.75)=3,583, giving **17,532 draws per epoch**. These are repeated training draws, not 17,532 unique windows. NumPy sampler seed is 165000 + epoch; the final index list is shuffled.

The archived notebook/design recipe uses effective batch 10: 1,754 effective windows per epoch, with a final two-example window. However, direct inspection of the selected checkpoint finds every populated optimizer step counter at **4,384**. Under the published sampler and four complete epochs, batch 10 would give 7,016 updates, whereas **batch 16 gives 1,096 updates per epoch and 4,384 total**. Thus batch 16 is checkpoint-compatible, but it is an inference rather than a recovered original argv. The existing public wrapper/config still represent the archived batch-10 recipe; do not present their default as an exact replay of the selected training run.

Within each effective window the trainer groups native and non-native cases separately, then forms physical microbatches up to the requested size (four in the archived recipe). This avoids mixing native availability inside a microbatch. Microbatch size and worker count cannot be recovered from the optimizer counter. The complete derived checkpoint inspection record is reproduced in Appendix B. Microbatching changes execution memory, not the intended effective example weighting.

Gradient accumulation and truncated backpropagation

For each effective window, zero the optimizer gradients. For each physical microbatch, initialize from historical inputs and roll out the epoch horizon. Each lead's loss contributes loss x microbatch_size / (effective_window_count x horizon_steps). Backpropagate accumulated loss every four forecast leads, then detach every recurrent-state tensor. Gradients remain accumulated across all physical microbatches in that effective window. After the complete effective window, clip the global parameter gradient norm to 1 and perform one AdamW update.

Setting	Archived implementation / recipe
Optimizer	Recorded AdamW group: learning rate 3e-5, betas (0.9,0.999), epsilon 1e-8, weight decay 0.01, amsgrad=false. Full flags and step counters appear in the derived checkpoint record.
Seeds	Python random, NumPy and torch CPU seed 168 at fresh start; sampler has its own epoch seed. Accelerator-specific RNG state is not separately archived.
Precision	FP32. Historical AMP positional option is accepted but does not enable mixed-precision pressure dynamics.
Recipe versus checkpoint	Notebook: effective batch 10 / microbatch 4 / workers 3. Checkpoint: 4,384 updates, compatible with effective batch 16 for the published four-epoch sampler. Original argv, microbatch and workers unrecorded.
Backbone execution	The v173 trainer replaces EvolutionNet.block with a direct module call, disabling its optional activation-checkpoint wrapper.
Finite checks	Non-finite losses or gradient norms raise an error before a new invalid checkpoint is claimed.

8. Curriculum, validation and saved history

Epoch	TRAIN horizon	Validation	Saved TRAIN loss	Saved val. loss
1	4 steps / 24 h	20 steps / 120 h	0.141103579	0.701740651
2	8 steps / 48 h	20 steps / 120 h	0.186764770	0.693910088
3	12 steps / 72 h	20 steps / 120 h	0.232481983	0.720075137
4	20 steps / 120 h	20 steps / 120 h	0.355777216	0.728326361

Validation iterates every eligible validation window in order, without the TRAIN replay sampler and without gradient updates. model.train(False) and disabled gradient calculation put evaluation on the predicted twenty-step trajectory. The metrics accumulator tracks native and non-native groups separately, computes each group's sample-weighted mean loss, then gives nonempty groups equal weight in the reported split loss. These dimensionless aggregate losses are not kilometres, hPa MAE or whole-storm benchmark scores.

Checkpoint selection must be described accurately. The code updates best.pt when this aggregate validation loss falls. The saved history's minimum is epoch 2, not epoch 4. The publicly reported selected release is the completed epoch-4 checkpoint. A fresh run's best.pt can therefore differ from the report's epoch_004.pt. The source defaults to twelve epochs; the portable released-recipe wrapper explicitly requests four. Losses across TRAIN epochs also use different horizons and are not directly comparable as an improvement curve.

What was and was not checked again

The publication preflight rehashed the exact corpus, reproduced all split/native counts and loaded the original warm start into the full parameter count. A CPU-only real-TRAIN execution check used one coarse and one native example: finite forward/loss/backward, nonzero attention gradient, exact field-to-pressure sampling, future-input rejection and complete TRAIN sampler coverage all passed. It performed zero optimizer updates. This is execution evidence, not a new four-epoch reproduction, GPU qualification or skill result. Original Metal tests are included as historical source; they were not rerun by this publication.

9. Checkpoint durability and resume behavior

The mandatory warm start is the verified v168 epoch-3 checkpoint. The trainer checks its exact SHA, version 1.2.68, epoch 3 and dataset identity. It transfers all inherited parameters; only keys under context.* may be missing. The new attention output is zero-initialized; transferred parameters remain trainable, rather than frozen. A fresh AdamW optimizer is created for v173.

A full checkpoint contains version, architecture, dataset manifest SHA, model state, optimizer state, physical data contract, completed epoch count, saved history, best validation loss, optional mid-epoch progress, Python RNG, NumPy RNG and torch CPU RNG. Exact checkpoint hashes distinguish the warm start, selected full checkpoint and inference-only weight export.

Within TRAIN, last.pt is atomically saved every fifty effective windows, at the final window, or after a requested stop reaches an optimizer boundary. Mid-epoch progress stores the next effective-window offset, accumulated native/coarse metric totals and effective batch. A resumed partial epoch requires the same effective batch. On completed validation, epoch_NNN.pt and last.pt are saved; best.pt is also saved if validation improves. history.json is updated. Atomic save writes a temporary file, flushes/fsyncs it and replaces the destination.

The per-output flock prevents two trainers from using the same output. SIGINT/SIGTERM set a stop flag; the original trainer finishes its current optimizer boundary, saves and returns. A crash between periodic saves can lose unsaved work. The original trainer is not an every-update durable journal and does not store separate accelerator RNG state or a complete runtime environment. There is no learning-rate schedule in this v173 loop.

10. Reproduce using the public package

Obtain the GitHub source or the mirrored Hugging Face source. Binary downloads are public and ungated; no Drive links or authentication token are required. The immutable asset upload revision is pinned below. The downloader verifies each size and SHA and refuses to overwrite a differing existing file. Keep data and output in your own writable directories, separate from any active training run.

```
python training/trackformer_1_2/download_assets.py \
  --revision 2d1464e9a310e6a6e8840ef1c23017fdededd57b
python training/trackformer_1_2/download_assets.py --verify-only
python -m pip install -r training/trackformer_1_2/requirements.txt

python training/trackformer_1_2/train.py \
  --device mps --output runs/retrain-1.2 --preflight
python training/trackformer_1_2/test_training_cpu.py

# Explicitly start the four-epoch archived recipe:
python training/trackformer_1_2/train.py \
  --device mps --output runs/retrain-1.2
```

Use cuda on a compatible NVIDIA system or cpu for CPU execution. The requirements file gives minimum packages, not an exact reconstruction of the original environment. Install dependencies in your own environment; the public loader does not install them automatically. The data reader creates an approximately 390 MB writable mmap; each full checkpoint is approximately 258 MB, with additional space needed for temporary atomic saves and output copies. Runtime and memory depend on hardware. No complete original wall-clock training record is claimed.

To keep the downloaded assets on another disk, pass --destination to download_assets.py, then pass the resulting data path and warm-start checkpoint path as --data-root and --warm-start to train.py. Running the same output again resumes its last.pt; a new output begins from the pinned warm start. To inspect the supplied selected release without retraining, use the separate inference package instead.

Checkpoint-compatible batch reconstruction

To match the observed number of optimizer updates under the published sampler, explicitly request batch 16. This is a documented reconstruction, not a claim of bitwise replay. Physical microbatch/worker settings remain user-selected execution settings because the original values are not in the checkpoint.

```
python training/trackformer_1_2/train.py \
+  --device mps --batch 16 --microbatch 4 --workers 3 \
+  --output runs/reconstruct-1.2-batch16
```

Original direct entry point

```
python training/trackformer_1_2/source/v173/v173_pressure_train.py \
  10 3 0 --microbatch 4 --epochs 4 --device mps \
  --manifest training/trackformer_1_2/data/manifest.json \
  --warm-start training/trackformer_1_2/checkpoints/v168_epoch_003.pt \
  --output runs/retrain-1.2
```

11. Export, inference, evaluation and reuse

The public export_field_1_2.py checks the selected full checkpoint SHA, architecture, version, epoch and four saved history entries before writing inference-only weights and a release manifest. The neural parameters are not changed by stripping optimizer/training state. The public predict.py loads the matching original architecture and normalized causal issue packet, advances twenty steps and saves physical basin, fixed-regional and moving-core pressure with coordinates, valid times, coverage and field-derived centre/pressure. A map renderer operates on those actual saved fields, not a synthetic replacement vortex.

The inference-only weights have SHA db49f36e85a3766defc4c172746897a1f783705d1ce8e6f9dfb8e87ae1d902cb. The full selected checkpoint has SHA f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0. Treat Python-pickle full checkpoints as trusted project artifacts only after exact hash verification. Physical pressure export and renderer source are reproduced in Appendix B.

The public report comparison uses 1,473 starts across 270 storms, not the training loss accumulator. Common pressure scoring uses 134 starts / 40 storms / 2,637 valid records against JMA. Equal-storm aggregation and masked common support differ from the trainer's native/coarse equal-group objective. Reused historical and showcase cases are development evidence. The 50-input-member policy has normalized noise amplitudes 0.025 for basin and 0.015 for regional history, with smoothing kernels 5 and 9 respectively; it is not a change to the weights or proof of uncertainty calibration.

The project code and project weights/checkpoints use MIT, permitting modification, retraining, redistribution and commercial use subject to its notice requirements. The dataset retains upstream NOAA/IBTrACS/Copernicus/ERA5 terms and attribution; MIT does not replace them. Research/safety cautions describe demonstrated capability, not additional non-commercial licensing restrictions. Licenses, openness inventory and contributing guidance are reproduced in the appendices.

Permanent evidence links

GitHub release source: https://github.com/yu314-coder/typhoon-predict/tree/9006d43348d8aeb9c1ae500a83ac9edc420028a6

Hugging Face source/card snapshot: https://huggingface.co/euler314/typhoon-predict/tree/1f4ed985a05589076d37f170cc8053e26cec9fba

Exact training assets: https://huggingface.co/euler314/typhoon-predict/tree/2d1464e9a310e6a6e8840ef1c23017fdededd57b/training/trackformer_1_2

NOAA NCEP Reanalysis: https://psl.noaa.gov/data/gridded/data.ncep.reanalysis.html

NOAA IBTrACS: https://www.ncei.noaa.gov/products/international-best-track-archive

ARCO-ERA5 mirror: https://github.com/google-research/arco-era5

Appendix A. Exact artifact identity ledger

The following full hashes and byte counts identify the downloadable training assets. These are exact arrays/checkpoints, not replacements generated from current upstream downloads. The release manifest, normalization, split provenance and source file checksums are also reproduced in full in Appendix B.

manifest.json

training/trackformer_1_2/data/manifest.json

814 bytes; SHA-256:

`04fd0d30620137aad20173dcd59e22da2a6c20fae2f17de593e91a59517459c1`

track_windows_v13.npz

training/trackformer_1_2/data/track_windows_v13.npz

42,755,049 bytes; SHA-256:

`cb8d7291aa8140124ca522feb448a1d44124b018efc4ba1d54824b60a4f73451`

basin_all_int8.npz

training/trackformer_1_2/data/basin_all_int8.npz

274,115,577 bytes; SHA-256:

`71e5f1bb46c44e5936c506a3f8af021c1fe94c238ed030c05c66e8f342374b6d`

basin_slp_atlas_float16.npy

training/trackformer_1_2/data/basin_slp_atlas_float16.npy

111,394,928 bytes; SHA-256:

`9f6a4ec7b2f6beadfa1075af9176d2a653e39a824d504d989e5e56e19c571521`

pressure_hpa.npy

training/trackformer_1_2/data/pressure_hpa.npy

1,698,356,128 bytes; SHA-256:

`16cc1b153aeee5eff4b67337b599892dcd86a5716554596b8031048035d247ba`

plan.json

training/trackformer_1_2/data/plan.json

295,005 bytes; SHA-256:

`117eb291e385a326191be2f73182ed6dcbf0e9f99745780ac9e49a29def3d8da`

geography.npz

training/trackformer_1_2/data/geography.npz

2,547,017 bytes; SHA-256:

`f5471471da4a1e15f85fbfb2529065806281357c8e50cb272fcd2b02e8553b96`

v168_epoch_003.pt

training/trackformer_1_2/checkpoints/v168_epoch_003.pt

256,826,555 bytes; SHA-256:

`99ed776097e05ad6df03375465d9618741bed63a4ee57724d3ba98e295039d30`

v173_epoch_004.pt

training/trackformer_1_2/checkpoints/v173_epoch_004.pt

257,969,971 bytes; SHA-256:

`f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0`

Frozen dataset manifest

`04fd0d30620137aad20173dcd59e22da2a6c20fae2f17de593e91a59517459c1`

The exact same source files as the public package are embedded in this PDF as an attached ZIP. This attachment is optional: every text file is also printed in Appendix B, so a reader does not need an attachment-capable PDF viewer or separate Markdown pages. Large weather/checkpoint binaries are linked assets, not embedded copies.

Appendix B. Complete source and record archive

This appendix reproduces every text file from the open training package, plus the release input/output specification, export/renderer source and key provenance/license documents. Listings preserve original content and add display-only line numbers and wrapping. For exact executable bytes use the embedded source ZIP or immutable repositories. Earlier preparation research/pilot documents retain their original historical labels; their unimplemented proposals are not claims about v173.

The printed source is authoritative for implementation details not repeated in the narrative. In particular, use v173 model.objective() and baseline_model.objective(), not unused older objective functions in inherited source. Source checksums below identify the exact text; the PDF does not contain credentials or binary training archives.

B01. training/trackformer_1_2/source/v173/DESIGN.md

3,278 bytes | SHA-256: 163ae39ff572f23c519d65e27ee6a8db23b47df9461670709229ccf95130049a

```
# v1.2.73: controlled moving-core improvement

Baseline: v168 E3, SHA256
99ed776097e05ad6df03375465d9618741bed63a4ee57724d3ba98e295039d30.
The baseline is frozen as an archived reference, not frozen during v173 training.

The 21,452,595-parameter model transfers every baseline parameter. A two-scale
environment encoder pools physical state, recurrent memory and geographic
features. Two transformer blocks exchange regional context; cross attention
returns a correction to the environmental recurrent state. A zero-initialized
output makes the initial forecast equal the baseline. The moving core continues
to evolve pressure through relative transport and learned tendencies.

Both objectives matter: spatial pressure evolution and derived storm numbers.
The existing area-weighted atmospheric, outer-core pressure/gradient, track,
central-pressure and auxiliary wind losses remain. New masked losses supervise
six-hour pressure change and displacement against corresponding label changes.
Future fields and best tracks enter only the objective. Validation always uses
120 hours; training horizons are 24, 48, 72 then 120 hours. Best selection uses
validation, never Tip or the displayed benchmark storms.

The route is a pressure association within the moving core; central pressure is
sampled from that same field at that location. Maximum wind remains an auxiliary
scalar estimate: MSLP alone does not uniquely determine surface wind. The fixed
regional rendering is a resampled composite; the core is authoritative for the
extracted central pressure. Display +0 from the initialized core, and distinguish
it from raw input analysis. Do not connect raw +0 to conditioned +6.

Deliberately retained limitations: the v168 initialization includes a 100 km
Gaussian correction using issue-time observed pressure. It is a reconstruction,
not resolved truth. Fine inner geometry is underdetermined with only 800 native
training windows. Replacing the initializer, tracker, transport and backbone all
at once would destroy the controlled baseline. Learned initialization and a
probabilistic ensemble are later separate ablations, not implemented claims.

Only the verified full existing corpus is accepted: 13949/1041/1195 windows and
800/100/100 native windows. No new weather downloads. Tip is excluded, and its
repeatedly inspected forecasts are development diagnostics. Region is Western
Pacific, not a global weather model. Report native/coarse cases separately and
validate on untouched storms before calling this an improvement.

Mac Metal tests passed: baseline equivalence before optimization, coarse/native
four-step finite backward with attention gradients, field-to-pressure equality,
120h finite rollout, label invariance, rejected future inputs, missing intensity,
optimizer resume and full sampler coverage. CUDA/T4 and skill remain unverified.

Training command after the existing notebook's setup cell:
`python v173_pressure_train.py 10 3 1 --microbatch 4`
The final positional flag is accepted for compatibility; dynamics use FP32.
Increase physical microbatch only after measuring throughput and memory. Effective
batch alone does not increase GPU memory. Checkpoints and train.log persist to
MyDrive/typhoon_predict/training/v173; restarting resumes last.pt.
```

B02. training/trackformer_1_2/source/v173/baseline_model.py

10,040 bytes | SHA-256: 6c05941fd93baaca04508877edc69f8f0394b70e0a64e734ef0d2c1e8832246a

```
"""Observation-initialized moving pressure state; no independent pressure head."""
from pathlib import Path
import sys
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F

import v165_base as base

VERSION = '1.2.68'
ARCHITECTURE = 'moving-pressure-core-v168-prototype'


def weighted(value, mask):
    return (value * mask).sum() / mask.expand_as(value).sum().clamp_min(1)


class CoreForecaster(base.TransportForecaster):
    def __init__(self, manifest, tiny=False):
        super().__init__(manifest, tiny)
        del self.fine, self.fine_memory, self.wind
        self.core_net = base.EvolutionNet(1 + 8 + 32 + 4, 3, 8 if tiny else 64, tiny)
        self.core_memory = base.RecurrentFeatures(9)
        self.motion_net = nn.Sequential(nn.Linear(32 + 8 + 4, 64), nn.GELU(), nn.Linear(64, 3))
        nn.init.zeros_(self.motion_net[-1].weight)
        nn.init.zeros_(self.motion_net[-1].bias)
        # 20 km internal grid is a reconstruction, not new resolved observations.
        yy, xx = torch.meshgrid(torch.linspace(640, -640, 65), torch.linspace(-640, 640, 65), indexing='ij')
        self.register_buffer('east', xx[None])
        self.register_buffer('north', yy[None])
        radius = (xx.square() + yy.square()).sqrt()
        self.register_buffer('taper', ((640-radius)/160).clamp(0, 1)[None, None])

    def coordinates(self, center):
        lat = center[:, 0, None, None] + self.north / 111.2
        lon = center[:, 1, None, None] + self.east / (111.2 * center[:, 0, None,
None].deg2rad().cos().clamp_min(.2))
        return lat, lon

    def basin_grid(self, lat, lon):
        return torch.stack([(lon-100)/40-1, 1-lat/30], -1)

    def core_grid(self, lat, lon, origin):
        east = (lon-origin[:, 1, None, None])*111.2*origin[:, 0, None, None].deg2rad().cos().clamp_min(.2)
        north = (lat-origin[:, 0, None, None])*111.2
        return torch.stack([east/640, -north/640], -1)

    def initial(self, inputs):
        required = {'global_history','regional_history','global_static','regional_static','detail_available','ce
nter','motion','issue_intensity','issue_mask'}
        if set(inputs) != required:
            raise ValueError('Only declared past and issue-time inputs are allowed')
        history = inputs['global_history']
        lat, lon = self.coordinates(inputs['center'])
        grid = self.basin_grid(lat, lon)
        regional = inputs['regional_static']
        rlat, rlon = regional[:, 0]*90, (regional[:, 1]+1)*180
        native_grid = torch.stack([2*(lon-rlon[:, :1, :1])/(rlon[:, :1, -1:]-rlon[:, :1, :1])-1,
                                   2*(lat-rlat[:, :1, :1])/(rlat[:, -1:, :1]-rlat[:, :1, :1])-1], -1)
        parent = base.sample_field(history[:, -1, :1], grid)
        available = inputs['detail_available'][:, :, None, None].bool()
        past = []
        for i in range(9):
            coarse = base.sample_field(history[:, i, :1], grid)
            native = base.sample_field(inputs['regional_history'][:, i], native_grid)
            past.append(torch.where(available, native, coarse))
        core = past[-1]
        observed = (inputs['issue_intensity'][:, 1] - self.mean[0, 0, 0, 0]) / self.std[0, 0, 0, 0]
        # Only initialization uses an explicit compact core prior. Subsequent
        # changes are learned field tendencies, not repeated scalar insertion.
        radius2 = self.east.square() + self.north.square()
        prior = torch.exp(-radius2/(2*100**2))[:, None] * self.taper
        difference = observed-core[:, 0, 32, 32]
        core = core + difference[:, None, None, None]*prior*inputs['issue_mask'][:, 1, None, None, None]
        return dict(g=history[:, -1], gh=self.coarse_memory.encode(history.flatten(1, 2)),
                    core=core, ch=self.core_memory.encode(torch.cat(past, 1)),
                    anomaly=core-parent, origin=inputs['center'], center=inputs['center'],
motion=inputs['motion'],
                    vmax=inputs['issue_intensity'][:, 0], gs=inputs['global_static'], rs=regional,
                    issue=inputs['issue_intensity'], issue_mask=inputs['issue_mask'])

```

```
    def step(self, s):
        delta, features = self.coarse(torch.cat([s['g'], s['gh'], s['gs']], 1))
        velocity = self.steering(s['g']) + 15*torch.tanh(delta[:, 8:])
        departure = base.departure_grid(s['g'], velocity, self.lat, 2.5)
        g = base.sample_field(s['g'], departure) + .2*torch.tanh(delta[:, :8])
        gh = self.coarse_memory(base.sample_field(s['gh'], departure), features)
        at = self.basin_grid(s['center'][:, 0, None, None], s['center'][:, 1, None, None])
        local = base.sample_field(torch.cat([gh, g], 1), at)[:, :, 0, 0]
        conditioning = torch.cat([s['motion']/100, s['issue_mask']], 1)
        tendencies = self.motion_net(torch.cat([local, conditioning], 1))
        flow = base.sample_field(velocity, at)[:, :, 0, 0]
        motion = .7*flow*21.6 + .3*s['motion'] + 100*torch.tanh(tendencies[:, :2])
        origin = s['center'] + torch.stack([motion[:, 1]/111.2,
                    motion[:, 0]/(111.2*s['center'][:, 0].deg2rad().cos().clamp_min(.2))], 1)
        lat, lon = self.coordinates(origin)
        grid = self.basin_grid(lat, lon)
        environment = base.sample_field(g, grid)
        static = base.sample_field(s['gs'], grid)
        # Translate the moving frame without resampling away bulk storm motion.
        old_lat, old_lon = self.coordinates(s['origin'])
        old_grid = self.basin_grid(old_lat, old_lon)
        old_environment = base.sample_field(s['g'], old_grid)
        rd, rf = self.core_net(torch.cat([s['core'], old_environment, s['ch'], static], 1))
        local_flow = base.sample_field(velocity, grid) + 5*torch.tanh(rd[:, 1:])
        relative = local_flow - (motion/21.6)[:, :, None, None]
        shift = (s['center']-s['origin'])
        source_lat = old_lat + shift[:, 0, None, None] - relative[:, 1]*21.6/111.2
        source_lon = old_lon + shift[:, 1, None, None] - relative[:,
0]*21.6/(111.2*old_lat.deg2rad().cos().clamp_min(.2))
        dep = self.core_grid(source_lat, source_lon, s['origin'])
        anomaly = base.sample_field(s['anomaly'], dep, 'zeros') + .1*rd[:, :1]
        core = environment[:, :1] + anomaly*self.taper
        ch = self.core_memory(base.sample_field(s['ch'], dep, 'zeros'), rf)
        physical = core*self.std[:, :1]+self.mean[:, :1]
        # Field-based association, constrained to this storm's local neighborhood.
        score = -physical[:, 0]/2 - (self.east.square()+self.north.square())/(2*180**2)
        score = score.masked_fill((self.east.square()+self.north.square()) > 300**2, -1e9)
        pdf = score.flatten(1).softmax(1).reshape_as(score)
        center = torch.stack([(pdf*lat).sum((1,2)), (pdf*lon).sum((1,2))], 1)
        cg = self.core_grid(center[:, 0, None, None], center[:, 1, None, None], origin)
        pressure = base.sample_field(physical, cg)[:, 0, 0, 0]
        rlat, rlon = s['rs'][:, 0]*90, (s['rs'][:, 1]+1)*180
        rg = self.basin_grid(rlat, rlon)
        sampling = self.core_grid(rlat, rlon, origin)
        reconstructed = base.sample_field(g[:, :1], rg) + base.sample_field(anomaly*self.taper, sampling,
'zeros')
        # Resolved basin output remains at source resolution; reconstruction has
        # separate provenance and is never relabeled as raw coarse analysis.
        vmax = (s['vmax']+5*tendencies[:, 2]).clamp_min(0)
        valid = (grid.abs()<=1).all(-1)[:, None]
        out = dict(global_=g, regional=reconstructed, regional_valid=(rg.abs()<=1).all(-1)[:,None],
                   core=core, core_lat=lat, core_lon=lon, core_valid=valid,
                   center=center, pressure=pressure, grid_pressure=pressure, vmax=vmax,
                   track_valid=(center[:,0]>0)&(center[:,0]<60)&(center[:,1]>100)&(center[:,1]<180),
                   native_route=torch.ones_like(pressure,dtype=torch.bool),
                   pressure_tendency=pressure-base.sample_field(s['core']*self.std[:,:1]+self.mean[:,:1],

self.core_grid(s['center'][:,0,None,None],s['center'][:,1,None,None],s['origin']))[:,0,0,0])
        out['global']=out.pop('global_')
        return
dict(s,g=g,gh=gh,core=core,ch=ch,anomaly=anomaly,origin=origin,center=center,motion=motion,vmax=vmax), out


def objective(out, target, inputs, manifest):
    terms={}
    area=inputs['global_static'][:,:1].mul(90).deg2rad().cos().clamp_min(0)
    terms['environment']=weighted(F.smooth_l1_loss(out['global'],target['global'],reduction='none'),area*target[
'global_mask'])
    # Reanalysis does not resolve the observed central pressure. Exclude the
    # inner 150 km from absolute reconstruction targets instead of erasing it.
    lat,lon=inputs['regional_static'][:,0]*90,(inputs['regional_static'][:,1]+1)*180
    ct=target['center']
    d2=((lat-ct[:,0,None,None])*111.2).square()+((lon-ct[:,1,None,None])*111.2*ct[:,0,None,None].deg2rad().cos()
).square()
    mask=target['regional_mask']*out['regional_valid']*(d2>=150**2)[:,None]
    terms['regional']=weighted(F.smooth_l1_loss(out['regional'],target['regional'],reduction='none'),mask)
    for axis in (-2,-1):
        pred=out['regional'].diff(dim=axis);truth=target['regional'].diff(dim=axis)
        w=mask.narrow(axis,0,pred.shape[axis])*mask.narrow(axis,1,pred.shape[axis])
        terms['regional']=terms['regional']+.2*weighted((pred-truth).abs(),w)

displacement=(out['center']-ct)*torch.stack([torch.full_like(ct[:,0],111.2),111.2*ct[:,0].deg2rad().cos()],1)
    terms['route']=weighted(F.smooth_l1_loss(displacement/100,torch.zeros_like(displacement),reduction='none'),t
arget['storm_mask'][:,None])
    terms['pressure']=weighted(F.smooth_l1_loss(out['pressure']/20,target['pressure']/20,reduction='none'),targe
t['pressure_mask'])
```

```
157
     terms['wind']=weighted(F.smooth_l1_loss(out['vmax']/30,target['vmax']/30,reduction='none'),target['vmax_mask'])
158      return terms['environment']+terms['regional']+.2*terms['route']+.5*terms['pressure']+.05*terms['wind'],terms
```

B03. training/trackformer_1_2/source/v173/model.py

3,428 bytes | SHA-256: 5dab06c20224cf48c51302122455ee7e16948f977bf4b760fd8aff5d443ab0bc

```
"""v173: moving pressure core with multiscale environmental attention.

The public track and central pressure are extracted from the evolving core.
Wind is an auxiliary scalar estimate, not a resolved wind field.
"""
import torch
from torch import nn
from torch.nn import functional as F
from baseline_model import CoreForecaster as Baseline, objective as baseline_loss, weighted, base

VERSION = '1.2.73'
ARCHITECTURE = 'v173-moving-core-multiscale-attention'


class EnvironmentalAttention(nn.Module):
    def __init__(self):
        super().__init__()
        self.project = nn.Conv2d(44, 64, 1)
        layer = nn.TransformerEncoderLayer(64, 4, 128, dropout=0., batch_first=True,
                                           norm_first=True)
        self.encoder = nn.TransformerEncoder(layer, 2, enable_nested_tensor=False)
        self.query = nn.Linear(32, 64)
        self.cross = nn.MultiheadAttention(64, 4, dropout=0., batch_first=True)
        self.output = nn.Linear(64, 32)
        nn.init.zeros_(self.output.weight)
        nn.init.zeros_(self.output.bias)

    def forward(self, state):
        h, w = state['gh'].shape[-2:]
        features = self.project(torch.cat([state['g'], state['gh'], state['gs']], 1))
        tokens = torch.cat([F.avg_pool2d(features, kernel_size=size, stride=size,
                                         ceil_mode=True).flatten(2)
                            for size in (3, 5)], 2).transpose(1, 2)
        tokens = self.encoder(tokens)
        queries = self.query(state['gh'].flatten(2).transpose(1, 2))
        context, _ = self.cross(queries, tokens, tokens, need_weights=False)
        return self.output(context).transpose(1, 2).reshape(-1, 32, h, w)


class CoreForecaster(Baseline):
    def __init__(self, manifest, tiny=False):
        super().__init__(manifest, tiny)
        # Meshgrid views overlap on CPU; checkpoint loading requires owned storage.
        self.east = self.east.clone()
        self.north = self.north.clone()
        self.context = EnvironmentalAttention()

    def step(self, state):
        conditioned = dict(state, gh=state['gh'] + self.context(state))
        next_state, out = super().step(conditioned)
        out['previous_center'] = state['center']
        out['previous_pressure'] = out['pressure'] - out['pressure_tendency']
        return next_state, out


def objective(out, target, inputs, manifest):
    loss, terms = baseline_loss(out, target, inputs, manifest)
    if 'previous_pressure' in target:
        mask = target['pressure_mask'] * target['previous_pressure_mask']
        true_delta = target['pressure'] - target['previous_pressure']
        terms['pressure_change'] = weighted(F.smooth_l1_loss(
            out['pressure_tendency']/10, true_delta/10, reduction='none'), mask)
        loss = loss + .25 * terms['pressure_change']
    if 'previous_center' in target:
        center = target['center']
        scale = torch.stack([torch.full_like(center[:, 0], 111.2),
                             111.2*center[:, 0].deg2rad().cos()], 1)
        predicted = (out['center']-out['previous_center'])*scale
        truth = (center-target['previous_center'])*scale
        mask = target['storm_mask']*target['previous_storm_mask']
        terms['motion_change'] = weighted(F.smooth_l1_loss(
            predicted/100, truth/100, reduction='none'), mask[:, None])
        loss = loss + .1 * terms['motion_change']
    return loss, terms
```

B04. training/trackformer_1_2/source/v173/test_model.py

5,579 bytes | SHA-256: 493e0db88a098c89bb4db5e6c577fa24b4b7bcb83ce547dceac35237dbce7206

```
"""Real-data Metal preflight, pressure coupling, gradients and resume."""
from pathlib import Path
import tempfile
import torch
from model import CoreForecaster, objective, base
from v173_pressure_train import save, restore, warm_start, batches
from baseline_model import CoreForecaster as Baseline


def main():
    root=Path(__file__).resolve().parents[1]
    torch.set_num_threads(4);torch.manual_seed(168)
    base.EvolutionNet.block=lambda self,module,x:module(x)
    m=base.load_reuse(root/'data/v164_reuse/manifest.json')
    assert m['counts']==dict(train=13949,validation=1041,test=1195)
    assert not any('1979275N06159'==r['storm_id'] for r in m['rows'])
    model=CoreForecaster(m).to('mps');warm_start(model,root/'v168/runs/mac/epoch_003.pt',m)
    reference=Baseline(m).to('mps')
    reference.load_state_dict(torch.load(root/'v168/runs/mac/epoch_003.pt',map_location='cpu',weights_only=False)['model'])
    reference.eval();model.eval()
    optimizer=torch.optim.AdamW(model.parameters(),lr=3e-5,weight_decay=.01)
    data=base.ReuseDataset(m,'train')
    choices=[next(i for i,r in enumerate(data.rows) if r['patch']<0 and r['issue_mask'][1]),
             next(i for i,r in enumerate(data.rows) if r['patch']>=0 and r['issue_mask'][1])]
    for index in choices:
        x,y=data[index];x={k:v[None].to('mps') for k,v in x.items()}
        if index==choices[0]:
            with torch.no_grad():
                expected=reference(x,1);actual=model(x,1)
                for key in expected: assert torch.allclose(actual[key],expected[key],atol=1e-5,rtol=1e-5), key
            print('PASS zero-correction warm start equals v168 E3 field and route',flush=True)
        model.train()
        optimizer.zero_grad(set_to_none=True);state=model.initial(x)
        initial_p=state['core'][:,0,32,32]*model.std[0,0,0,0]+model.mean[0,0,0,0]
        assert torch.allclose(initial_p,x['issue_intensity'][:,1],atol=1e-3)
        total=0
        for lead in range(4):
            state,out=model.step(state)
            target={k:v[lead:lead+1].to('mps') for k,v in y.items()}
            target['previous_pressure']=y['pressure'][lead-1:lead].to('mps') if lead else x['issue_intensity'][:,1]
            target['previous_pressure_mask']=y['pressure_mask'][lead-1:lead].to('mps') if lead else x['issue_mask'][:,1]
            target['previous_center']=y['center'][lead-1:lead].to('mps') if lead else x['center']
            target['previous_storm_mask']=y['storm_mask'][lead-1:lead].to('mps') if lead else torch.ones_like(target['storm_mask'])
            loss,_=objective(out,target,x,m);total=total+loss/4
            assert all(bool(torch.isfinite(v).all()) for v in out.values())
            grid=model.core_grid(out['center'][:,0,None,None],out['center'][:,1,None,None],state['origin'])
            field_pressure=base.sample_field(out['core']*model.std[:,:1]+model.mean[:,:1],grid)[:,0,0,0]
            assert torch.equal(field_pressure,out['pressure'])
        total.backward()
        assert all(bool(torch.isfinite(p.grad).all()) for p in model.parameters() if p.grad is not None)
        for layer in [model.core_net.out,model.coarse.out,model.motion_net[-1]]:
            assert layer.weight.grad is not None and layer.weight.grad.abs().sum()>0
        assert model.context.output.weight.grad.abs().sum()>0
        torch.nn.utils.clip_grad_norm_(model.parameters(),1.);optimizer.step()
        print('PASS real 4-step backward, anchored pressure and field-derived output;native=',bool(x['detail_available'].item()),flush=True)
    model.eval()
    with torch.no_grad():
        out=model(x,20)
        assert all(bool(torch.isfinite(v).all()) for v in out.values())
        first=model(x,1)
        altered={k:v.clone() for k,v in target.items()};altered['center']+=10
        objective({k:v[:,0] for k,v in first.items()},altered,x,m)
        again=model(x,1)
        assert all(torch.equal(first[k],again[k]) for k in first)
        try:model.initial(dict(x,future_pressure=target['regional']))
        except ValueError:pass
        else:raise AssertionError('Future input accepted')
        missing={k:v.clone() for k,v in x.items()};missing['issue_mask'].zero_();missing['issue_intensity'].zero_()
        absent=model(missing,1)
        assert all(bool(torch.isfinite(v).all()) for v in absent.values())
    print('PASS 120h rollout, label invariance, missing intensity and future-input rejection',flush=True)
    with tempfile.TemporaryDirectory(dir=root/'v173') as tmp:

path=Path(tmp)/'resume.pt';save(path,model,optimizer,m,0,[],100.,dict(window=1,totals=[[1.,1],[1.,1]],batch=10))
        def update():
            model.train();optimizer.zero_grad(set_to_none=True)
```

```
            o=model(x,1);loss,_=objective({k:v[:,0] for k,v in o.items()},target,x,m)
            loss.backward();torch.nn.utils.clip_grad_norm_(model.parameters(),1.);optimizer.step()
            return model.core_net.out.weight.detach().clone()
        expected=update();payload=restore(path,model,optimizer,m);actual=update()
        assert torch.allclose(expected,actual,atol=1e-6,rtol=1e-5)
        assert payload['progress']['window']==1
    indices=base.training_indices(data.rows,1)
    grouped=list(batches(indices,data.rows,10,2))
    assert sorted(i for group,_ in grouped for part in group for i in part)==sorted(indices)
    assert set(indices)==set(range(len(data)))
    print('PASS checkpoint optimizer resume and full dataset coverage',flush=True)
    print('parameters=',sum(p.numel() for p in model.parameters()),flush=True)


if __name__=='__main__':main()
```

B05. training/trackformer_1_2/source/v173/v165_base.py

39,040 bytes | SHA-256: 5c9473993a19d0fed00055494b5c1f50c1006aaf46551ff50145f67048c3374a

```
#!/usr/bin/env python3
"""Standalone v1.2.65. Existing full archives; no web downloads or pilot fallback."""
from __future__ import annotations

import argparse
import hashlib
import json
import math
import os
from pathlib import Path
import random
import socket
import time
from contextlib import nullcontext

import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils.checkpoint import checkpoint
from torch.utils.data import Dataset, DataLoader

ROOT = Path(__file__).resolve().parent
HOUR = 3_600_000_000_000

def digest(path):
    h = hashlib.sha256()
    with Path(path).open("rb") as f:
        for block in iter(lambda: f.read(8 * 1024**2), b""):
            h.update(block)
    return h.hexdigest()


def atomic_json(path, data):
    path = Path(path)
    tmp = path.with_suffix(path.suffix + ".tmp")
    tmp.write_text(json.dumps(data, indent=2, allow_nan=False) + "\n")
    tmp.replace(path)


def atomic_checkpoint(path, data):
    path = Path(path)
    tmp = path.with_suffix(".tmp")
    with tmp.open("wb") as f:
        torch.save(data, f)
        f.flush()
        os.fsync(f.fileno())
    tmp.replace(path)


def device_for(name):
    if name == "auto":
        name = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu"
    if name == "cuda" and not torch.cuda.is_available():
        raise RuntimeError("CUDA unavailable in this Python. Install a CUDA PyTorch wheel before training.")
    if name == "mps" and not torch.backends.mps.is_available():
        raise RuntimeError("MPS requested but unavailable")
    return torch.device(name)


"""Embedded in the standalone v164 trainer: coarse archive plus native MSLP patches."""
REUSE_ARCH = 'wp-coarse8-native-pressure1-v164'
REUSE_CHANNELS = ['mslp', 'hgt500', 'uwnd850', 'vwnd850', 'uwnd500', 'vwnd500', 'uwnd200', 'vwnd200']


def load_reuse(path):
    path = Path(path)
    if not path.is_file():
        raise FileNotFoundError('Combined dataset missing. Run the updated Drive setup cell first: '+str(path))
    m = json.loads(path.read_text())
    if m.get('schema') != 'v164-reuse-pressure-v1' or m.get('pilot', True):
        raise ValueError('Combined non-pilot archive required; no pilot fallback')
    for name, entry in m['files'].items():
        p = path.parent/entry['path']
        print('Checking dataset file:', name, flush=True)
        if digest(p) != entry['sha256']:
            raise ValueError('Dataset checksum mismatch: '+name)
        entry['resolved'] = str(p)
    with np.load(m['files']['basin']['resolved'], allow_pickle=False) as z:
        if z['channels'].tolist() != REUSE_CHANNELS[1:]:
            raise ValueError('Coarse channel order mismatch')
        times, lat, lon = z['time'], z['lat'], z['lon']
```

```
        q = z['q']
        scale, offset = z['scale'], z['offset']
    if q.shape != (len(times),7,25,33) or not np.all(np.diff(times) == 6*HOUR):
        raise ValueError('Coarse grid/time mismatch')
    qpath = path.parent/'basin_q_verified.npy'
    # Refresh the compact worker mmap from the checksum-verified source every preflight.
    temporary = qpath.with_suffix('.pending.npy')
    np.save(temporary, q)
    os.replace(temporary, qpath)
    slp = np.load(m['files']['slp']['resolved'], mmap_mode='r')
    if slp.shape != (len(times),25,33) or not np.isfinite(slp).all():
        raise ValueError('Coarse pressure coverage invalid')
    with np.load(m['files']['track']['resolved'], allow_pickle=False) as z:
        track = {k:z[k] for k in ['base_time','storm_id','base_lat','base_lon','target','target_mask']}
        issue_intensity = z['track'][:,-1,4:6]*z['track_std'][4:6]+z['track_mean'][4:6]
    plan = json.loads(Path(m['files']['plan']['resolved']).read_text())
    if len(plan['rows']) != 1000 or plan['shape'] != [1000,29,121,121] or plan['lead_hours'] !=
list(range(-48,121,6)):
        raise ValueError('Expected the verified 1000-window native pressure package, not the six-sample pilot')
    pressure = np.load(m['files']['pressure']['resolved'], mmap_mode='r')
    if pressure.shape != (1000,29,121,121):
        raise ValueError('Native pressure shape mismatch')
    patches = {}
    for pi,row in enumerate(plan['rows']):
        ti = row['track_archive_row']
        if int(track['base_time'][ti]) != row['issue_ns'] or str(track['storm_id'][ti]) != row['storm_id']:
            raise ValueError('Native patch/track identity mismatch')
        patches[ti] = pi
    first_year = {}
    years = track['base_time'].astype('datetime64[ns]').astype('datetime64[Y]').astype(int)+1970
    for sid,y in zip(track['storm_id'],years):
        first_year[str(sid)] = min(first_year.get(str(sid),9999),int(y))
    lookup = {(str(s),int(t)):i for i,(s,t) in enumerate(zip(track['storm_id'],track['base_time']))}
    rows = []
    for i,ns in enumerate(track['base_time']):
        sid = str(track['storm_id'][i])
        y = first_year[sid]
        split = 'train' if 2000 <= y <= 2021 else 'validation' if 2022 <= y <= 2023 else 'test' if 2024 <= y <=
2025 else None
        if split is None:
            continue
        low,high = {'train':(2000,2021),'validation':(2022,2023),'test':(2024,2025)}[split]
        bounds = np.array([ns-48*HOUR,ns+120*HOUR],dtype='int64').astype('datetime64[ns]').astype('datetime64[Y]
').astype(int)+1970
        ai = int(np.searchsorted(times,ns))
        center = [float(track['base_lat'][i]),float(track['base_lon'][i])%360]
        if bounds[0] < low or bounds[1] > high or ai < 8 or ai+20 >= len(times) or times[ai] != ns:
            continue
        if not np.isfinite(center).all() or not (0 < center[0] < 60 and 100 < center[1] < 180):
            continue
        previous = lookup.get((sid,int(ns-6*HOUR)))
        motion = [0.,0.]
        if previous is not None:
            old = [float(track['base_lat'][previous]),float(track['base_lon'][previous])%360]
            if np.isfinite(old).all():
                motion = [(center[1]-old[1])*111.2*np.cos(np.deg2rad(center[0])),(center[0]-old[0])*111.2]
        future = [lookup.get((sid,int(ns+j*6*HOUR))) for j in range(1,21)]
        target_centers = [[float(track['base_lat'][j]),float(track['base_lon'][j])%360] if j is not None else
[0.,0.] for j in future]
        rm = [float(j is not None and np.isfinite(target_centers[k]).all()) for k,j in enumerate(future)]
        target_centers = np.nan_to_num(target_centers).tolist()
        wind,p = track['target'][i,:,2],track['target'][i,:,3]
        wm = track['target_mask'][i,:,2].astype(bool)&np.isfinite(wind)&(wind>=0)&(wind<250)
        pm = track['target_mask'][i,:,3].astype(bool)&np.isfinite(p)&(p>800)&(p<1100)
        pi = patches.get(i,-1)
        if pi >= 0 and plan['rows'][pi]['split'] != split:
            raise ValueError('Patch split mismatch')
        issue = issue_intensity[i]
        iv = [bool(np.isfinite(issue[0]) and 0<=issue[0]<250),bool(np.isfinite(issue[1]) and 800<issue[1]<1100)]
        rows.append({'split':split,'storm_id':sid,'atlas':ai,'patch':pi,'center':center,'motion':motion,
                     'issue_intensity':[float(issue[k]) if iv[k] else 0. for k in range(2)],'issue_mask':iv,
                     'target_centers':target_centers,'route_mask':rm,
                     'pressure':np.where(pm,p,0).tolist(),'pressure_mask':pm.tolist(),
                     'vmax':np.where(wm,wind,0).tolist(),'vmax_mask':wm.tolist()})
    counts = {s:sum(r['split']==s for r in rows) for s in ['train','validation','test']}
    patch_counts = {s:sum(r['split']==s and r['patch']>=0 for r in rows) for s in counts}
    if counts['train'] < 10000 or patch_counts != {'train':800,'validation':100,'test':100}:
        raise ValueError(f'Incomplete combined coverage: {counts}, native patches {patch_counts}')
    train_times =
np.flatnonzero((times.astype('datetime64[ns]').astype('datetime64[Y]').astype(int)+1970>=2000)&
                               (times.astype('datetime64[ns]').astype('datetime64[Y]').astype(int)+1970<=2021))
    sums,squares = np.zeros(8),np.zeros(8)
    n = 0
    for start in range(0,len(train_times),256):
        ii = train_times[start:start+256]
        b = np.concatenate([np.asarray(slp[ii],dtype='float32')[:,None],q[ii].astype('float32')*scale[None,:,Non
```

```
    e,None]+offset[None,:,None,None]],1).astype('float64')
164         if not np.isfinite(b).all():
165             raise ValueError('Nonfinite coarse training fields')
166         sums += b.sum((0,2,3)); squares += (b*b).sum((0,2,3)); n += len(ii)*25*33
167     mean = sums/n
168     std = np.maximum(np.sqrt(np.maximum(squares/n-mean*mean,0)),1e-4)
169     m.update(rows=rows, plan=plan, qpath=str(qpath), scale=scale.tolist(), offset=offset.tolist(),
170              channels=REUSE_CHANNELS,units=['hPa','m','m/s','m/s','m/s','m/s','m/s','m/s'],
171              global_lat=lat.tolist(),global_lon=lon.tolist(),regional_lat=list(np.linspace(15,-15,121)),
172              regional_lon=list(np.linspace(-15,15,121)),normalization={'mean':mean.tolist(),'std':std.tolist(),'
    fit_split':'train','end_year':2021},
173              identity=digest(path),counts=counts,patch_counts=patch_counts)
174     print(json.dumps({'all_archive_windows':counts,'native_pressure_windows':patch_counts,'architecture':REUSE_A
    RCH}),flush=True)
175     return m
176
177
178 class ReuseDataset(Dataset):
179     def __init__(self,m,split,training=False):
180         self.m = m
181         self.rows = [r for r in m['rows'] if r['split']==split]
182         self.arrays = None
183
184     def __len__(self):
185         return len(self.rows)
186
187     def __getitem__(self,i):
188         m,row = self.m,self.rows[i]
189         if self.arrays is None:
190             self.arrays = {k:np.load(m['files'][k]['resolved'],mmap_mode='r') for k in ['slp','pressure']}
191             self.arrays['q'] = np.load(m['qpath'],mmap_mode='r')
192             with np.load(m['files']['geography']['resolved'],allow_pickle=False) as geo:
193                 self.arrays['land'] = geo['land_fraction']
194                 self.arrays['height'] = geo['elevation_m']/8000
195         ai = row['atlas']
196         ii = slice(ai-8,ai+21)
197         coarse = np.concatenate([np.array(self.arrays['slp'][ii],dtype='float32')[:,None],
198               np.array(self.arrays['q'][ii],dtype='float32')*np.array(m['scale'],dtype='float32')[None,:,None,No
    ne]+np.array(m['offset'],dtype='float32')[None,:,None,None]],1)
199         mean,std =
    np.array(m['normalization']['mean'],dtype='float32'),np.array(m['normalization']['std'],dtype='float32')
200         coarse = (coarse-mean[None,:,None,None])/std[None,:,None,None]
201         pi = row['patch']
202         anchor = m['plan']['rows'][pi] if pi>=0 else
    {'center_lat':round(row['center'][0]*4)/4,'center_lon':round(row['center'][1]*4)/4}
203         lat = anchor['center_lat']+np.linspace(15,-15,121,dtype='float32')
204         lon = anchor['center_lon']+np.linspace(-15,15,121,dtype='float32')
205         yy,xx = np.meshgrid(lat,lon,indexing='ij')
206         y = np.rint((90-yy)*4).astype(int); x = np.rint(xx*4).astype(int)%1440
207         rs = np.stack([yy/90,xx/180-1,self.arrays['land'][y,x],self.arrays['height'][y,x]]).astype('float32')
208         cy,cx = np.meshgrid(m['global_lat'],m['global_lon'],indexing='ij')
209         iy = np.rint((90-cy)*4).astype(int); ix = np.rint(cx*4).astype(int)%1440
210         gs =
    np.stack([cy/90,cx/180-1,self.arrays['land'][iy,ix],self.arrays['height'][iy,ix]]).astype('float32')
211         detail = np.zeros((29,1,121,121),dtype='float32')
212         if pi>=0:
213             detail[:,0] = (np.array(self.arrays['pressure'][pi],dtype='float32')-mean[0])/std[0]
214             if not np.isfinite(detail).all():
215                 raise ValueError('Native pressure contains nonfinite fields')
216         inputs = {'global_history':coarse[:9], 'regional_history':detail[:9],
217                   'global_static':gs,'regional_static':rs,'detail_available':np.array([pi>=0],dtype='float32'),
218
    'center':np.array(row['center'],dtype='float32'),'motion':np.array(row['motion'],dtype='float32'),
219                   'issue_intensity':np.array(row['issue_intensity'],dtype='float32'),'issue_mask':np.array(row['
    issue_mask'],dtype='float32')}
220         targets = {'global':coarse[9:],'global_mask':np.ones_like(coarse[9:]),
221
    'regional_lat_weight':np.broadcast_to(np.cos(np.deg2rad(lat))[None,None,:,None],(20,1,121,1)).astype('float32'),
222                    'regional':detail[9:],'regional_mask':np.full_like(detail[9:],float(pi>=0)),
223                    'center':np.array(row['target_centers'],dtype='float32'),
224                    'storm_mask':np.array(row['route_mask'],dtype='float32')}
225         for k in ['pressure','pressure_mask','vmax','vmax_mask']:
226             targets[k] = np.array(row[k],dtype='float32')
227         return {k:torch.from_numpy(v.copy()) for k,v in inputs.items()},{k:torch.from_numpy(v.copy()) for k,v in
     targets.items()}
228
229
230 """v165 implementation, embedded with verified archive readers by the builder."""
231 VERSION = '1.2.65'
232 ARCHITECTURE = REUSE_ARCH = 'wp-causal-transport-evolution-field-tracker-v165'
233 TRAIN_HORIZONS = [4, 8, 12, 20]
234
235
236 def sample_field(field, grid, padding='border'):
237     # Geometry and interpolation stay FP32, including under CUDA autocast.
238     with torch.autocast(device_type=field.device.type, enabled=False):
```

```
        return F.grid_sample(field.float(), grid.float(), align_corners=True,
                             mode='bilinear', padding_mode=padding)


def identity_grid(field):
    h, w = field.shape[-2:]
    yy, xx = torch.meshgrid(torch.linspace(-1, 1, h, device=field.device),
                            torch.linspace(-1, 1, w, device=field.device), indexing='ij')
    return torch.stack([xx, yy], -1)[None].expand(field.shape[0], -1, -1, -1)


def departure_grid(field, velocity, latitude, spacing):
    # East/north m/s -> backward departure coordinates on a north-to-south grid.
    h, w = field.shape[-2:]
    u, v = velocity.float().unbind(1)
    dx = u * 21.6 / (111.2 * latitude.float().deg2rad().cos().clamp_min(.2) * spacing)
    dy = -v * 21.6 / (111.2 * spacing)
    return identity_grid(field) - torch.stack([2*dx/(w-1), 2*dy/(h-1)], -1)


class TransportBlock(nn.Module):
    def __init__(self, width):
        super().__init__()
        self.net = nn.Sequential(nn.Conv2d(width, width, 5, padding=2, groups=width),
                                 nn.GroupNorm(8, width), nn.Conv2d(width, width*4, 1),
                                 nn.GELU(), nn.Conv2d(width*4, width, 1))
        self.scale = nn.Parameter(torch.full((1, width, 1, 1), .01))

    def forward(self, x):
        return x + self.scale * self.net(x)


class EvolutionNet(nn.Module):
    def __init__(self, channels, outputs, base=72, tiny=False):
        super().__init__()
        widths = [base, base*2, base*4, base*6]
        depths = [1]*4 if tiny else [2, 2, 4, 4]
        self.stem = nn.Conv2d(channels, base, 3, padding=1)
        self.enc = nn.ModuleList(nn.Sequential(*[TransportBlock(w) for _ in range(d)])
                                 for w, d in zip(widths, depths))
        self.down = nn.ModuleList(nn.Conv2d(a, b, 3, stride=2, padding=1)
                                  for a, b in zip(widths, widths[1:]))
        self.up = nn.ModuleList(nn.Conv2d(widths[i+1], widths[i], 1) for i in (2, 1, 0))
        self.dec = nn.ModuleList(TransportBlock(widths[i]) for i in (2, 1, 0))
        self.out = nn.Conv2d(base, outputs, 1)
        self.features = nn.Conv2d(base, 32, 1)
        nn.init.zeros_(self.out.weight)
        nn.init.zeros_(self.out.bias)

    def block(self, module, x):
        return checkpoint(module, x, use_reentrant=False) if self.training and torch.is_grad_enabled() else
    module(x)

    def forward(self, x):
        x = self.stem(x)
        skips = []
        for i, layer in enumerate(self.enc):
            x = self.block(layer, x)
            skips.append(x)
            if i < 3:
                x = self.down[i](x)
        for up, layer, skip in zip(self.up, self.dec, reversed(skips[:-1])):
            x = F.interpolate(up(x), size=skip.shape[-2:], mode='bilinear', align_corners=False)+skip
            x = self.block(layer, x)
        return self.out(x).float(), self.features(x).float()


class RecurrentFeatures(nn.Module):
    def __init__(self, history_channels):
        super().__init__()
        self.encode = nn.Sequential(nn.Conv2d(history_channels, 32, 3, padding=1),
                                    nn.GELU(), TransportBlock(32))
        self.gates = nn.Conv2d(64, 64, 3, padding=1)
        self.candidate = nn.Conv2d(64, 32, 3, padding=1)

    def forward(self, old, features):
        reset, update = torch.sigmoid(self.gates(torch.cat([old, features], 1))).chunk(2, 1)
        proposal = torch.tanh(self.candidate(torch.cat([old*reset, features], 1)))
        return ((1-update)*old + update*proposal).float()


def track_field(pressure, lat, lon, prior, domain=None):
    """Associate a local pressure minimum; no independent learned route head."""
    lat0, lon0 = prior[:, 0, None, None], prior[:, 1, None, None]
    distance2 = ((lat-lat0)*111.2).square()+((lon-lon0)*111.2*lat0.deg2rad().cos()).square()
    candidate = distance2 <= 650**2
```

```
    if domain is not None:
        candidate = candidate & domain
    score = -pressure/3 - distance2/(2*650**2)
    peak = score.masked_fill(~candidate, -1e9).flatten(1).argmax(1)
    h, w = pressure.shape[-2:]
    y, x = torch.arange(h, device=pressure.device)[None, :, None], torch.arange(w, device=pressure.device)[None, None, :]
    local = ((y-(peak//w)[:, None, None]).abs() <= 1) & ((x-(peak%w)[:, None, None]).abs() <= 1) & candidate
    pdf = torch.softmax(score.masked_fill(~local, -1e9).flatten(1), 1).reshape_as(score)
    center = torch.stack([(pdf*lat).sum((1, 2)), (pdf*lon).sum((1, 2))], 1)
    grid = torch.stack([2*(center[:, 1]-lon[:, 0, 0])/(lon[:, 0, -1]-lon[:, 0, 0])-1,
                        2*(center[:, 0]-lat[:, 0, 0])/(lat[:, -1, 0]-lat[:, 0, 0])-1], -1)[:, None, None]
    central = sample_field(pressure[:, None], grid)[:, 0, 0, 0]
    annulus = (distance2 >= 300**2) & (distance2 <= 650**2)
    background = (pressure*annulus).sum((1, 2))/annulus.sum((1, 2)).clamp_min(1)
    interior = (peak//w > 0) & (peak//w < h-1) & (peak%w > 0) & (peak%w < w-1)
    valid = candidate.flatten(1).any(1) & interior & (background-central > 1.)
    return center, central, valid, background-central


class TransportForecaster(nn.Module):
    def __init__(self, manifest, tiny=False):
        super().__init__()
        base = 8 if tiny else 72
        self.coarse_memory, self.fine_memory = RecurrentFeatures(72), RecurrentFeatures(9)
        self.coarse = EvolutionNet(8+32+4, 8+2, base, tiny)
        self.fine = EvolutionNet(1+8+32+4+1, 1+2, base, tiny)
        self.steering_logits = nn.Parameter(torch.tensor([.4, .4, .2]).log())
        # Wind intensity is explicitly a scalar estimate, not a resolved 10-m wind map.
        self.wind = nn.Sequential(nn.Linear(32+2, 64), nn.GELU(), nn.Linear(64, 1))
        self.register_buffer('mean', torch.tensor(manifest['normalization']['mean'], dtype=torch.float32)[None, :, None, None])
        self.register_buffer('std', torch.tensor(manifest['normalization']['std'], dtype=torch.float32)[None, :, None, None])
        yy, xx = np.meshgrid(manifest['global_lat'], manifest['global_lon'], indexing='ij')
        self.register_buffer('lat', torch.tensor(yy, dtype=torch.float32)[None])
        self.register_buffer('lon', torch.tensor(xx, dtype=torch.float32)[None])

    def initial(self, inputs):
        required = {'global_history', 'regional_history', 'global_static', 'regional_static',
                    'detail_available', 'center', 'motion', 'issue_intensity', 'issue_mask'}
        if set(inputs) != required:
            raise ValueError('Only declared issue-time and historical inputs are allowed')
        gh, rh, rs = inputs['global_history'], inputs['regional_history'], inputs['regional_static']
        grid = torch.stack([((rs[:, 1]+1)*180-100)/40-1, 1-rs[:, 0]*3], -1)
        parent = sample_field(gh[:, -1, :1], grid)
        available = inputs['detail_available'][:, :, None, None]
        r = torch.where(available.bool(), rh[:, -1], parent)
        return dict(g=gh[:, -1], r=r, gh=self.coarse_memory.encode(gh.flatten(1, 2)).float(),
                    rh=self.fine_memory.encode(rh.flatten(1, 2)).float(), gs=inputs['global_static'], rs=rs,
                    grid=grid, available=available, anchor=(r-parent)*available,
                    coordinates=identity_grid(r).permute(0, 3, 1, 2), source=torch.zeros_like(r),
                    center=inputs['center'], motion=inputs['motion'], issue=inputs['issue_intensity'],
                    issue_mask=inputs['issue_mask'])

    def steering(self, g):
        physical = g.float()*self.std+self.mean
        winds = physical[:, 2:].reshape(g.shape[0], 3, 2, *g.shape[-2:])
        return (winds*torch.softmax(self.steering_logits, 0)[None, :, None, None, None]).sum(1)

    def step(self, s):
        delta, features = self.coarse(torch.cat([s['g'], s['gh'], s['gs']], 1))
        velocity = self.steering(s['g']) + 15*torch.tanh(delta[:, 8:])
        departure = departure_grid(s['g'], velocity, self.lat, 2.5)
        g = sample_field(s['g'], departure) + .2*torch.tanh(delta[:, :8])
        gh = self.coarse_memory(sample_field(s['gh'], departure), features)
        parent = sample_field(g, s['grid'])
        fine_velocity = sample_field(velocity, s['grid'])
        r, rh, coordinates, source = s['r'], s['rh'], s['coordinates'], s['source']
        parent_valid = (s['grid'].abs() <= 1).all(-1)[:, None].float()
        if bool(s['available'].any()):
            rd, rf = self.fine(torch.cat([r, parent, rh, s['rs'], parent_valid], 1))
            fine_velocity = fine_velocity + 10*torch.tanh(rd[:, 1:])
            dep = departure_grid(r, fine_velocity, s['rs'][:, 0]*90, .25)
            # Compose material coordinates and sample the ORIGINAL anomaly once per lead.
            # This avoids repeatedly interpolating away its core. Learned source terms
            # can strengthen, weaken or deform it; survival is not hard-coded.
            displacement = coordinates-identity_grid(r).permute(0, 3, 1, 2)
            coordinates = dep.permute(0, 3, 1, 2)+sample_field(displacement, dep)
            source = sample_field(source, dep, 'zeros') + .2*torch.tanh(rd[:, :1])
            anomaly = sample_field(s['anchor'], coordinates.permute(0, 2, 3, 1), 'zeros')+source
            proposed = parent[:, :1]+anomaly*parent_valid
            r = torch.where(s['available'].bool(), proposed, parent[:, :1])
            rh = self.fine_memory(sample_field(rh, dep), rf)
        else:
            r = parent[:, :1]
```

```
        # Association prior uses only predicted circulation, never a future center.
        at = torch.stack([(s['center'][:, 1]-100)/40-1, 1-s['center'][:, 0]/30], -1)[:, None, None]
        steering = sample_field(velocity, at)[:, :, 0, 0]
        displacement_km = .7*steering*21.6 + .3*s['motion']
        prior = s['center']+torch.stack([displacement_km[:, 1]/111.2,
                    displacement_km[:, 0]/(111.2*s['center'][:, 0].deg2rad().cos().clamp_min(.2))], 1)
        gp, rp = (g*self.std+self.mean)[:, 0], (r*self.std[:, :1]+self.mean[:, :1])[:, 0]
        coarse_center, coarse_p, coarse_ok, coarse_depth = track_field(gp, self.lat, self.lon, prior)
        fine_lat, fine_lon = s['rs'][:, 0]*90, (s['rs'][:, 1]+1)*180
        fine_center, fine_p, fine_ok, fine_depth = track_field(rp, fine_lat, fine_lon, prior, parent_valid[:, 0].bool())
        inside = (prior[:, 0] < fine_lat[:, 0, 0]) & (prior[:, 0] > fine_lat[:, -1, 0]) & \
                 (prior[:, 1] > fine_lon[:, 0, 0]) & (prior[:, 1] < fine_lon[:, 0, -1])
        inside = inside & (prior[:, 0] >= 0) & (prior[:, 0] <= 60) & (prior[:, 1] >= 100) & (prior[:, 1] <= 180)
        use_fine = s['available'][:, 0, 0, 0].bool() & inside
        center = torch.where(use_fine[:, None], fine_center, coarse_center)
        pressure = torch.where(use_fine, fine_p, coarse_p)
        ok = torch.where(use_fine, fine_ok, coarse_ok)
        wind_features = torch.cat([gh.mean((2, 3)), s['issue'][:, :1]/100, s['issue_mask'][:, :1]], 1)
        vmax = F.softplus(self.wind(wind_features)[:, 0])*30
        motion = torch.stack([(center[:, 1]-s['center'][:, 1])*111.2*((center[:, 0]+s['center'][:, 0])/2).deg2rad().cos(),
                              (center[:, 0]-s['center'][:, 0])*111.2], 1)
        out = dict(global_=g, regional=r, center=center, pressure=pressure, grid_pressure=pressure,
                   vmax=vmax, track_valid=ok, native_route=use_fine, regional_valid=parent_valid.bool(),
                   pressure_depth=torch.where(use_fine, fine_depth, coarse_depth),
                   global_velocity=velocity, regional_velocity=fine_velocity)
        out['global'] = out.pop('global_')
        return dict(s, g=g, r=r, gh=gh, rh=rh, coordinates=coordinates, source=source,
                    center=center, motion=motion), out

    def forward(self, inputs, steps=20):
        state, outputs = self.initial(inputs), []
        for _ in range(steps):
            state, output = self.step(state)
            outputs.append(output)
        return {k: torch.stack([o[k] for o in outputs], 1) for k in outputs[0]}


def weighted_mean(value, weight):
    return (value*weight).sum()/weight.expand_as(value).sum().clamp_min(1)


def objective165(output, target, inputs, manifest):
    terms = {}
    for key in ('global', 'regional'):
        pred, truth, mask = output[key].float(), target[key].float(), target[key+'_mask'].float()
        if key == 'regional':
            mask = mask*output['regional_valid']
        latitude = inputs['global_static' if key == 'global' else 'regional_static'][:, :1]*90
        area = latitude.deg2rad().cos().clamp_min(0)*mask
        terms[key] = weighted_mean(F.smooth_l1_loss(pred, truth, reduction='none'), area)
        for axis in (-2, -1):
            a, b = pred.diff(dim=axis), truth.diff(dim=axis)
            w = area.narrow(axis, 0, a.shape[axis])
            terms[key] = terms[key]+.2*weighted_mean((a-b).abs(), w)
    # Labels localize supervision only. They are never passed to initial()/step().
    lat, lon = inputs['regional_static'][:, 0]*90, (inputs['regional_static'][:, 1]+1)*180
    ct = target['center']
    dist2 = ((lat-ct[:, 0, None, None])*111.2).square()+ \
            ((lon-ct[:, 1, None, None])*111.2*ct[:, 0, None, None].deg2rad().cos()).square()
    region = (dist2 < 600**2)[:, None]*target['regional_mask']*output['regional_valid']*target['storm_mask'][:, None, None, None]
    p, truth = output['regional'].float(), target['regional'].float()
    terms['storm_field'] = weighted_mean((p-truth).abs(), region)
    # Compare resolved native pressure minima against the SAME resolved target field,
    # not subgrid best-track pressure (e.g. Tip's 875 hPa is absent in this archive).
    local = region.flatten(1).bool()
    pmin = p.flatten(1).masked_fill(~local, 1e4).min(1).values
    tmin = truth.flatten(1).masked_fill(~local, 1e4).min(1).values
    terms['resolved_core'] = weighted_mean((pmin-tmin).abs(), local.any(1).float())
    # Radial-band pressure contrasts retain size/asymmetry information without
    # imposing an idealized vortex or forcing a storm to survive.
    terms['radial_structure'] = 0
    for low, high in ((0, 150), (150, 300), (300, 600)):
        band = ((dist2 >= low**2) & (dist2 < high**2))[:, None]*region
        den = band.sum((1, 2, 3)).clamp_min(1)
        contrast = ((p-truth)*band).sum((1, 2, 3))/den
        terms['radial_structure'] = terms['radial_structure']+weighted_mean(contrast.abs(), (den > 1).float())/3
    displacement = (output['center']-ct)*torch.stack([torch.ones_like(ct[:, 0])*111.2,
                                  111.2*ct[:, 0].deg2rad().cos()], 1)
    terms['route'] = weighted_mean(F.smooth_l1_loss(displacement/100, torch.zeros_like(displacement), reduction='none'),
                                   target['storm_mask'][:, None])
    terms['wind'] = weighted_mean(F.smooth_l1_loss(output['vmax']/30, target['vmax']/30, reduction='none'), target['vmax_mask'])
```

```
    terms['flow_smooth'] = sum(v.diff(dim=a).abs().mean()/30 for v in
                    [output['global_velocity'], output['regional_velocity']] for a in [-2, -1])
    loss = terms['global']+terms['regional']+.5*terms['storm_field']+.2*terms['resolved_core']+ \
           .1*terms['radial_structure']+.1*terms['route']+.05*terms['wind']+.002*terms['flow_smooth']
    return loss, terms


def training_indices(rows, epoch):
    """Every coarse window once, plus storm-balanced native replays (~25% native)."""
    rng = np.random.default_rng(165000+epoch)
    indices = list(range(len(rows)))
    groups = {}
    for i, row in enumerate(rows):
        if row['patch'] >= 0:
            groups.setdefault(row['storm_id'], []).append(i)
    native = sum(len(v) for v in groups.values())
    extra = max(0, math.ceil((.25*len(rows)-native)/.75))
    keys = sorted(groups)
    for j in range(extra):
        # Round-robin storms, random window within each storm.
        indices.append(int(rng.choice(groups[keys[j % len(keys)]])))
    rng.shuffle(indices)
    return indices


def save_training_state(path, model, optimizer, scaler, manifest, epoch, history, best):
    atomic_checkpoint(path, dict(version=VERSION, architecture=ARCHITECTURE,
dataset_sha256=manifest['identity'],
        data_contract={k:manifest[k] for k in ('channels', 'units', 'global_lat', 'global_lon', 'regional_lat',
'regional_lon', 'normalization')},
        epoch=epoch, model=model.state_dict(), optimizer=optimizer.state_dict(), scaler=scaler.state_dict(),
        history=history, best_value=best, python_rng=random.getstate(), numpy_rng=np.random.get_state(),
        torch_rng=torch.get_rng_state(), cuda_rng=torch.cuda.get_rng_state_all() if torch.cuda.is_available()
else None))


def restore_training_state(path, model, optimizer, scaler, manifest):
    saved = torch.load(path, map_location='cpu', weights_only=False)
    if saved['architecture'] != ARCHITECTURE or saved['dataset_sha256'] != manifest['identity']:
        raise ValueError('Resume requires the identical v165 architecture and dataset; v164 is not compatible')
    model.load_state_dict(saved['model'])
    optimizer.load_state_dict(saved['optimizer'])
    scaler.load_state_dict(saved['scaler'])
    random.setstate(saved['python_rng']); np.random.set_state(saved['numpy_rng']);
torch.set_rng_state(saved['torch_rng'])
    if torch.cuda.is_available() and saved.get('cuda_rng') is not None:
        torch.cuda.set_rng_state_all(saved['cuda_rng'])
    return saved['epoch']+1, saved['history'], saved['best_value']


def train165(model, m, args, device):
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=.01)
    scaler = torch.amp.GradScaler('cuda', enabled=device.type == 'cuda' and bool(args.amp))
    last, best_path = args.output/'last.pt', args.output/'best.pt'
    start, history, best = 1, [], float('inf')
    if last.is_file():
        start, history, best = restore_training_state(last, model, optimizer, scaler, m)
        print('Resuming completed epoch', start-1, flush=True)
    atomic_json(args.output/'manifest.json', dict(version=VERSION, architecture=ARCHITECTURE,
        parameters=sum(p.numel() for p in model.parameters()), dataset_sha256=m['identity'],
        script_sha256=digest(__file__), samples=m['counts'], native_samples=m['patch_counts'],
        future_weather_as_inputs=False, official_forecasts_as_inputs=False, input_hours=list(range(-48, 1, 6)),
        forecast_hours=list(range(6, 121, 6)), pressure='resolved field at the reported field-derived center',
        native_sampling='all coarse windows plus storm-balanced native replays to 25 percent',
        validation='fixed 120h joint loss, balanced native/coarse strata; no test-set selection',
        physical_microbatch=1, effective_batch=args.batch, tbptt_steps=4,
        limitations=['regional WP, not global', 'native MSLP only on 1000 windows',
                     'no fine upper-air variables', 'not an exact conservation solver', 'Tip is a development
case']))
    for epoch in range(start, args.epochs+1):
        horizon = TRAIN_HORIZONS[min(epoch-1, len(TRAIN_HORIZONS)-1)]
        metrics = {}
        for split in ('train', 'validation'):
            training = split == 'train'
            model.train(training)
            data = ReuseDataset(m, split)
            sampler = training_indices(data.rows, epoch) if training else None
            loader = DataLoader(data, batch_size=1, sampler=sampler, num_workers=min(args.workers, 2),
                                **({'prefetch_factor':1} if args.workers else {}))
            groups, failures = {False:[], True:[]}, []
            optimizer.zero_grad(set_to_none=True)
            began = time.time()
            for index, (inputs, targets) in enumerate(loader, 1):
                inputs = {k:v.to(device) for k, v in inputs.items()}
                steps = horizon if training else 20
                total, block_loss = 0., 0
```

```
                with torch.set_grad_enabled(training):
                    state = model.initial(inputs)
                    for step in range(steps):
                        with torch.autocast('cuda', dtype=torch.float16) if scaler.is_enabled() else nullcontext():
                            state, output = model.step(state)
                        frame = {k:v[:, step].to(device) for k, v in targets.items()}
                        loss, terms = objective165(output, frame, inputs, m)
                        if not torch.isfinite(loss):
                            raise RuntimeError(f'Nonfinite loss epoch={epoch} sample={index} step={step}; no checkpoint advanced')
                        total += float(loss.detach())/steps
                        failures.append(float((~output['track_valid']).float().mean()))
                        if training:
                            block_loss = block_loss+loss/(steps*args.batch)
                            if (step+1) % 4 == 0 or step+1 == steps:
                                scaler.scale(block_loss).backward()
                                state = {k:v.detach() for k, v in state.items()}
                                block_loss = 0
                    if training and (index % args.batch == 0 or index == len(loader)):
                        scaler.unscale_(optimizer)
                        remainder = index % args.batch
                        if index == len(loader) and remainder:
                            for parameter in model.parameters():
                                if parameter.grad is not None:
                                    parameter.grad.mul_(args.batch/remainder)
                        norm = nn.utils.clip_grad_norm_(model.parameters(), 1.)
                        if not torch.isfinite(norm):
                            raise RuntimeError('Nonfinite gradient; checkpoint unchanged. Retry with AMP=0.')
                        scaler.step(optimizer); scaler.update(); optimizer.zero_grad(set_to_none=True)
                groups[bool(inputs['detail_available'][0, 0])].append(total)
                if index == 1 or index % 10 == 0:
                    print(f'{split} epoch={epoch}/{args.epochs} sample={index}/{len(loader)} horizon={steps} loss={total:.5f} elapsed_s={time.time()-began:.0f}', flush=True)
            # Keep rare native examples visible in selection, rather than 5.7% dilution.
            metrics[split+'_native'] = float(np.mean(groups[True]))
            metrics[split+'_coarse'] = float(np.mean(groups[False]))
            metrics[split] = (metrics[split+'_native']+metrics[split+'_coarse'])/2
            metrics[split+'_association_failure_fraction'] = float(np.mean(failures))
        promote = metrics['validation'] < best
        if promote:
            best = metrics['validation']
        history.append(dict(epoch=epoch, horizon=horizon, **metrics))
        save_training_state(last, model, optimizer, scaler, m, epoch, history, best)
        save_training_state(args.output/f'epoch_{epoch:03d}.pt', model, optimizer, scaler, m, epoch, history, best)
        if promote:
            save_training_state(best_path, model, optimizer, scaler, m, epoch, history, best)
        atomic_json(args.output/'history.json', history)
        print(f'Saved epoch {epoch} directly to {args.output}; best={promote}', flush=True)
    print('Training complete. Test set untouched; best.pt selected by full-120h validation.', flush=True)


def main165():
    parser = argparse.ArgumentParser(description='v1.2.65 causal pressure transport and evolution; Drive-only data')
    parser.add_argument('batch', type=int, nargs='?', default=10)
    parser.add_argument('workers', type=int, nargs='?', default=3)
    parser.add_argument('amp', type=int, nargs='?', default=1, choices=[0, 1])
    parser.add_argument('--manifest', type=Path, default=ROOT/'data/v164_reuse/manifest.json')
    persistent = Path('/content/drive/MyDrive/typhoon_predict') if Path('/content').is_dir() else ROOT
    parser.add_argument('--output', type=Path, default=persistent/'training/v165_transport_pressure')
    parser.add_argument('--epochs', type=int, default=12)
    parser.add_argument('--device', choices=['auto', 'cpu', 'mps', 'cuda'], default='auto')
    parser.add_argument('--preflight', action='store_true')
    args = parser.parse_args()
    if args.batch < 1 or args.workers < 0 or args.epochs < 1:
        parser.error('Invalid batch, workers or epochs')
    if Path('/content').is_dir() and (not (persistent.parent).is_dir() or not args.output.resolve().is_relative_to(persistent.resolve())):
        raise ValueError('Mount Google Drive first; Colab checkpoints must persist under MyDrive/typhoon_predict')
    manifest = load_reuse(args.manifest)
    for split in ('train', 'validation', 'test'):
        data = ReuseDataset(manifest, split)
        data[0]; data[next(i for i, r in enumerate(data.rows) if r['patch'] >= 0)]
    if args.preflight:
        print('PASS full existing archive checksums, splits and native samples. No training started.', flush=True)
        return
    device = device_for(args.device)
    random.seed(165); np.random.seed(165); torch.manual_seed(165)
    model = TransportForecaster(manifest).to(device)
    print(json.dumps(dict(version=VERSION, architecture=ARCHITECTURE, parameters=sum(p.numel() for p in model.parameters()),
        device=str(device), samples=manifest['counts'], native_samples=manifest['patch_counts'],
```

```
     output=str(args.output))), flush=True)
646      args.output.mkdir(parents=True, exist_ok=True)
647      lock = args.output/'training.lock'
648      fd = os.open(lock, os.O_CREAT | os.O_EXCL | os.O_WRONLY)
649      os.write(fd, json.dumps(dict(pid=os.getpid(), host=socket.gethostname())).encode()); os.close(fd)
650      try:
651          train165(model, manifest, args, device)
652      finally:
653          lock.unlink(missing_ok=True)
654
655
656  if __name__ == '__main__':
657      main165()
```

B06. training/trackformer_1_2/source/v173/v173_pressure_train.py

11,200 bytes | SHA-256: f40691cec3e787f4ac3c189f647efb0953c1eaa7e03fc86edaf4fee06990b2be

```
"""Full-dataset, resumable Metal training for the v168 moving-core prototype."""
import argparse
import fcntl
import importlib.util
import json
import os
from pathlib import Path
import random
import signal
import socket
import time
import sys
import numpy as np
import torch
from torch.utils.data import DataLoader
from model import CoreForecaster, objective, base, VERSION, ARCHITECTURE

ROOT=Path(__file__).resolve().parents[1]
HORIZONS=[4,8,12,20]


class Tee:
    def __init__(self, terminal, log): self.terminal,self.log=terminal,log
    def write(self, value):
        self.terminal.write(value);self.log.write(value);self.log.flush()
    def flush(self): self.terminal.flush();self.log.flush()


def batches(indices, rows, effective, physical):
    for start in range(0,len(indices),effective):
        window=indices[start:start+effective];parts=[]
        for native in (True,False):
            group=[i for i in window if (rows[i]['patch']>=0)==native]
            parts.extend(group[j:j+physical] for j in range(0,len(group),physical))
        yield parts,len(window)


def save(path,model,optimizer,m,epoch,history,best,progress=None):
    base.atomic_checkpoint(path,dict(version=VERSION,architecture=ARCHITECTURE,
        dataset_sha256=m['identity'],model=model.state_dict(),optimizer=optimizer.state_dict(),
        data_contract={k:m[k] for k in
('channels','units','global_lat','global_lon','regional_lat','regional_lon','normalization')},
        epoch=epoch,history=history,best=best,progress=progress,
        python_rng=random.getstate(),numpy_rng=np.random.get_state(),torch_rng=torch.get_rng_state()))


def restore(path,model,optimizer,m):
    p=torch.load(path,map_location='cpu',weights_only=False)
    if p['architecture']!=ARCHITECTURE or p['dataset_sha256']!=m['identity']:
        raise ValueError('Architecture/dataset mismatch')
    model.load_state_dict(p['model'],strict=True);optimizer.load_state_dict(p['optimizer'])
    random.setstate(p['python_rng']);np.random.set_state(p['numpy_rng']);torch.set_rng_state(p['torch_rng'])
    return p


def warm_start(model,path,m):
    if base.digest(path)!='99ed776097e05ad6df03375465d9618741bed63a4ee57724d3ba98e295039d30':
        raise ValueError('Expected the verified v168 epoch 3 baseline checkpoint')
    p=torch.load(path,map_location='cpu',weights_only=False)
    if p['version']!='1.2.68' or p['epoch']!=3 or p['dataset_sha256']!=m['identity']:
        raise ValueError('Warm-start identity mismatch')
    weights=p['model']
    missing,extra=model.load_state_dict(weights,strict=False)
    if extra or any(not k.startswith('context.') for k in missing):
        raise ValueError('Incomplete environmental transfer')


def main():
    p=argparse.ArgumentParser()
    for name,default in
[('batch',10),('workers',2),('amp',0)]:p.add_argument(name,type=int,nargs='?',default=default)
    p.add_argument('--microbatch',type=int,default=2)
    p.add_argument('--epochs',type=int,default=12)
    p.add_argument('--device',default='cuda' if torch.cuda.is_available() else
'mps',choices=['mps','cuda','cpu'])
    here=Path(__file__).resolve().parent
    colab=Path('/content/drive/MyDrive/typhoon_predict')
    p.add_argument('--manifest',type=Path,default=(here if colab.exists() else
ROOT)/'data/v164_reuse/manifest.json')
    p.add_argument('--output',type=Path,default=colab/'training/v173' if colab.exists() else here/'runs/mac')
    p.add_argument('--warm-start',type=Path,default=here/'baseline_epoch_003.pt' if colab.exists() else
ROOT/'v168/runs/mac/epoch_003.pt')
```

```
    p.add_argument('--preflight',action='store_true')
    a=p.parse_args()
    if min(a.batch,a.microbatch,a.epochs)<1 or a.workers<0 or a.amp not in (0,1):p.error('Positive batches/epochs required; amp must be 0 or 1')
    if a.amp: print('AMP request recorded; pressure dynamics use FP32 for numerical stability.',flush=True)
    if a.preflight:
        m=base.load_reuse(a.manifest)
        assert m['counts']==dict(train=13949,validation=1041,test=1195)
        assert m['patch_counts']==dict(train=800,validation=100,test=100)
        model=CoreForecaster(m);warm_start(model,a.warm_start,m)
        print('Verified full dataset and v168 E3 warm start; parameters=',sum(p.numel() for p in model.parameters()))
        return
    torch.set_num_threads(4);random.seed(168);np.random.seed(168);torch.manual_seed(168)
    base.EvolutionNet.block=lambda self,module,x:module(x)
    device=base.device_for(a.device);a.output.mkdir(parents=True,exist_ok=True)
    handle=(a.output/'training.lock').open('a+')
    try:fcntl.flock(handle,fcntl.LOCK_EX|fcntl.LOCK_NB)
    except BlockingIOError:raise RuntimeError('Training already active for this output')
    handle.seek(0);handle.truncate();json.dump(dict(pid=os.getpid(),host=socket.gethostname()),handle);handle.flush()
    log=(a.output/'train.log').open('a',buffering=1)
    sys.stdout=Tee(sys.stdout,log);sys.stderr=Tee(sys.stderr,log)
    stop=[False]
    for sig in (signal.SIGTERM,signal.SIGINT):signal.signal(sig,lambda *_:stop.__setitem__(0,True))
    m=base.load_reuse(a.manifest)
    assert m['counts']==dict(train=13949,validation=1041,test=1195)
    assert m['patch_counts']==dict(train=800,validation=100,test=100)
    model=CoreForecaster(m).to(device)
    optimizer=torch.optim.AdamW(model.parameters(),lr=3e-5,weight_decay=.01)
    last=a.output/'last.pt';history=[];best=float('inf');start=1;progress=None
    if last.exists():
        saved=restore(last,model,optimizer,m)
        history,best,start,progress=saved['history'],saved['best'],saved['epoch']+1,saved['progress']
        if progress and progress['batch']!=a.batch:raise ValueError('Mid-epoch resume needs identical effective batch')
    else:warm_start(model,a.warm_start,m)
    manifest=dict(version=VERSION,architecture=ARCHITECTURE,parameters=sum(x.numel() for x in model.parameters()),
        samples=m['counts'],native_samples=m['patch_counts'],dataset_sha256=m['identity'],device=str(device),
        effective_batch=a.batch,microbatch=a.microbatch,precision='float32',future_weather_as_inputs=False,
        future_best_track_as_inputs=False,official_forecasts_as_inputs=False,
        pressure_map='moving learned core plus resolved environment; observation-conditioned initialization, not raw analysis',

internal_core_spacing_km=20,initial_core_prior_sigma_km=100,inner_absolute_reanalysis_loss_exclusion_km=150,
        core_structure_status='prototype; unresolved inner structure is not spatially validated',
        pressure_change_supervision=True,motion_change_supervision=True,
        environmental_attention='two spatial pooling scales, two transformer blocks, cross attention; zero initialized correction',
        route='pressure-field association; no independent displayed route or pressure head',
        warm_start=str(a.warm_start),warm_start_sha256=base.digest(a.warm_start),
        model_sha256=base.digest(Path(__file__).with_name('model.py')),trainer_sha256=base.digest(__file__))
    base.atomic_json(a.output/'manifest.json',manifest)
    print(json.dumps(manifest),flush=True)
    for epoch in range(start,a.epochs+1):
        metrics={}
        for split in ('train','validation'):
            training=split=='train';model.train(training);data=base.ReuseDataset(m,split)
            indices=base.training_indices(data.rows,epoch) if training else list(range(len(data)))
            windows=list(batches(indices,data.rows,a.batch,a.microbatch))
            offset=progress['window'] if training and progress else 0
            totals=progress['totals'] if training and progress else [[0.,0],[0.,0]]
            parts=[part for group,_ in windows[offset:] for part in group]
            loader=iter(DataLoader(data,batch_sampler=parts,num_workers=a.workers,**({'prefetch_factor':1} if a.workers else {})))
            began=time.time();steps=HORIZONS[min(epoch-1,3)] if training else 20
            for wi,(group,count) in enumerate(windows[offset:],offset):
                optimizer.zero_grad(set_to_none=True)
                for _ in group:
                    x,y=next(loader);x={k:v.to(device) for k,v in x.items()};size=x['center'].shape[0]
                    with torch.set_grad_enabled(training):
                        state=model.initial(x);block=0;total=0.
                        for lead in range(steps):
                            state,out=model.step(state);target={k:v[:,lead].to(device) for k,v in y.items()}
                            target['previous_pressure']=y['pressure'][:,lead-1].to(device) if lead else x['issue_intensity'][:,1]
                            target['previous_pressure_mask']=y['pressure_mask'][:,lead-1].to(device) if lead else x['issue_mask'][:,1]
                            target['previous_center']=y['center'][:,lead-1].to(device) if lead else x['center']
                            target['previous_storm_mask']=y['storm_mask'][:,lead-1].to(device) if lead else torch.ones_like(target['storm_mask'])
                            loss,terms=objective(out,target,x,m)
                            if not bool(torch.isfinite(loss)):raise RuntimeError('Nonfinite loss; previous checkpoint retained')
                            total+=float(loss.detach())/steps
```

```
                            if training:
                                block=block+loss*size/(count*steps)
                                if (lead+1)%4==0 or lead+1==steps:
                                    block.backward();block=0;state={k:v.detach() for k,v in state.items()}

native=int(bool(x['detail_available'][0,0]));totals[native][0]+=total*size;totals[native][1]+=size
                if training:
                    norm=torch.nn.utils.clip_grad_norm_(model.parameters(),1.)
                    if not bool(torch.isfinite(norm)):raise RuntimeError('Nonfinite gradient; previous
checkpoint retained')
                    optimizer.step()
                    if (wi+1)%50==0 or wi+1==len(windows) or stop[0]:

save(last,model,optimizer,m,epoch-1,history,best,dict(window=wi+1,totals=totals,batch=a.batch))
                if wi%5==0 or wi+1==len(windows):
                    print(f'{split} epoch={epoch}/{a.epochs} window={wi+1}/{len(windows)} horizon={steps}
loss={total:.5f} pressure={float(terms["pressure"].detach()):.5f} route={float(terms["route"].detach()):.5f}
elapsed_s={time.time()-began:.0f}',flush=True)
                if stop[0]:print('Stopped at optimizer boundary; resume from last.pt',flush=True);return
            metrics[split]=sum(v/n for v,n in totals if n)/sum(n>0 for _,n in totals);progress=None
        promote=metrics['validation']<best
        if promote:best=metrics['validation']
        history.append(dict(epoch=epoch,horizon=HORIZONS[min(epoch-1,3)],**metrics))
        for path in [a.output/f'epoch_{epoch:03d}.pt',last]+([a.output/'best.pt'] if promote else []):
            save(path,model,optimizer,m,epoch,history,best)
        base.atomic_json(a.output/'history.json',history)
        print(f'Saved completed epoch {epoch}; best={promote}',flush=True)
    print('Training complete; best selected on fixed 120h validation.',flush=True)


if __name__=='__main__':main()
```

B07. training/trackformer_1_2/EXECUTION_CHECKS.json

738 bytes | SHA-256: baa5800cdca8cbcffdfd5380382d76722613f76169cadf152a4d32553da9c09f


```
{
  "status": "passed",
  "device": "cpu",
  "optimizer_updates": 0,
  "full_training_rerun": false,
  "samples": [
    {
      "native": false,
      "finite_backward": true,
      "attention_gradient_nonzero": true,
      "field_pressure_exact": true,
      "future_input_rejected": true
    },
    {
      "native": true,
      "finite_backward": true,
      "attention_gradient_nonzero": true,
      "field_pressure_exact": true,
      "future_input_rejected": true
    }
  ],
  "sampler_all_train_rows": true,
  "counts": {
    "train": 13949,
    "validation": 1041,
    "test": 1195
  },
  "native_counts": {
    "train": 800,
    "validation": 100,
    "test": 100
  },
  "preflight": "passed",
  "new_full_training_run": false
}
```

B08. training/trackformer_1_2/LICENSE

1,083 bytes | SHA-256: 090a97ebbccef9de77cb3898c473f5c9ada7e140777957f1195fc6ab89b9a313

```
MIT License

Copyright (c) 2026 Yu Yao-Hsing (yu314-coder)

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

B09. training/trackformer_1_2/PUBLICATION.md

1,505 bytes | SHA-256: 1bcaa51a1999422925db2a100927985b927b454a2ff4250e318f7049ceec81c8

```
# Immutable publication and execution checks

- Source publication on GitHub:
https://github.com/yu314-coder/typhoon-predict/commit/ac05e1974ac90ad7819bf0483da8e8637d880525
- Source and all nine assets on Hugging Face:
https://huggingface.co/euler314/typhoon-predict/commit/2d1464e9a310e6a6e8840ef1c23017fdededd57b
- All asset hashes remain fixed in `assets.json`.

Pin the binary download to the completed publication:

```bash
python training/trackformer_1_2/download_assets.py --revision 2d1464e9a310e6a6e8840ef1c23017fdededd57b
```

The original trainer's CPU preflight passed, including all six input-file hashes, exact split counts,
normalization and original warm-start loading (21,452,595 parameters). The portable CPU check passed on one
coarse and one native TRAIN example: finite forward/loss/backward, nonzero attention gradient, pressure exactly
read from the predicted field, rejection of an added future input, and full TRAIN sampler coverage. Zero
optimizer updates were performed. See `EXECUTION_CHECKS.json`. This is an execution check, not a new end-to-end
training reproduction or an accuracy claim.

```bash
python training/trackformer_1_2/test_training_cpu.py
```

This test also reconstructs the loader's local basin mmap; use a dedicated writable data copy, never another
trainer's data directory. Original GPU-specific tests are archived without rewriting their historical
relative-path assumptions in `source/v173/test_model.py`; they were not rerun on GPU for this publication.
```

B10. training/trackformer_1_2/README.md

6,560 bytes | SHA-256: 342c43d94710355101ddc5bcad44923f47707f473eb803398f44fe2d77e8ace9

```
# TrackFormer 1.2: open training package

This is the actual released pressure-field model: development **1.2.73 (v173), epoch 4**, 21,452,595 parameters. It is separate from older route-only 1.2 experiments and later rainfall models.

## Published contents

- `source/v173/`: original, byte-identical model, baseline architecture, archive loader, objective, curriculum, sampler, trainer, resume logic, design and original Metal tests.
- `source/data_preparation/`: original historical ERA5 download/plan utilities and dataset contracts. The small full-field pilot described there is **not** the six-file training corpus.
- `assets.json`: exact sizes and SHA-256 values for all six original derived archives, the original manifest, warm-start checkpoint and selected full training checkpoint.
- `data/`: those exact public arrays, including original split plan, labels/masks and static geography. Binary files are hosted on Hugging Face at the identical path, not committed into GitHub source history.
- `checkpoints/v168_epoch_003.pt`: the original, mandatory warm start, previously split across Drive files.
- `checkpoints/v173_epoch_004.pt`: the selected full training checkpoint, with model, optimizer, normalization, four-epoch history and Python/NumPy/Torch CPU RNG states.
- `config.json`, `checkpoint-history.json`, `SOURCE_MANIFEST.json`: archived recipe, exact saved loss history and source hashes.
- `train.py`: a path-portability wrapper; it calls the unmodified original trainer with explicit paths and settings. No module import starts training.
- Model export, physical pressure-map rendering and benchmark tools remain in the repository's `release_tools/` and `models/trackformer_1_2_field/`.

Both [GitHub](https://github.com/yu314-coder/typhoon-predict/tree/main/training/trackformer_1_2) and [Hugging Face](https://huggingface.co/euler314/typhoon-predict/tree/main/training/trackformer_1_2) carry the source and instructions. Hugging Face serves the large assets (2,644,261,044 bytes in total); no Drive permission is required.

## Download and verify

From the repository root, after obtaining this source folder:

```bash
python training/trackformer_1_2/download_assets.py
python training/trackformer_1_2/download_assets.py --verify-only
python -m pip install -r training/trackformer_1_2/requirements.txt
```

The downloader uses only Python's standard library. It publishes each local file only after exact size/SHA verification and does not overwrite a differing existing file. Use `--revision <immutable-HF-commit>` from `PUBLICATION.md` to pin the upload; even the default `main` URL must match the fixed hashes. For a writable data/output directory on an external disk, copy the package there or set `--destination`; then pass its `data` and `checkpoints` paths to the launcher.

Preflight, then explicitly start training

```bash
python training/trackformer_1_2/train.py --device mps --output runs/retrain-1.2 --preflight
python training/trackformer_1_2/train.py --device mps --output runs/retrain-1.2
```

Use `--device cuda` for NVIDIA or `--device cpu` for CPU. This wrapper explicitly sets the archived notebook/design recipe: effective batch 10, microbatch 4, workers 3, four epochs; pass flags to change execution settings deliberately. It uses FP32 even though the historical notebook's compatibility AMP positional flag was 1. The unchanged original entry point is also available:

```bash
python training/trackformer_1_2/source/v173/v173_pressure_train.py 10 3 0 \
  --microbatch 4 --epochs 4 --device mps \
  --manifest training/trackformer_1_2/data/manifest.json \
  --warm-start training/trackformer_1_2/checkpoints/v168_epoch_003.pt \
  --output runs/retrain-1.2
```

The original loader hashes all arrays, reconstructs a writable basin mmap beside the dataset, recomputes train-only normalization and enforces counts **13,949 / 1,041 / 1,195** with native-pressure subsets **800 / 100 / 100**. Allow space for that extra approximately 390 MB mmap and approximately 260 MB checkpoint files. Do not point preflight/training at an active trainer's data/output directories. The trainer uses a per-output lock; the same output resumes its `last.pt`. SIGINT/SIGTERM request a checkpoint at an optimizer boundary. Training horizons are 4/8/12/20 six-hour steps; validation always uses 20 steps. Four-step truncated backpropagation, gradient clipping, AdamW settings and all losses are in the original source.

Provenance and honest reproduction limits

- Dataset manifest: `04fd0d30620137aad20173dcd59e22da2a6c20fae2f17de593e91a59517459c1`.
- Warm start: `99ed776097e05ad6df03375465d9618741bed63a4ee57724d3ba98e295039d30`.
- Selected full checkpoint: `f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0`.
- Inference-only weights: `db49f36e85a3766defc4c172746897a1f783705d1ce8e6f9dfb8e87ae1d902cb`.

The selected full checkpoint is epoch **4**, used by the report; **it is not the smallest saved aggregate
```

validation loss** (epoch 2 is 0.6939101; epoch 4 is 0.7283264). A fresh run's `best.pt` follows the original minimum-validation criterion and need not be the report's epoch-4 checkpoint. Do not conflate these choices. The supplied warm start makes the actual v173 training path runnable without reconstructing every earlier experimental model. Historical v168 optimization logs and complete original runtime package versions/accelerator RNG states are not archived here. The recipe is preserved from the original notebook/design, not a newly recovered complete original process argv. Bitwise end-to-end retraining across devices has **not** been demonstrated.

The exact derived data are published; the raw IBTrACS release/download ID was not recorded in the frozen manifest. Current upstream downloads may differ and cannot be claimed byte-identical. Fitting uses 2000–2021, validation 2022–2023, original test 2024–2025, with complete storms separated. Later cache records are not fitting years. All future fields/positions are labels only. Repeatedly inspected tests/showcases are development evidence, not a fresh unused holdout. This publication does not rerun training, change released weights or modify active rainfall training.

Licenses and attribution

Project source and project checkpoints retain the repository's MIT license. Upstream weather/track data retain their own terms and attributions, not the MIT software license; see `THIRD_PARTY_NOTICES.md`. Downloaded full checkpoints use Python pickle containers: load these identified, checksum-verified project artifacts only.

B11. training/trackformer_1_2/SOURCE_MANIFEST.json

6,943 bytes | SHA-256: 28c3c50e61448aa46e5b9d3aae81858d879e8d9d869b766c01c8e71f898a97da


```
{
  "schema": "trackformer-1.2-open-training-source-v1",
  "unchanged_original_sources": true,
  "model_weights_modified": false,
  "files": [
    {
      "path": "training/trackformer_1_2/EXECUTION_CHECKS.json",
      "sha256": "baa5800cdca8cbcffdfd5380382d76722613f76169cadf152a4d32553da9c09f",
      "bytes": 738
    },
    {
      "path": "training/trackformer_1_2/LICENSE",
      "sha256": "090a97ebbccef9de77cb3898c473f5c9ada7e140777957f1195fc6ab89b9a313",
      "bytes": 1083
    },
    {
      "path": "training/trackformer_1_2/PUBLICATION.md",
      "sha256": "1bcaa51a1999422925db2a100927985b927b454a2ff4250e318f7049ceec81c8",
      "bytes": 1505
    },
    {
      "path": "training/trackformer_1_2/README.md",
      "sha256": "342c43d94710355101ddc5bcad44923f47707f473eb803398f44fe2d77e8ace9",
      "bytes": 6560
    },
    {
      "path": "training/trackformer_1_2/THIRD_PARTY_NOTICES.md",
      "sha256": "f764dd203eb124fa6192edd3b7b49a7a546cd5d69d3a8a250589a14ce43e2115",
      "bytes": 1697
    },
    {
      "path": "training/trackformer_1_2/assets.json",
      "sha256": "a59148e9ba2617599614784b47789c3654df1a7809ee039e78bcd00fe982b211",
      "bytes": 1993
    },
    {
      "path": "training/trackformer_1_2/checkpoint-history.json",
      "sha256": "8b9593a7ed58c1bd28dd27c8049c89a0c5c5644a30bfe5203a4b54c445ff2fb4",
      "bytes": 459
    },
    {
      "path": "training/trackformer_1_2/config.json",
      "sha256": "813f5d05a3dec5ec160b4e91eb80970fb11d88416fc7709ffc9d147fc880316b",
      "bytes": 820
    },
    {
      "path": "training/trackformer_1_2/download_assets.py",
      "sha256": "01d21d09c001394f49d807314e38b22a4a8c179783a3f22ddaa4c27e8c3f2011",
      "bytes": 3049
    },
    {
      "path": "training/trackformer_1_2/requirements.txt",
      "sha256": "37e15a665eff6c8ee94c47ac2da0f7c7e2d699b7e21623b0289da4cf5d3995d0",
      "bytes": 45
    },
    {
      "path": "training/trackformer_1_2/source/data_preparation/requirements.txt",
      "sha256": "54918b8f29330299e18b8924e9cd93d05807e7f8cd99b2efd4805e9be94003c1",
      "bytes": 55
    },
    {
      "path": "training/trackformer_1_2/source/data_preparation/v164_dataset_contract.md",
      "sha256": "cfd838651e632aba0d2439ba202a59895bd4df22137185974df1d97f0e2525d4",
      "bytes": 4664
    },
    {
      "path": "training/trackformer_1_2/source/data_preparation/v164_download_pressure_geography.py",
      "sha256": "b8602cd2fba131a287dc9e25781c8f037d3325fec8a23203576916c2f7d2189c",
      "bytes": 9256
    },
    {
      "path": "training/trackformer_1_2/source/data_preparation/v164_global_regional_train.py",
      "sha256": "c0808a1467695d2bc2ad8483aa4f4c6de1463628221024ac559f6204effe7978",
      "bytes": 52581
    },
    {
      "path": "training/trackformer_1_2/source/data_preparation/v164_prepare_data.py",
      "sha256": "c1746e0c0f985890a0f02061a2d84b521cc7b29042c92972aa1ebab3354e4c95",
      "bytes": 19312
    },
    {
      "path": "training/trackformer_1_2/source/data_preparation/v164_prepare_mac.py",
```

```
      "sha256": "7e7313f865369d42f67645196b4af0e1b2b84216303f7b2b61e577d3734df79a",
      "bytes": 1309
    },
    {
      "path": "training/trackformer_1_2/source/data_preparation/v164_reuse_impl.py",
      "sha256": "2da0078d4a74faa93f8cf874e42707b01f286406f0ceaa4db1814aab0eceb7d1",
      "bytes": 18414
    },
    {
      "path": "training/trackformer_1_2/source/data_preparation/v164_structure_research.md",
      "sha256": "a4b40e50b7d0d1433a162438f14ed110541702c4cec2a386959857293ea78141",
      "bytes": 17119
    },
    {
      "path": "training/trackformer_1_2/source/v173/DESIGN.md",
      "sha256": "163ae39ff572f23c519d65e27ee6a8db23b47df9461670709229ccf95130049a",
      "bytes": 3278
    },
    {
      "path": "training/trackformer_1_2/source/v173/baseline_model.py",
      "sha256": "6c05941fd93baaca04508877edc69f8f0394b70e0a64e734ef0d2c1e8832246a",
      "bytes": 10040
    },
    {
      "path": "training/trackformer_1_2/source/v173/model.py",
      "sha256": "5dab06c20224cf48c51302122455ee7e16948f977bf4b760fd8aff5d443ab0bc",
      "bytes": 3428
    },
    {
      "path": "training/trackformer_1_2/source/v173/test_model.py",
      "sha256": "493e0db88a098c89bb4db5e6c577fa24b4b7bcb83ce547dceac35237dbce7206",
      "bytes": 5579
    },
    {
      "path": "training/trackformer_1_2/source/v173/v165_base.py",
      "sha256": "5c9473993a19d0fed00055494b5c1f50c1006aaf46551ff50145f67048c3374a",
      "bytes": 39040
    },
    {
      "path": "training/trackformer_1_2/source/v173/v173_pressure_train.py",
      "sha256": "f40691cec3e787f4ac3c189f647efb0953c1eaa7e03fc86edaf4fee06990b2be",
      "bytes": 11200
    },
    {
      "path": "training/trackformer_1_2/test_training_cpu.py",
      "sha256": "35e9379c40f5928f72f8267dcbdbea892387e0261d74e476730ddcbdf143ba1b",
      "bytes": 3279
    },
    {
      "path": "training/trackformer_1_2/train.py",
      "sha256": "4502a87bfafb11667015070c6fad37b723b4a53723789febda075fba1c7770e8",
      "bytes": 1436
    }
  ],
  "assets": [
    {
      "path": "training/trackformer_1_2/data/manifest.json",
      "sha256": "04fd0d30620137aad20173dcd59e22da2a6c20fae2f17de593e91a59517459c1",
      "bytes": 814,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/track_windows_v13.npz",
      "sha256": "cb8d7291aa8140124ca522feb448a1d44124b018efc4ba1d54824b60a4f73451",
      "bytes": 42755049,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/basin_all_int8.npz",
      "sha256": "71e5f1bb46c44e5936c506a3f8af021c1fe94c238ed030c05c66e8f342374b6d",
      "bytes": 274115577,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/basin_slp_atlas_float16.npy",
      "sha256": "9f6a4ec7b2f6beadfa1075af9176d2a653e39a824d504d989e5e56e19c571521",
      "bytes": 111394928,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/pressure_hpa.npy",
      "sha256": "16cc1b153aeee5eff4b67337b599892dcd86a5716554596b8031048035d247ba",
      "bytes": 1698356128,
      "group": "data"
    },
    {
```

```
      "path": "training/trackformer_1_2/data/plan.json",
      "sha256": "117eb291e385a326191be2f73182ed6dcbf0e9f99745780ac9e49a29def3d8da",
      "bytes": 295005,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/geography.npz",
      "sha256": "f5471471da4a1e15f85fbfb2529065806281357c8e50cb272fcd2b02e8553b96",
      "bytes": 2547017,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/checkpoints/v168_epoch_003.pt",
      "sha256": "99ed776097e05ad6df03375465d9618741bed63a4ee57724d3ba98e295039d30",
      "bytes": 256826555,
      "group": "checkpoints"
    },
    {
      "path": "training/trackformer_1_2/checkpoints/v173_epoch_004.pt",
      "sha256": "f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0",
      "bytes": 257969971,
      "group": "checkpoints"
    }
  ]
}
```

B12. training/trackformer_1_2/THIRD_PARTY_NOTICES.md

1,697 bytes | SHA-256: f764dd203eb124fa6192edd3b7b49a7a546cd5d69d3a8a250589a14ce43e2115

```
# Data sources and attribution

The MIT license covers project code and project model artifacts; it does not replace upstream data terms.

- **NOAA PSL NCEP/NCAR Reanalysis 1**: basin mean sea-level pressure, 500 hPa height and winds at 850/500/200 hPa. Data provided by NOAA PSL, Boulder, Colorado, USA. https://psl.noaa.gov/data/gridded/data.ncep.reanalysis.html
- **NOAA/NCEI IBTrACS**: derived storm observations, labels and masks; credit the original contributing meteorological agencies and NOAA/NCEI. Knapp et al. (2010), doi:10.1175/2009BAMS2755.1. https://www.ncei.noaa.gov/products/international-best-track-archive . Exact derived archive SHA is in assets.json; its original raw snapshot ID was not recorded.
- **Copernicus Climate Change Service (C3S), ERA5, accessed through Google's ARCO-ERA5 mirror**: native mean-sea-level-pressure patches and static land fraction/elevation. Contains modified Copernicus Climate Change Service information (2026). ERA5 hourly single-level data, doi:10.24381/cds.adbb2d47, https://cds.climate.copernicus.eu/datasets/reanalysis-era5-single-levels . The current dataset lists a CC-BY licence; attribution and upstream terms apply. ECMWF or the European Commission does not endorse this model. ARCO mirror and recipes: https://github.com/google-research/arco-era5 . The mirror's code licence is separate from the underlying meteorological data licence.

The published arrays are transformed subsets: quantized/coarse basin grids, float16 pressure, issue-centred patches, normalized derived track features and masks. These transformations do not create new native observational resolution. No claim of endorsement by NOAA, ECMWF, C3S or Google is made.
```

B13. training/trackformer_1_2/assets.json

1,993 bytes | SHA-256: a59148e9ba2617599614784b47789c3654df1a7809ee039e78bcd00fe982b211


```
{
  "repository": "euler314/typhoon-predict",
  "repo_type": "model",
  "files": [
    {
      "path": "training/trackformer_1_2/data/manifest.json",
      "sha256": "04fd0d30620137aad20173dcd59e22da2a6c20fae2f17de593e91a59517459c1",
      "bytes": 814,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/track_windows_v13.npz",
      "sha256": "cb8d7291aa8140124ca522feb448a1d44124b018efc4ba1d54824b60a4f73451",
      "bytes": 42755049,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/basin_all_int8.npz",
      "sha256": "71e5f1bb46c44e5936c506a3f8af021c1fe94c238ed030c05c66e8f342374b6d",
      "bytes": 274115577,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/basin_slp_atlas_float16.npy",
      "sha256": "9f6a4ec7b2f6beadfa1075af9176d2a653e39a824d504d989e5e56e19c571521",
      "bytes": 111394928,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/pressure_hpa.npy",
      "sha256": "16cc1b153aeee5eff4b67337b599892dcd86a5716554596b8031048035d247ba",
      "bytes": 1698356128,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/plan.json",
      "sha256": "117eb291e385a326191be2f73182ed6dcbf0e9f99745780ac9e49a29def3d8da",
      "bytes": 295005,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/data/geography.npz",
      "sha256": "f5471471da4a1e15f85fbfb2529065806281357c8e50cb272fcd2b02e8553b96",
      "bytes": 2547017,
      "group": "data"
    },
    {
      "path": "training/trackformer_1_2/checkpoints/v168_epoch_003.pt",
      "sha256": "99ed776097e05ad6df03375465d9618741bed63a4ee57724d3ba98e295039d30",
      "bytes": 256826555,
      "group": "checkpoints"
    },
    {
      "path": "training/trackformer_1_2/checkpoints/v173_epoch_004.pt",
      "sha256": "f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0",
      "bytes": 257969971,
      "group": "checkpoints"
    }
  ]
}
```

B14. training/trackformer_1_2/checkpoint-history.json

459 bytes | SHA-256: 8b9593a7ed58c1bd28dd27c8049c89a0c5c5644a30bfe5203a4b54c445ff2fb4


```
[
  {
    "epoch": 1,
    "horizon": 4,
    "train": 0.1411035786937901,
    "validation": 0.7017406511544373
  },
  {
    "epoch": 2,
    "horizon": 8,
    "train": 0.18676476989154306,
    "validation": 0.6939100875934489
  },
  {
    "epoch": 3,
    "horizon": 12,
    "train": 0.23248198316925273,
    "validation": 0.7200751373592149
  },
  {
    "epoch": 4,
    "horizon": 20,
    "train": 0.3557772162892601,
    "validation": 0.7283263609942383
  }
]
```

B15. training/trackformer_1_2/config.json

820 bytes | SHA-256: 813f5d05a3dec5ec160b4e91eb80970fb11d88416fc7709ffc9d147fc880316b


```
{
  "public_version": "1.2",
  "development_version": "1.2.73",
  "seed": 168,
  "optimizer": {
    "name": "AdamW",
    "lr": 3e-05,
    "weight_decay": 0.01
  },
  "gradient_clip_norm": 1.0,
  "epochs": 4,
  "training_horizon_steps": [
    4,
    8,
    12,
    20
  ],
  "validation_horizon_steps": 20,
  "precision": "float32",
  "effective_batch": 10,
  "microbatch": 4,
  "workers": 3,
  "checkpoint_every_windows": 50,
  "launch_provenance": "Archived notebook/design command; complete original machine/runtime/argv not recorded in selected checkpoint",
  "selected_epoch": 4,
  "minimum_saved_validation_loss_epoch": 2,
  "warm_start_sha256": "99ed776097e05ad6df03375465d9618741bed63a4ee57724d3ba98e295039d30",
  "selected_checkpoint_sha256": "f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0"
}
```

B16. training/trackformer_1_2/download_assets.py

3,049 bytes | SHA-256: 01d21d09c001394f49d807314e38b22a4a8c179783a3f22ddaa4c27e8c3f2011

```
"""Fetch exact public training inputs/checkpoints, verifying every SHA-256.

Uses Python's standard library. Does not train, install packages, or use Drive.
"""
import argparse
import hashlib
import json
from pathlib import Path
import os
import urllib.parse
import urllib.request


def sha(path):
    h = hashlib.sha256()
    with path.open('rb') as f:
        for block in iter(lambda: f.read(8 * 1024**2), b''):
            h.update(block)
    return h.hexdigest()


def download(entry, root, base, prefix):
    relative = Path(entry['path']).relative_to(prefix)
    path = root / relative
    if path.exists():
        if path.stat().st_size == entry['bytes'] and sha(path) == entry['sha256']:
            print('Verified existing', relative, flush=True)
            return
        raise ValueError('Existing file differs; preserve/move it before retrying: ' + str(path))
    path.parent.mkdir(parents=True, exist_ok=True)
    temp = path.with_name(path.name + '.downloading')
    url = base + urllib.parse.quote(entry['path'], safe='/')
    print('Downloading', relative, entry['bytes'], 'bytes', flush=True)
    try:
        h = hashlib.sha256()
        size = 0
        with urllib.request.urlopen(url, timeout=120) as response, temp.open('wb') as out:
            for block in iter(lambda: response.read(8 * 1024**2), b''):
                out.write(block)
                h.update(block)
                size += len(block)
            out.flush()
            os.fsync(out.fileno())
        if size != entry['bytes'] or h.hexdigest() != entry['sha256']:
            raise ValueError('Downloaded content differs: ' + str(relative))
        temp.replace(path)
    except Exception:
        if temp.is_file():
            temp.unlink()
        raise
    print('Verified', relative, flush=True)


def main():
    here = Path(__file__).resolve().parent
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('--destination', type=Path, default=here)
    p.add_argument('--revision', default='main', help='Use the immutable HF revision in PUBLICATION.md for exact
 publication')
    p.add_argument('--group', choices=['all', 'data', 'checkpoints'], default='all')
    p.add_argument('--verify-only', action='store_true')
    a = p.parse_args()
    manifest = json.loads((here / 'assets.json').read_text())
    base = 'https://huggingface.co/' + manifest['repository'] + '/resolve/' + urllib.parse.quote(a.revision,
safe='') + '/'
    for entry in manifest['files']:
        if a.group != 'all' and entry['group'] != a.group:
            continue
        if a.verify_only:
            path = a.destination / Path(entry['path']).relative_to('training/trackformer_1_2')
            if not path.is_file() or path.stat().st_size != entry['bytes'] or sha(path) != entry['sha256']:
                raise ValueError('Missing or different asset: ' + str(path))
            print('Verified', path, flush=True)
        else:
            download(entry, a.destination, base, 'training/trackformer_1_2')


if __name__ == '__main__':
    main()
```

B17. training/trackformer_1_2/requirements.txt

45 bytes | SHA-256: 37e15a665eff6c8ee94c47ac2da0f7c7e2d699b7e21623b0289da4cf5d3995d0

```
1  numpy>=1.26
2  torch>=2.1
3  huggingface_hub>=0.24
```

B18. training/trackformer_1_2/test_training_cpu.py

3,279 bytes | SHA-256: 35e9379c40f5928f72f8267dcbdbea892387e0261d74e476730ddcbdf143ba1b

```
"""CPU-only execution check of real TRAIN inputs; zero optimizer updates."""
import argparse
import json
from pathlib import Path
import sys


def main():
    here = Path(__file__).resolve().parent
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('--data-root', type=Path, default=here / 'data')
    p.add_argument('--warm-start', type=Path, default=here / 'checkpoints/v168_epoch_003.pt')
    a = p.parse_args()
    sys.path.insert(0, str(here / 'source/v173'))
    import torch
    from model import CoreForecaster, objective, base
    from v173_pressure_train import warm_start, batches
    torch.set_num_threads(2)
    torch.manual_seed(168)
    base.EvolutionNet.block = lambda self, module, x: module(x)
    m = base.load_reuse(a.data_root / 'manifest.json')
    model = CoreForecaster(m).cpu()
    warm_start(model, a.warm_start, m)
    data = base.ReuseDataset(m, 'train')
    samples = [next(i for i, r in enumerate(data.rows) if (r['patch'] >= 0) == native and r['issue_mask'][1])
               for native in (False, True)]
    results = []
    for index in samples:
        x, y = data[index]
        x = {k: v[None] for k, v in x.items()}
        model.train()
        model.zero_grad(set_to_none=True)
        state = model.initial(x)
        state, prediction = model.step(state)
        target = {k: v[0:1] for k, v in y.items()}
        target.update(previous_pressure=x['issue_intensity'][:, 1],
                      previous_pressure_mask=x['issue_mask'][:, 1],
                      previous_center=x['center'],
                      previous_storm_mask=torch.ones_like(target['storm_mask']))
        loss, terms = objective(prediction, target, x, m)
        assert bool(torch.isfinite(loss))
        loss.backward()
        assert all(bool(torch.isfinite(v.grad).all()) for v in model.parameters() if v.grad is not None)
        assert model.context.output.weight.grad.abs().sum() > 0
        grid = model.core_grid(prediction['center'][:, 0, None, None],
                               prediction['center'][:, 1, None, None], state['origin'])
        pressure = base.sample_field(prediction['core'] * model.std[:, :1] + model.mean[:, :1], grid)[:, 0, 0,
0]
        assert torch.equal(pressure, prediction['pressure'])
        try:
            model.initial(dict(x, future_pressure=target['regional']))
        except ValueError:
            pass
        else:
            raise AssertionError('Future target accepted as input')
        results.append(dict(native=bool(x['detail_available'].item()), finite_backward=True,
                            attention_gradient_nonzero=True, field_pressure_exact=True,
                            future_input_rejected=True))
    indices = base.training_indices(data.rows, 1)
    assert set(indices) == set(range(len(data)))
    for parts, count in batches(indices, data.rows, 10, 4):
        assert sum(map(len, parts)) == count
        assert all(len({data.rows[i]['patch'] >= 0 for i in part}) == 1 for part in parts)
    print(json.dumps(dict(status='passed', device='cpu', optimizer_updates=0,
        full_training_rerun=False, samples=results, sampler_all_train_rows=True,
        counts=m['counts'], native_counts=m['patch_counts']), indent=2))


if __name__ == '__main__':
    main()
```

B19. training/trackformer_1_2/train.py

1,436 bytes | SHA-256: 4502a87bfafb11667015070c6fad37b723b4a53723789febda075fba1c7770e8

```
"""Portable launcher for the unchanged archived v173 trainer; no auto-training."""
import argparse
from pathlib import Path
import runpy
import sys


def main():
    here = Path(__file__).resolve().parent
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('--data-root', type=Path, default=here / 'data')
    p.add_argument('--warm-start', type=Path, default=here / 'checkpoints/v168_epoch_003.pt')
    p.add_argument('--output', type=Path, required=True)
    p.add_argument('--device', choices=['cpu', 'mps', 'cuda'], required=True)
    p.add_argument('--epochs', type=int, default=4)
    p.add_argument('--batch', type=int, default=10)
    p.add_argument('--microbatch', type=int, default=4)
    p.add_argument('--workers', type=int, default=3)
    p.add_argument('--preflight', action='store_true')
    a = p.parse_args()
    source = here / 'source/v173'
    sys.path.insert(0, str(source))
    sys.argv = [str(source / 'v173_pressure_train.py'), str(a.batch), str(a.workers), '0',
        '--microbatch', str(a.microbatch), '--epochs', str(a.epochs),
        '--device', a.device, '--manifest', str(a.data_root.resolve() / 'manifest.json'),
        '--warm-start', str(a.warm_start.resolve()), '--output', str(a.output.resolve())]
    if a.preflight:
        sys.argv.append('--preflight')
    runpy.run_path(str(source / 'v173_pressure_train.py'), run_name='__main__')


if __name__ == '__main__':
    main()
```

B20. CITATION.cff

450 bytes | SHA-256: c02df97de3c0b496896a28b726eef0649709424073c2894be325a872c72c0e08

```
cff-version: 1.2.0
message: "If you use TrackFormer, cite this software and the upstream weather/track data described in the source notices."
type: software
title: "TrackFormer 1.2: Northwest Pacific pressure-field and typhoon forecasting"
version: "1.2"
authors:
  - family-names: "Yu"
    given-names: "Yao-Hsing"
repository-code: "https://github.com/yu314-coder/typhoon-predict"
url: "https://huggingface.co/euler314/typhoon-predict"
license: MIT
```

B21. CONTRIBUTING.md

2,108 bytes | SHA-256: 86e953e9a804566cefb142340d373eb00669a30937002857de8f2b46665ea11b

```
# Contributing to TrackFormer

Anyone may inspect, fork, modify and retrain the public release under its MIT license. No contributor agreement or account approval is required to download or use the code and weights. Submitting a GitHub pull request requires your own GitHub account; local use does not.

## Reproduce first

1. Follow `training/trackformer_1_2/README.md` and pin the asset revision from `PUBLICATION.md`.
2. Verify dataset/checkpoint SHA-256 values before changing anything. Keep original weights and source artifacts unchanged and write experiments into new output directories.
3. Use `--preflight` for the archive/architecture check and `test_training_cpu.py` for the portable CPU loss/backward check. These reconstruct a writable local mmap, so use your own data copy rather than an active trainer's directory.
4. For pressure export changes, use the existing `release_tools/test_pressure_field_export.py` checks as applicable. There is no requirement to run the full multi-hour training job for a documentation correction.

## Share a change

Open an issue or pull request at https://github.com/yu314-coder/typhoon-predict . Explain the problem, changed behavior, exact model/data revision and relevant checks. Document new dependencies and retain third-party notices. Never commit credentials, private data or unrelated large caches. New project contributions should use the existing MIT license; third-party material must keep its own license and attribution.

For scientific claims, preserve issue-time causal inputs and whole-storm partitions; keep future observations as labels only. Compare on identical cases, grids, times and masks. Report misses and false alarms as well as mean errors, and identify previously reused development data. Do not infer forecast skill from a smoother map or training loss. Label changed architectures and weights as new candidates rather than silently replacing released 1.2.

The MIT license permits broad reuse; scientific review criteria apply to what this repository accepts or claims, not to your permission to fork or experiment independently.
```

B22. LICENSE

1,083 bytes | SHA-256: 090a97ebbccef9de77cb3898c473f5c9ada7e140777957f1195fc6ab89b9a313

```
MIT License

Copyright (c) 2026 Yu Yao-Hsing (yu314-coder)

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

B23. OPENNESS.md

3,640 bytes | SHA-256: 114b3dfef38441d29cb3a9e335c337f36598a760c192e925597466436304d8d9

```
# Open access and reuse

TrackFormer 1.2 is publicly accessible without account approval, a download gate, a paid API or Google Drive permissions. Project code and project model weights/checkpoints are distributed under the [MIT license](LICENSE). You may use, study, modify, retrain, redistribute, sublicense and sell copies, including commercial use, subject to retaining the required copyright and permission notice. No research-only or non-commercial restriction is added by this project. The research/safety cautions describe capability limits; they do not replace or narrow the MIT license.

## What is public

| Component | Public location |
| --- | --- |
| Architecture, inference and physical pressure exporter | `models/trackformer_1_2_field/` on both repositories |
| Original training source, losses, sampler, curriculum and resume logic | `training/trackformer_1_2/source/v173/` |
| Portable launcher, configuration, tests and exact saved training history | `training/trackformer_1_2/` |
| Exact six derived dataset files, masks and original dataset manifest | Hugging Face: `training/trackformer_1_2/data/` |
| Original warm start and selected full training checkpoint with optimizer/RNG state | Hugging Face: `training/trackformer_1_2/checkpoints/` |
| Hashes and anonymous checksum-verifying asset downloader | `training/trackformer_1_2/assets.json` and `download_assets.py` |
| Benchmark protocols, measured results and figure provenance | `evaluation/`, `docs/` and `release_tools/` |

[GitHub source](https://github.com/yu314-coder/typhoon-predict) · [Hugging Face source and binary assets](https://huggingface.co/euler314/typhoon-predict) · [Training instructions](training/trackformer_1_2/README.md) · [Immutable publication links](training/trackformer_1_2/PUBLICATION.md) · [Contributing](CONTRIBUTING.md)

Large arrays and optimizer checkpoints are hosted on Hugging Face and linked from GitHub, rather than duplicated in ordinary Git history. Downloads are standard public HTTPS; the asset downloader uses Python's standard library, verifies exact hashes and requires no token. Training/inference still require the documented open-source runtime dependencies. There is no hosted-service dependency in the published training launcher.

## Upstream data terms

NOAA/IBTrACS and Copernicus/ERA5 data retain their own provenance, attribution and applicable upstream terms. The project's MIT license cannot remove third-party conditions. See [the source notices](training/trackformer_1_2/THIRD_PARTY_NOTICES.md) and [data provenance](docs/trackformer_1_2_training_data.md). The root `license: mit` metadata describes project software and model artifacts, not a blanket relicensing of all weather data.

## What is not claimed

The frozen derived arrays and their exact hashes are public, but the original raw IBTrACS snapshot/download ID was not recorded. A complete original runtime lockfile, original process argv and earlier warm-start training history have not been recovered. The archived recipe and warm-start checkpoint make the released v173 path runnable; bitwise full retraining across devices has not been demonstrated. The report selects epoch 4, while the saved aggregate validation-loss minimum is epoch 2. Previously inspected tests and examples are development evidence, not an untouched holdout.

Public availability is not a claim that every earlier 1.2 experiment or later rainfall experiment is this released model. No unavailable artifact, qualification result or score is invented to claim complete reproducibility. These remaining gaps stay visible so others can audit and improve the release.
```

B24. derived-records/selected-checkpoint-training.json

1,537 bytes | SHA-256: 28a8b86dfd66cc718d7cb0b165af885cbee3f39627fdb9cda0be91c021260180


```
{
  "source_checkpoint_sha256": "f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0",
  "version": "1.2.73",
  "architecture": "v173-moving-core-multiscale-attention",
  "completed_epoch": 4,
  "history": [
    {
      "epoch": 1,
      "horizon": 4,
      "train": 0.1411035786937901,
      "validation": 0.7017406511544373
    },
    {
      "epoch": 2,
      "horizon": 8,
      "train": 0.18676476989154306,
      "validation": 0.6939100875934489
    },
    {
      "epoch": 3,
      "horizon": 12,
      "train": 0.23248198316925273,
      "validation": 0.7200751373592149
    },
    {
      "epoch": 4,
      "horizon": 20,
      "train": 0.3557772162892601,
      "validation": 0.7283263609942383
    }
  ],
  "saved_best": 0.6939100875934489,
  "progress": null,
  "optimizer_step_values": [
    4384
  ],
  "optimizer_parameter_states": 375,
  "optimizer_groups": [
    {
      "lr": 3e-05,
      "betas": [
        0.9,
        0.999
      ],
      "eps": 1e-08,
      "weight_decay": 0.01,
      "amsgrad": false,
      "maximize": false,
      "foreach": null,
      "capturable": false,
      "differentiable": false,
      "fused": null,
      "decoupled_weight_decay": true
    }
  ],
  "epoch_draws_from_public_sampler": 17532,
  "archived_notebook_effective_batch": 10,
  "expected_steps_four_epochs_batch10": 7016,
  "inferred_checkpoint_compatible_effective_batch": 16,
  "expected_steps_four_epochs_batch16": 4384,
  "inference_is_not_recorded_original_argv": true,
  "optimizer_updates_performed": 0
}
```

B25. docs/trackformer_1_2_training_data.md

6,918 bytes | SHA-256: 196de062102735e2ae9fa4f5c4297612635d9049aba47870a3256db9a8698083

```
# Trackformer 1.2 training data

The **released 1.2 model's fitting cutoff is 2021**, not 2025 or 2026. Its selected checkpoint is development version 1.2.73, epoch 4. The dataset and normalization identities are frozen in the [release manifest](../models/trackformer_1_2_field/manifest.json). This document describes that release, not later 1.3 training or live-input downloads.

## Sources actually consumed by the released training loader

| Component | Dataset | Variables and sampling |
| --- | --- | --- |
| Storm observations / supervised labels | [NOAA IBTrACS](https://www.ncei.noaa.gov/products/international-best-track-archive)-derived `track_windows_v13.npz` | Position, prior motion, available maximum wind and central pressure, exact storm/time identities and label masks. Only eligible Western Pacific-domain windows enter this release. |
| Basin weather | [NOAA PSL NCEP/NCAR Reanalysis 1](https://psl.noaa.gov/data/gridded/data.ncep.reanalysis.html), `basin_all_int8.npz` and the exact-time SLP atlas | MSLP, 500 hPa geopotential height, zonal/meridional winds at 850/500/200 hPa. Six-hour analyses on 100–180°E, 0–60°N; 2.5° grid, 25 × 33 cells. |
| Native regional pressure | [ARCO-ERA5](https://github.com/google-research/arco-era5), `pressure_hpa.npy` with its frozen `plan.json` | Mean sea-level pressure on 0.25° fixed issue-centred 121 × 121 patches. 1,000 original windows across all three partitions, not native detail for every coarse window. |
| Static geography | Same ARCO-ERA5 source, `geography.npz` | Land fraction and surface geopotential / 9.80665 to obtain elevation in metres; latitude/longitude encodings. Static context is not an additional temporal fitting period. |

The eight weather channels are **MSLP, HGT500, U850, V850, U500, V500, U200, V200**. No satellite cloud imagery, humidity/vapour, SST or ocean-heat-content channel is present in this released input contract. Such features in other experiments must not be attributed to 1.2.

The eastward/northward **u/v pairs encode wind direction and speed together**, not direction alone. These environmental winds evolve during the model rollout; their presence is not a separate claim of validated monsoon or surface-wind skill. The [Chinese research explanation](research_report_zh.md) uses these same source variables and partition counts.

The selected dataset manifest identifies the **derived IBTrACS archive by SHA-256**, not by an independently recorded raw IBTrACS release/download ID. A newer downloaded IBTrACS file elsewhere is not evidence of which raw snapshot built these frozen windows. Available wind labels and the learned auxiliary head do not establish a harmonized agency averaging period or a validated surface-wind/radius forecast.

## Temporal partitions

| Role | Whole-storm years | Eligible windows / storms | Native-pressure windows | First → last eligible UTC issue |
| --- | --- | --- | ---: | --- |
| Fitting and normalization | 2000–2021 | 13,949 / 611 | 800 | 2000-05-05 18:00 → 2021-12-20 12:00 |
| Validation / checkpoint selection | 2022–2023 | 1,041 / 46 | 100 | 2022-03-31 00:00 → 2023-12-17 18:00 |
| Original test / evaluation | 2024–2025 | 1,195 / 56 | 100 | 2024-05-22 00:00 → 2025-12-02 12:00 |

These are loader-eligible window counts, not independent storms or the number of gradient updates. Native-pressure windows are a **subset** of the corresponding eligible windows, not additional cases. Storm membership uses each storm's first year in the frozen archive. Windows whose −48 h history or +120 h targets cross a partition boundary are excluded. Weather normalization uses the 2000–2021 basin frames only.

Every forecast history comprises nine consecutive six-hour analyses at **−48, −42, …, 0 h**. Supervised fields and storm labels at **+6, +12, …, +120 h** are targets only; they cannot be inference inputs. Missing native detail and missing intensity labels retain their masks. Validation is not gradient fitting, but it does inform checkpoint selection. Previously inspected original tests and showcases are development evidence, not a new unused holdout.

The two-day span is the input format chosen for this implementation. No matched 24/48/72-hour experiment establishes it as the best history length. Likewise, the evaluation's 50 input-perturbed forecasts share one set of weights; the count is not a training-data multiplier or proof of a calibrated ensemble.

## Newer data do not change the fitting cutoff

The checksum-pinned caches contain records beyond the fitting years:

- Derived storm-window archive: **1980-01-03 00 UTC to 2026-07-13 00 UTC**.
- Coarse NCEP weather archive: **1980-01-01 00 UTC to 2026-03-17 18 UTC**.
- Original native-pressure plan: fitting, validation and test windows in **2000–2025**, separated as above.

Those are **source-cache endpoints**, not years all used to fit 1.2. The 2026 live/backfill pipeline uses the unchanged released model with later causal inputs; GFS transfer is experimental, and generating a new forecast does not retrain the weights. Retrospective reanalysis availability is not proof that the same inputs were available operationally at each historical issue time.

## Verified identity and reproduction

The [training-data receipt](../evaluation/training_data/trackformer_1_2_provenance.json) rechecks all six derived dataset file hashes, the released dataset-manifest hash, exact split counts, storm separation and native-patch membership. It records the original cache endpoints and exact earliest histories/latest targets.
```

```
    This is not a new raw-source-download audit.
47
48  | Frozen identity | SHA-256 |
49  | --- | --- |
50  | Dataset manifest | `04fd0d30620137aad20173dcd59e22da2a6c20fae2f17de593e91a59517459c1` |
51  | Selected training checkpoint | `f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0` |
52  | Inference-only weights | `db49f36e85a3766defc4c172746897a1f783705d1ce8e6f9dfb8e87ae1d902cb` |
53
54  With the original frozen dataset available, reproduce the small receipt without inference, downloads or
    refreshing the training cache:
55
56  ```bash
57  python release_tools/audit_training_data.py --dataset-root /Volumes/D/typhoon_predict/data/v164_reuse --output
    /Volumes/D/typhoon_predict/remote_typhoon_predict_repo/evaluation/training_data/trackformer_1_2_provenance.json
58  ```
59
60  The exact derived arrays, original manifest, warm start and selected full training checkpoint are now publicly
    available in [the open training package](../training/trackformer_1_2/README.md). Original source bytes and
    frozen model weights are unchanged. The additive publication does not run training or modify any active trainer.
61
62  The source archive includes historical ERA5 preparation utilities; its small full-field pilot is not the
    released six-file corpus. Use the published exact corpus to reproduce the released data boundary. No missing raw
     IBTrACS provenance has been invented.
```

B26. evaluation/training_data/trackformer_1_2_provenance.json

4,336 bytes | SHA-256: 9d72a47bff0332d1c0f1cba87827713931967e63ab50ff9d133c0629b2e9af53


```
{
  "schema": "trackformer-1.2-training-data-audit-v1",
  "status": "verified",
  "public_version": "1.2",
  "source_checkpoint_version": "1.2.73",
  "source_checkpoint_sha256": "f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0",
  "inference_weights_sha256": "db49f36e85a3766defc4c172746897a1f783705d1ce8e6f9dfb8e87ae1d902cb",
  "dataset_manifest_sha256": "04fd0d30620137aad20173dcd59e22da2a6c20fae2f17de593e91a59517459c1",
  "verified_archive_files": {
    "track": {
      "file": "track_windows_v13.npz",
      "sha256": "cb8d7291aa8140124ca522feb448a1d44124b018efc4ba1d54824b60a4f73451",
      "bytes": 42755049
    },
    "basin": {
      "file": "basin_all_int8.npz",
      "sha256": "71e5f1bb46c44e5936c506a3f8af021c1fe94c238ed030c05c66e8f342374b6d",
      "bytes": 274115577
    },
    "slp": {
      "file": "basin_slp_atlas_float16.npy",
      "sha256": "9f6a4ec7b2f6beadfa1075af9176d2a653e39a824d504d989e5e56e19c571521",
      "bytes": 111394928
    },
    "pressure": {
      "file": "pressure_hpa.npy",
      "sha256": "16cc1b153aeee5eff4b67337b599892dcd86a5716554596b8031048035d247ba",
      "bytes": 1698356128
    },
    "plan": {
      "file": "plan.json",
      "sha256": "117eb291e385a326191be2f73182ed6dcbf0e9f99745780ac9e49a29def3d8da",
      "bytes": 295005
    },
    "geography": {
      "file": "geography.npz",
      "sha256": "f5471471da4a1e15f85fbfb2529065806281357c8e50cb272fcd2b02e8553b96",
      "bytes": 2547017
    }
  },
  "sources": {
    "track": "NOAA IBTrACS-derived track_windows_v13",
    "basin": "NOAA PSL NCEP/NCAR Reanalysis 1",
    "native_pressure":
"https://storage.googleapis.com/gcp-public-data-arco-era5/ar/full_37-1h-0p25deg-chunk-1.zarr-v3",
    "geography":
"https://storage.googleapis.com/gcp-public-data-arco-era5/ar/full_37-1h-0p25deg-chunk-1.zarr-v3",
    "raw_ibtracs_release": "Not recorded in the selected dataset manifest; derived archive hash is
authoritative"
  },
  "channels": [
    "mslp",
    "hgt500",
    "uwnd850",
    "vwnd850",
    "uwnd500",
    "vwnd500",
    "uwnd200",
    "vwnd200"
  ],
  "normalization_fit_years": [
    2000,
    2021
  ],
  "partitions": {
    "train": {
      "years": [
        2000,
        2021
      ],
      "windows": 13949,
      "storms": 611,
      "native_pressure_windows": 800,
      "first_issue_utc": "2000-05-05T18:00:00Z",
      "last_issue_utc": "2021-12-20T12:00:00Z",
      "earliest_history_utc": "2000-05-03T18:00:00Z",
      "latest_target_utc": "2021-12-25T12:00:00Z"
    },
    "validation": {
      "years": [
        2022,
        2023
```

```
      ],
      "windows": 1041,
      "storms": 46,
      "native_pressure_windows": 100,
      "first_issue_utc": "2022-03-31T00:00:00Z",
      "last_issue_utc": "2023-12-17T18:00:00Z",
      "earliest_history_utc": "2022-03-29T00:00:00Z",
      "latest_target_utc": "2023-12-22T18:00:00Z"
    },
    "test": {
      "years": [
        2024,
        2025
      ],
      "windows": 1195,
      "storms": 56,
      "native_pressure_windows": 100,
      "first_issue_utc": "2024-05-22T00:00:00Z",
      "last_issue_utc": "2025-12-02T12:00:00Z",
      "earliest_history_utc": "2024-05-20T00:00:00Z",
      "latest_target_utc": "2025-12-07T12:00:00Z"
    }
  },
  "whole_storm_splits": true,
  "boundary_crossing_windows_excluded": true,
  "history_leads_hours": [
    -48,
    -42,
    -36,
    -30,
    -24,
    -18,
    -12,
    -6,
    0
  ],
  "target_leads_hours": [
    6,
    12,
    18,
    24,
    30,
    36,
    42,
    48,
    54,
    60,
    66,
    72,
    78,
    84,
    90,
    96,
    102,
    108,
    114,
    120
  ],
  "upstream_cache_coverage": {
    "track": {
      "first_issue_utc": "1980-01-03T00:00:00Z",
      "last_issue_utc": "2026-07-13T00:00:00Z"
    },
    "basin": {
      "first_analysis_utc": "1980-01-01T00:00:00Z",
      "last_analysis_utc": "2026-03-17T18:00:00Z"
    }
  },
  "cutoff_interpretation": "Fitting through 2021; validation through 2023; original test through 2025. Newer cache/live records are not gradient fitting.",
  "inference_performed": false,
  "model_weights_modified": false,
  "training_data_modified": false,
  "audit_scope": "All six derived archive file hashes, release manifest identity and loader-equivalent split/patch membership. Not a new raw-download or operational-availability audit."
}
```

B27. models/trackformer_1_2_field/README.md

11,984 bytes | SHA-256: cb4f3f1182547bb1ca19f7dc1d3c9e5aa292cec51b739b94c418065d211f1423

```
# Trackformer 1.2 field-model inference

The released pressure-field model predicts Western Pacific storm tracks, central pressure and spatial MSLP through +120 h. This is the inference reference; see the [model announcement](../../README.md) for capabilities, the pressure-map showcase and development results.

This folder contains the exact source, input contract and inference-only weights for the research model in the [main README](../../README.md). `model.py`, `baseline_model.py` and `v165_base.py` are source-identical to the selected training implementation. The weight file is distributed via the [GitHub release](https://github.com/yu314-coder/typhoon-predict/releases/tag/trackformer-1.2) and [Hugging Face](https://huggingface.co/euler314/typhoon-predict/tree/main/models/trackformer_1_2_field), not Git.

**Training-data cutoff:** fitting and weather normalization use **2000–2021**; validation/checkpoint selection uses **2022–2023**; the original test partition uses **2024–2025**. Inputs come from IBTrACS-derived storm records, NOAA PSL NCEP/NCAR Reanalysis basin fields, ARCO-ERA5 regional pressure and static land/elevation. Newer 2026 archive/live inputs are inference, not retraining. [Sources and exact split coverage](../../docs/trackformer_1_2_training_data.md) · [Verified training-data receipt](../../evaluation/training_data/trackformer_1_2_provenance.json).

Use Python with PyTorch and NumPy. The export environment used PyTorch 2.13.0; the source was evaluated on macOS MPS. Example:

```bash
python models/trackformer_1_2_field/predict.py causal_issue_packet.npz forecast.npz --device cpu
```

The `.npz` packet must contain exactly these arrays, with no object/pickle content:

The nine history timestamps span **−48 to 0 h**, including both endpoints. This is the release's fixed two-day input format, not evidence that two days outperforms one or three. The eight basin channels are MSLP, HGT500, U850, V850, U500, V500, U200 and V200: each u/v pair represents **both horizontal wind direction and speed**. Static coordinates, land fraction and elevation remain part of the actual input contract below. See the [plain-language Chinese report](../../docs/research_report_zh.md) for the research motivation and household-computer equipment.

Key	Shape	Meaning
`global_history`	`(9, 8, 25, 33)`	Past/issue-time basin analyses in manifest channel order, normalized with manifest means/stds.
`regional_history`	`(9, 1, 121, 121)`	Native MSLP patches when present, otherwise benchmark's coarse fallback; normalized using MSLP statistics.
`global_static`	`(4, 25, 33)`	Latitude/90, longitude/180−1, land fraction, elevation metres/8000.
`regional_static`	`(4, 121, 121)`	Same four static channels on the regional grid.
`detail_available`	`(1,)`	One when native history exists, otherwise zero.
`center`	`(2,)`	Issue-time latitude and longitude in degrees.
`motion`	`(2,)`	Previous six-hour east/north displacement in km.
`issue_intensity`	`(2,)`	Issue-time maximum wind in knots and central pressure in hPa.
`issue_mask`	`(2,)`	Validity flags for issue-time wind and pressure.
`issue_time_ns`	scalar	Forecast issue timestamp in Unix nanoseconds.
`history_time_ns`	`(9,)`	Nine consecutive six-hour timestamps ending at issue time.

Output `.npz` contains twenty exact +6, +12, …, +120-hour forecasts. Pressure arrays are physical hPa. The moving core is exported by default, not just used internally for a central-pressure number:

Output	Shape	Meaning
`basin_mslp_hpa`	`(20,25,33)`	Unchanged coarse Western Pacific pressure fields.
`basin_latitude_deg`, `basin_longitude_deg`	`(25,33)` each	Basin geography from the actual input grid.
`regional_mslp_hpa`	`(20,121,121)`	Unchanged fixed issue-relative pressure composites.
`regional_latitude_deg`, `regional_longitude_deg`	`(121,121)` each	Fixed regional geography.
`regional_valid`	`(20,121,121)`	Original model domain-support masks; not a guarantee of moving-core detail throughout this fixed patch.
`core_mslp_hpa`	`(20,65,65)`	Actual learned moving-core fields, converted from model normalization to hPa.
`core_latitude_deg`, `core_longitude_deg`	`(20,65,65)` each	Original moving-frame geographic coordinates at every forecast lead.
`core_valid`	`(20,65,65)`	Original moving-core coverage masks. Mask false cells even when their stored pressure is finite.
`track_lat_lon`, `central_pressure_hpa`, `track_valid`	`(20,2)`, `(20,)`, `(20,)`	Unchanged associated track and bilinear core-pressure readout, with domain support.
`issue_time_ns`, `valid_time_ns`	scalar, `(20,)`	UTC Unix nanoseconds for the issue and each exact forecast lead.
`issue_center_lat_lon`, `member_count`	`(2,)`, scalar	Initialization location and the honest count of one clean member.
`pressure_export_json`	string scalar	Export schema, units, frozen checkpoint/weight hashes, reconstruction and coverage policy.

No field is shifted onto the forecast or observed route, and no central-pressure scalar is inserted into a display. The moving-core information spacing is **20 km computational reconstruction**, not new native high-resolution observations. Coordinates and masks must travel with the fields; an invalid cell is not a zero-pressure forecast. `central_pressure_hpa` is sampled from this same moving field at the associated centre, so it need not equal a discrete cell minimum or a labelled contour level.
```

```
51
52  ### Render a pressure map directly
53
54  Install Matplotlib in your inference environment only if you want PNG rendering. NumPy and PyTorch suffice for
    the numerical export.
55
56  ```bash
57  python models/trackformer_1_2_field/predict.py causal_issue_packet.npz forecast.npz --device cpu --pressure-map
    pressure_120h.png --map-lead 120 --isobar-interval 4
58  ```
59
60  The optional image shows the whole Western Pacific basin and the actual moving core side by side, on one
    pressure colour scale: **blue low / red high**. It keeps original geography and masks unsupported core cells. To
     draw another saved lead without repeating inference:
61
62  ```bash
63  python models/trackformer_1_2_field/plot_pressure.py forecast.npz pressure_24h.png --lead 24 --interval 2
64  ```
65
66  All original track, central-pressure, basin/regional and auxiliary-wind outputs retain their existing
    definitions. The exporter verifies the downloaded weight SHA-256 against `manifest.json`; the learned model
    modules, weights and forward equations are unchanged.
67
68  It also exposes the existing `maximum_wind_auxiliary_kt` scalar and experimental
69  pressure-derived `pressure_wind_estimate_kt`, `rmw_estimate_km` and
70  `r34_estimate_km` / `r50_estimate_km` / `r64_estimate_km`. Every pressure-derived
71  array has a matching `_valid` mask: zero padding where the mask is false is
72  **not a zero wind or radius forecast**. `wind_estimation_json` is a JSON string
73  with assumptions, member counts and unavailable reasons, not pickled objects.
74  The auxiliary wind also has `maximum_wind_auxiliary_kt_valid`, a numerical/domain
75  validity mask, not a claim of calibrated wind skill.
76  These are uncalibrated circular-equivalent ocean estimates, not resolved
77  surface winds, agency-compatible sustained winds or quadrant radii. See
78  [the estimation method and validation requirements](WIND_ESTIMATION.md).
79  No model weights, learned architecture or existing benchmark results changed.
80
81  **Causality:** Never populate arrays from positive-lead analysis, later best track, agency forecast or future
    pressure field. The wrapper checks timestamp order and shapes, but cannot authenticate the upstream archive. Do
    not silently replace missing values with future data. This is not a raw-weather downloader or live warning
    service.
82
83  The published 50-member mean is a separate evaluation policy: 50 deterministic seeds perturb only normalized
    historical basin/regional fields with smooth zero-centred noise, then average routes, central pressures and
    common-grid fields. The weights are the same for every member; this is **not 50 separately trained models**.
    Issue-time centre, motion, intensity, masks and static geography are not perturbed. `predict.py` computes one
    clean forecast; do not label it a 50-member mean. Neither this member count nor the 48-hour history has been
    established as optimal by a matched ablation; spread is not automatically calibrated uncertainty.
84
85  **Report's matched development evaluation:** route uses the frozen **1,473 daily starts / 270 equal-weight
    storms**, +6 to +120 h. Central pressure uses **40 of those 270 storms**, **134 common starts / 2,637 valid lead
     records**, comparing each model separately with JMA at identical valid times. This is an inherited
    common-scoring subset, not evidence that all other storms lack pressure observations. Missing labels and
    unsupported outputs are not zero-scored. TrackFormer 1.2's mean route error is **471.20 km**, +120 h error
    **1031.63 km**, six-hour direction error **34.96°**, and JMA central-pressure MAE **12.84 hPa**.
    Central-pressure error is not whole-map error. See the [report figures](../../docs/research_report_zh.md),
    [frozen common scores](../../evaluation/released_daily/released_daily_benchmark.json), [pressure
    protocol](../../docs/intensity_benchmark.md) and [detailed evaluation
    notes](../../docs/trackformer_1_2_evaluation.md).
86
87  **Completed DeepMind comparison:** Google's official **WeatherNext Cyclones Mini `<2024`**, run with
    **WeatherNext software v0.3.0**, completed all **1,473 daily starts / 270 storms** on an **NVIDIA GeForce RTX
    3070 using CUDA**. This is the 1° Mini checkpoint trained through 2023, not the full-sized WeatherNext model.
    Mini uses one seeded member; TrackFormer 1.2 uses a 50-member mean. Mini's report values are **498.37 km** mean
    track error, **1031.91 km** at +120 h, **39.76°** direction error and **10.20 hPa** JMA central-pressure MAE on
    the **134 starts / 40 storms / 2,637 common valid records**. The combined cohort favours 1.2 for position and
    heading, but Mini has lower pressure MAE. Mini also has lower recent-only track error: **258.7 versus 469.4 km**
     on the strict 2025–2026 subset (61 starts / 18 storms). Different input/member policies and prior case
    inspection prevent general superiority, equal-compute or untouched-holdout claims. [Completed results and model
    identity](../../docs/deepmind_daily_benchmark.md) · [CUDA completion
    receipt](../../evaluation/deepmind_daily/verification.json) · [Independent publication
    audit](../../evaluation/deepmind_daily/publication_audit.json). No older benchmark's DeepMind scores were
    substituted.
88
89  **Rainfall and radius scope:** ongoing Taiwan rainfall work is separate from these weights. Released 1.2 has no
    rainfall head or humidity input. The report's three R34 diagrams are **observed IBTrACS quadrant radii**, not
    native 1.2 radius forecasts. Experimental pressure-derived radius outputs described above remain separately
    labelled diagnostics.
90
91  **Pressure-display coverage:** `regional_mslp_hpa` remains the original fixed issue-relative composite. Use the
    separately exported `core_mslp_hpa` and its per-lead coordinates/mask when the storm moves away from that fixed
    patch. Do not extend it by inventing a vortex or moving it onto a route. If the moving core leaves supported
    basin geography, the corresponding masked cells remain unavailable. For an ensemble, register each physical
    member field onto a common geographic grid **before** averaging; do not average moving-frame array indices. The
    Mangkhut film uses that separate 50-member policy. Older fixed-patch films retain their original data in [the
    showcase archive](../../docs/showcase_archive.md). The automatic History archive and its separate core recovery
    expose geography and coverage through [the public
```

```
API](https://trackformer-weatherlab.rudin-euler-8253.chatgpt.site/history-api).
```

B28. models/trackformer_1_2_field/WIND_ESTIMATION.md

2,996 bytes | SHA-256: e3bceb94600872521ec983affff6c8a5ca5b0a53774bd91fcb4f342a5c53c64d

```
# Experimental wind and circular-radius diagnostics

The frozen 1.2 weights and neural architecture are unchanged. Maximum wind is
the existing learned auxiliary `vmax` output, in knots, not a resolved wind map.
Its averaging period is not harmonized to JMA, and skill has not been established
by the route/pressure benchmark. Do not treat this product as operational advice.

`wind_estimation.py` optionally diagnoses each model-generated pressure member:

1. Sample complete circular rings about that member's predicted centre, no more
   finely than the source/reconstruction grid, out to at most 480 km.
2. Compute the azimuthal-mean radial pressure gradient (hPa converted to Pa).
3. Solve `Vg^2/r + |f| Vg = (1/rho) dp/dr`, with air density 1.15 kg/m3, then
   apply a fixed 0.8 ocean surface-reduction assumption. This is uncalibrated;
   motion, land friction, asymmetry, boundary-layer jets and averaging periods
   are not represented. No future observations or official routes are inputs.
4. Report a resolved gradient-wind peak/RMW and outward 34/50/64-kt crossings.
   These are **circular equivalent radius estimates**, not quadrant radii and
   not isobar radii. Coarse (>50 km) source grids, incomplete rings, unresolved
   peaks and missing crossings produce nulls, not invented zeros or edge radii.
5. Diagnose members individually, then summarize. A mean is supplied only when
   every member has that estimate. Valid counts and p10/p90 describe spread;
   they are not calibrated confidence intervals. Never diagnose a mean map and
   call it a mean of 50 radius predictions.

The 20-km moving core is a learned reconstruction. A resampled 0.25-degree
regional image does not add resolved observations. Even an available estimate
therefore remains experimental. A single gridded basin mean cannot reconstruct
the missing member diagnostics. Legacy forecasts without saved member/core
outputs retain unavailable estimates instead of being rerun or relabelled.

For the saved 50-member regional composite, core origins were not archived.
That release's core spans +/-640 km and centre association is restricted to
300 km about its origin. Consequently this read-only legacy exporter samples
only rings within 300 km of each predicted centre, conservatively inside the
core footprint; a missing crossing beyond that limit remains unavailable.
New forecasts use the actual moving-core coordinates and their valid masks.

Defaults are fixed physical assumptions, not fitted to showcase outcomes. Before
claiming skill, freeze eligible whole-storm held-out cases; score +6 to +120 h
maximum winds and agency-compatible wind radii separately, with coverage,
unresolved cases, land/ocean breakdowns and whole-storm uncertainty. No such
validation or calibration is claimed by this implementation.

References: [NOAA PaHM theory and limitations](https://noaa-ocs-modeling.github.io/PaHM/pahm_manual.pdf)
and [Chavas, Reed and Knaff (2017)](https://doi.org/10.1038/s41467-017-01546-9).
```

B29. models/trackformer_1_2_field/manifest.json

5,497 bytes | SHA-256: 35b0988394c17851625ec32b53b14d2af5ee8d7e29b147c1351aecdcb487a178


```
{
  "public_version": "1.2",
  "release_status": "research_candidate_not_operational",
  "source_checkpoint_version": "1.2.73",
  "source_checkpoint_epoch": 4,
  "architecture": "v173-moving-core-multiscale-attention",
  "source_checkpoint_sha256": "f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0",
  "inference_weights_sha256": "db49f36e85a3766defc4c172746897a1f783705d1ce8e6f9dfb8e87ae1d902cb",
  "dataset_sha256": "04fd0d30620137aad20173dcd59e22da2a6c20fae2f17de593e91a59517459c1",
  "source_module_sha256": {
    "model.py": "5dab06c20224cf48c51302122455ee7e16948f977bf4b760fd8aff5d443ab0bc",
    "baseline_model.py": "6c05941fd93baaca04508877edc69f8f0394b70e0a64e734ef0d2c1e8832246a",
    "v165_base.py": "5c9473993a19d0fed00055494b5c1f50c1006aaf46551ff50145f67048c3374a"
  },
  "data_contract": {
    "channels": [
      "mslp",
      "hgt500",
      "uwnd850",
      "vwnd850",
      "uwnd500",
      "vwnd500",
      "uwnd200",
      "vwnd200"
    ],
    "units": [
      "hPa",
      "m",
      "m/s",
      "m/s",
      "m/s",
      "m/s",
      "m/s",
      "m/s"
    ],
    "global_lat": [
      60.0,
      57.5,
      55.0,
      52.5,
      50.0,
      47.5,
      45.0,
      42.5,
      40.0,
      37.5,
      35.0,
      32.5,
      30.0,
      27.5,
      25.0,
      22.5,
      20.0,
      17.5,
      15.0,
      12.5,
      10.0,
      7.5,
      5.0,
      2.5,
      0.0
    ],
    "global_lon": [
      100.0,
      102.5,
      105.0,
      107.5,
      110.0,
      112.5,
      115.0,
      117.5,
      120.0,
      122.5,
      125.0,
      127.5,
      130.0,
      132.5,
      135.0,
      137.5,
      140.0,
      142.5,
      145.0,
```

```
        147.5,
        150.0,
        152.5,
        155.0,
        157.5,
        160.0,
        162.5,
        165.0,
        167.5,
        170.0,
        172.5,
        175.0,
        177.5,
        180.0
      ],
      "regional_lat": [
        15.0,
        14.75,
        14.5,
        14.25,
        14.0,
        13.75,
        13.5,
        13.25,
        13.0,
        12.75,
        12.5,
        12.25,
        12.0,
        11.75,
        11.5,
        11.25,
        11.0,
        10.75,
        10.5,
        10.25,
        10.0,
        9.75,
        9.5,
        9.25,
        9.0,
        8.75,
        8.5,
        8.25,
        8.0,
        7.75,
        7.5,
        7.25,
        7.0,
        6.75,
        6.5,
        6.25,
        6.0,
        5.75,
        5.5,
        5.25,
        5.0,
        4.75,
        4.5,
        4.25,
        4.0,
        3.75,
        3.5,
        3.25,
        3.0,
        2.75,
        2.5,
        2.25,
        2.0,
        1.75,
        1.5,
        1.25,
        1.0,
        0.75,
        0.5,
        0.25,
        0.0,
        -0.25,
        -0.5,
        -0.75,
        -1.0,
        -1.25,
        -1.5,
        -1.75,
        -2.0,
        -2.25,
```

```
        -2.5,
        -2.75,
        -3.0,
        -3.25,
        -3.5,
        -3.75,
        -4.0,
        -4.25,
        -4.5,
        -4.75,
        -5.0,
        -5.25,
        -5.5,
        -5.75,
        -6.0,
        -6.25,
        -6.5,
        -6.75,
        -7.0,
        -7.25,
        -7.5,
        -7.75,
        -8.0,
        -8.25,
        -8.5,
        -8.75,
        -9.0,
        -9.25,
        -9.5,
        -9.75,
        -10.0,
        -10.25,
        -10.5,
        -10.75,
        -11.0,
        -11.25,
        -11.5,
        -11.75,
        -12.0,
        -12.25,
        -12.5,
        -12.75,
        -13.0,
        -13.25,
        -13.5,
        -13.75,
        -14.0,
        -14.25,
        -14.5,
        -14.75,
        -15.0
      ],
      "regional_lon": [
        -15.0,
        -14.75,
        -14.5,
        -14.25,
        -14.0,
        -13.75,
        -13.5,
        -13.25,
        -13.0,
        -12.75,
        -12.5,
        -12.25,
        -12.0,
        -11.75,
        -11.5,
        -11.25,
        -11.0,
        -10.75,
        -10.5,
        -10.25,
        -10.0,
        -9.75,
        -9.5,
        -9.25,
        -9.0,
        -8.75,
        -8.5,
        -8.25,
        -8.0,
        -7.75,
        -7.5,
        -7.25,
        -7.0,
```

```
        -6.75,
        -6.5,
        -6.25,
        -6.0,
        -5.75,
        -5.5,
        -5.25,
        -5.0,
        -4.75,
        -4.5,
        -4.25,
        -4.0,
        -3.75,
        -3.5,
        -3.25,
        -3.0,
        -2.75,
        -2.5,
        -2.25,
        -2.0,
        -1.75,
        -1.5,
        -1.25,
        -1.0,
        -0.75,
        -0.5,
        -0.25,
        0.0,
        0.25,
        0.5,
        0.75,
        1.0,
        1.25,
        1.5,
        1.75,
        2.0,
        2.25,
        2.5,
        2.75,
        3.0,
        3.25,
        3.5,
        3.75,
        4.0,
        4.25,
        4.5,
        4.75,
        5.0,
        5.25,
        5.5,
        5.75,
        6.0,
        6.25,
        6.5,
        6.75,
        7.0,
        7.25,
        7.5,
        7.75,
        8.0,
        8.25,
        8.5,
        8.75,
        9.0,
        9.25,
        9.5,
        9.75,
        10.0,
        10.25,
        10.5,
        10.75,
        11.0,
        11.25,
        11.5,
        11.75,
        12.0,
        12.25,
        12.5,
        12.75,
        13.0,
        13.25,
        13.5,
        13.75,
        14.0,
        14.25,
        14.5,
```

```
        14.75,
        15.0
      ],
      "normalization": {
        "mean": [
          1012.6534350724769,
          5702.17255505101,
          0.6556878249021729,
          0.07710723631313336,
          6.437907875944145,
          -0.22113481896089796,
          14.724588065342962,
          -0.2013532977844505
        ],
        "std": [
          8.265883617782512,
          233.83147357071002,
          7.441798736924629,
          5.2934176921269875,
          13.162218957604635,
          7.578773743787778,
          22.626341660579083,
          11.05678874016786
        ],
        "fit_split": "train",
        "end_year": 2021
      }
    }
  }
```

B30. models/trackformer_1_2_field/plot_pressure.py

4,877 bytes | SHA-256: b684808e13f38c7b14fb138c4e09fa7bf83ac4ceb3bbd1cb6c2b3a8cc120ecd4

```
"""Plot the released model's saved geographic pressure fields without inference.

The basin remains at 2.5 degrees. The moving core is the model's learned 20-km
reconstruction, not extra native observations. Invalid coverage stays masked.
"""
from __future__ import annotations

import argparse
from datetime import datetime, timezone
from pathlib import Path

import numpy as np


def render_pressure_map(data, output, lead=120, interval=4):
    if not np.isfinite(interval) or interval <= 0:
        raise ValueError("Isobar interval must be positive and finite")
    indices = np.flatnonzero(np.asarray(data["lead_hours"]) == lead)
    if len(indices) != 1:
        raise ValueError("Choose a saved +6 to +120-hour forecast lead")
    required = {"core_mslp_hpa", "core_latitude_deg", "core_longitude_deg", "core_valid",
                "basin_latitude_deg", "basin_longitude_deg", "track_valid", "valid_time_ns", "member_count"}
    if not required.issubset(data):
        raise ValueError("This renderer requires the moving-core pressure export v2")
    if int(np.asarray(data["member_count"])) != 1:
        raise ValueError("This renderer accepts the clean one-member export, not unregistered ensemble cores")
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    i = int(indices[0])
    basin = np.ma.masked_invalid(data["basin_mslp_hpa"][i])
    core = np.ma.masked_where(~np.asarray(data["core_valid"][i], bool), data["core_mslp_hpa"][i])
    core = np.ma.masked_invalid(core)
    finite = np.concatenate((basin.compressed(), core.compressed()))
    if not finite.size:
        raise ValueError("No supported pressure field for this lead")
    low = np.floor(finite.min() / interval) * interval
    high = max(low + interval, np.ceil(finite.max() / interval) * interval)
    levels = np.arange(low, high + interval * .5, interval)
    fig, axes = plt.subplots(1, 2, figsize=(12, 5), layout="constrained")
    plots = (
        (basin, data["basin_longitude_deg"], data["basin_latitude_deg"], "Western Pacific · 2.5° basin field"),
        (core, data["core_longitude_deg"][i], data["core_latitude_deg"][i], "Moving core · learned 20-km reconstruction"),
    )
    route = np.asarray(data["track_lat_lon"][:i + 1])
    valid_route = np.asarray(data["track_valid"][:i + 1], bool)
    for ax, (pressure, lon, lat, title) in zip(axes, plots):
        ax.set_title(title, fontsize=11)
        ax.set_xlabel("Longitude °E")
        ax.set_ylabel("Latitude °N")
        if pressure.count():
            image = ax.pcolormesh(lon, lat, pressure, cmap="RdBu_r", vmin=low, vmax=high,
                                  shading="auto", rasterized=True)
            if pressure.max() > pressure.min():
                lines = ax.contour(lon, lat, pressure, levels=levels, colors="#35424d", linewidths=.65)
                ax.clabel(lines, levels[::2], fmt="%d", fontsize=7)
        else:
            ax.text(.5, .5, "Moving core outside supported basin coverage", ha="center", va="center", transform=ax.transAxes)
        ax.plot(np.where(valid_route, route[:, 1], np.nan), np.where(valid_route, route[:, 0], np.nan),
                color="#ad287b", linewidth=1.6)
        if valid_route[i] and (ax is axes[0] or bool(np.asarray(data["core_valid"][i]).any())):
            ax.scatter(route[i, 1], route[i, 0], s=25, c="#ad287b", edgecolors="white", zorder=4)
        ax.grid(alpha=.2)
    axes[0].set_xlim(100, 180)
    axes[0].set_ylim(0, 60)
    axes[1].set_xlim(float(np.min(plots[1][1])), float(np.max(plots[1][1])))
    axes[1].set_ylim(float(np.min(plots[1][2])), float(np.max(plots[1][2])))
    valid_time = datetime.fromtimestamp(int(np.asarray(data["valid_time_ns"])[i]) / 1e9, timezone.utc)
    value = (f"{float(data['central_pressure_hpa'][i]):.1f} hPa" if valid_route[i] else "unavailable outside domain")
    fig.suptitle(f"Trackformer 1.2 · +{lead:03d} h · {valid_time:%Y-%m-%d %H:%M UTC}\n"
                 f"1 clean member · central pressure {value}", fontsize=12)
    fig.colorbar(image, ax=axes, label="Model MSLP (hPa) · blue low / red high", shrink=.85)
    fig.supxlabel(f"{interval:g} hPa isobars · original model geography · unsupported core cells masked; no pressure or route shifting", fontsize=9)
    fig.savefig(Path(output), dpi=160)
    plt.close(fig)


```

```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("forecast", type=Path)
    parser.add_argument("output", type=Path)
    parser.add_argument("--lead", type=int, default=120, choices=range(6, 121, 6))
    parser.add_argument("--interval", type=float, default=4)
    args = parser.parse_args()
    with np.load(args.forecast, allow_pickle=False) as saved:
        render_pressure_map({key: saved[key] for key in saved.files}, args.output, args.lead, args.interval)


if __name__ == "__main__":
    main()
```

B31. models/trackformer_1_2_field/predict.py

9,080 bytes | SHA-256: 5f1dd492a8eb4f2747b6165279449aebb7804a651b5b970e3cae25d01e5837bc

```
"""Trackformer 1.2 research inference from a causal, normalized issue packet.

This wrapper deliberately does not fetch or construct weather inputs. The caller
must supply past/issue-time analyses on the exact grids in manifest.json.
"""
from __future__ import annotations

import argparse
import hashlib
import json
from pathlib import Path

import numpy as np
import torch

from model import CoreForecaster


INPUT_SHAPES = {
    "global_history": (9, 8, 25, 33),
    "regional_history": (9, 1, 121, 121),
    "global_static": (4, 25, 33),
    "regional_static": (4, 121, 121),
    "detail_available": (1,),
    "center": (2,),
    "motion": (2,),
    "issue_intensity": (2,),
    "issue_mask": (2,),
}

PRESSURE_EXPORT_SCHEMA = "trackformer-1.2-pressure-fields-v2"


def export_pressure_fields(outputs, inputs, contract, issue_time_ns):
    """Capture existing forward outputs; never relocate or fit a display vortex."""
    scale = float(contract["normalization"]["std"][0])
    offset = float(contract["normalization"]["mean"][0])
    global_static = inputs["global_static"][0].detach().cpu().numpy()
    regional_static = inputs["regional_static"][0].detach().cpu().numpy()
    leads = np.arange(6, 121, 6, dtype=np.int16)
    if len(outputs) != len(leads):
        raise ValueError("Detailed pressure export requires all twenty forecast leads")
    return {
        "core_mslp_hpa": torch.stack([
            o["core"][0, 0] * scale + offset for o in outputs]).cpu().numpy(),
        "core_latitude_deg": torch.stack([o["core_lat"][0] for o in outputs]).cpu().numpy(),
        "core_longitude_deg": torch.stack([o["core_lon"][0] for o in outputs]).cpu().numpy(),
        "core_valid": torch.stack([o["core_valid"][0, 0] for o in outputs]).cpu().numpy().astype(bool),
        "basin_latitude_deg": global_static[0] * 90,
        "basin_longitude_deg": (global_static[1] + 1) * 180,
        "regional_latitude_deg": regional_static[0] * 90,
        "regional_longitude_deg": (regional_static[1] + 1) * 180,
        "regional_valid": torch.stack([
            o["regional_valid"][0, 0] for o in outputs]).cpu().numpy().astype(bool),
        "track_valid": torch.stack([o["track_valid"][0] for o in outputs]).cpu().numpy().astype(bool),
        "issue_center_lat_lon": inputs["center"][0].detach().cpu().numpy(),
        "issue_time_ns": np.asarray(issue_time_ns, dtype=np.int64),
        "valid_time_ns": issue_time_ns + leads.astype(np.int64) * (3600 * 10**9),
        "member_count": np.asarray(1, dtype=np.int16),
        "core_information_spacing_km": np.asarray(20, dtype=np.float32),
    }


def load_packet(path: Path) -> dict[str, np.ndarray]:
    with np.load(path, allow_pickle=False) as packet:
        if set(packet.files) != set(INPUT_SHAPES) | {"issue_time_ns", "history_time_ns"}:
            raise ValueError("Packet keys must match the documented causal input schema exactly")
        issue = int(packet["issue_time_ns"])
        history = np.asarray(packet["history_time_ns"], dtype=np.int64)
        if history.shape != (9,) or history[-1] != issue or np.any(history > issue):
            raise ValueError("History must contain nine times ending at issue time; future inputs forbidden")
        if not np.array_equal(np.diff(history), np.full(8, 6 * 3600 * 10**9)):
            raise ValueError("History must use consecutive six-hour analyses")
        arrays = {}
        for key, shape in INPUT_SHAPES.items():
            value = np.asarray(packet[key], dtype=np.float32)
            if value.shape != shape or not np.isfinite(value).all():
                raise ValueError(f"{key}: expected finite array of shape {shape}, got {value.shape}")
            arrays[key] = value[None]
        return arrays


```

```python
def forecast(packet: Path, model_dir: Path, device: str = "cpu") -> dict[str, np.ndarray]:
    from wind_estimation import diagnose_outputs, summarize_members, KEYS
    metadata = json.loads((model_dir / "manifest.json").read_text())
    if metadata["public_version"] != "1.2":
        raise ValueError("Unexpected public model version")
    contract = metadata["data_contract"]
    model = CoreForecaster(contract).to(device).eval()
    weights = model_dir / "weights.pt"
    with weights.open("rb") as stream:
        weight_sha256 = hashlib.file_digest(stream, "sha256").hexdigest()
    if weight_sha256 != metadata["inference_weights_sha256"]:
        raise ValueError("Weights do not match the released 1.2 manifest")
    state = torch.load(weights, map_location="cpu", weights_only=True)
    model.load_state_dict(state, strict=True)
    inputs = {key: torch.from_numpy(value).to(device) for key, value in load_packet(packet).items()}
    with torch.inference_mode():
        running = model.initial(inputs)
        outputs = []
        for _ in range(20):
            running, output = model.step(running)
            outputs.append(output)
    scale = float(contract["normalization"]["std"][0])
    offset = float(contract["normalization"]["mean"][0])
    result = {
        "lead_hours": np.arange(6, 121, 6, dtype=np.int16),
        "track_lat_lon": torch.stack([o["center"][0] for o in outputs]).cpu().numpy(),
        "central_pressure_hpa": torch.stack([o["pressure"][0] for o in outputs]).cpu().numpy(),
        "basin_mslp_hpa": torch.stack([o["global"][0, 0] for o in outputs]).cpu().numpy() * scale + offset,
        "regional_mslp_hpa": torch.stack([o["regional"][0, 0] for o in outputs]).cpu().numpy() * scale + offset,
        "maximum_wind_auxiliary_kt": torch.stack([o["vmax"][0] for o in outputs]).cpu().numpy(),
    }
    with np.load(packet, allow_pickle=False) as source:
        issue_time_ns = int(source["issue_time_ns"])
    result.update(export_pressure_fields(outputs, inputs, contract, issue_time_ns))
    diagnostics = [summarize_members(diagnose_outputs(o, contract)) for o in outputs]
    result['maximum_wind_auxiliary_kt_valid'] = np.asarray(
        [d['estimates']['maximum_wind_auxiliary_kt']['mean'] is not None for d in diagnostics], dtype=bool)
    for key in KEYS[1:]:
        values = [d['estimates'][key]['mean'] for d in diagnostics]
        # Numeric zero is storage padding only. Consumers MUST use this mask.
        result[key] = np.asarray([0 if v is None else v for v in values], dtype=np.float32)
        result[key + '_valid'] = np.asarray([v is not None for v in values], dtype=bool)
    if not all(np.isfinite(value).all() for value in result.values()):
        raise ValueError("Non-finite forecast")
    result['wind_estimation_json'] = np.asarray(json.dumps(diagnostics, allow_nan=False))
    result['pressure_export_json'] = np.asarray(json.dumps({
        "schema": PRESSURE_EXPORT_SCHEMA,
        "public_version": "1.2",
        "architecture": metadata["architecture"],
        "inference_weights_sha256": weight_sha256,
        "source_checkpoint_sha256": metadata["source_checkpoint_sha256"],
        "members": 1,
        "units": {"pressure": "hPa", "latitude": "degrees_north", "longitude": "degrees_east"},
        "core_information_spacing_km": 20,
        "core_method": "unchanged learned moving pressure field; physical hPa with original geographic
coordinates",
        "native_high_resolution_observations": False,
        "native_detail_history_available": bool(inputs["detail_available"][0, 0].item()),
        "regional_grid": "fixed issue-relative composite; outside moving-core coverage only basin information
remains",
        "coverage_policy": "apply core_valid, regional_valid and track_valid; invalid finite storage is not a
supported forecast",
        "central_pressure_policy": "existing bilinear moving-core readout at the associated forecast centre; not
 an independently inserted scalar",
        "ensemble_policy": "one clean member; register physical fields on common geographic coordinates before
any ensemble average",
        "forecast_equations_changed": False,
    }, allow_nan=False))
    return result


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("packet", type=Path, help="Causal normalized .npz issue packet")
    parser.add_argument("output", type=Path, help="Output .npz path")
    parser.add_argument("--device", default="cpu", choices=("cpu", "mps", "cuda"))
    parser.add_argument("--pressure-map", type=Path, help="Optional PNG of basin and actual moving-core pressure
 (requires Matplotlib)")
    parser.add_argument("--map-lead", type=int, default=120, choices=range(6, 121, 6))
    parser.add_argument("--isobar-interval", type=float, default=4, help="Pressure contour interval in hPa")
    args = parser.parse_args()
    result = forecast(args.packet, Path(__file__).resolve().parent, args.device)
    np.savez_compressed(args.output, **result)
    print(f"Saved 20 six-hour forecasts to {args.output}")
    if args.pressure_map:
        from plot_pressure import render_pressure_map
```

```
        render_pressure_map(result, args.pressure_map, args.map_lead, args.isobar_interval)
        print(f"Saved pressure map to {args.pressure_map}")


if __name__ == "__main__":
    main()
```

B32. release_tools/export_field_1_2.py

2,424 bytes | SHA-256: 560ac05c598870a3612f2a972242bebeea3dcabbcd16cb413c620e2f1ecfd384

```
"""Export a verified training checkpoint as inference-only Trackformer 1.2 weights."""
from __future__ import annotations

import argparse
import hashlib
import json
from pathlib import Path

import torch


EXPECTED_SHA256 = "f194a23d3f91ea76ad776dfad942fabd669eeae8b3fd665815463095367e9ee0"
EXPECTED_ARCH = "v173-moving-core-multiscale-attention"


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(8 * 1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("checkpoint", type=Path)
    parser.add_argument("output_dir", type=Path)
    args = parser.parse_args()
    if sha256(args.checkpoint) != EXPECTED_SHA256:
        raise ValueError("Checkpoint SHA-256 does not match the selected release source")
    payload = torch.load(args.checkpoint, map_location="cpu", weights_only=False)
    if (payload.get("version"), payload.get("architecture"), payload.get("epoch")) != (
        "1.2.73", EXPECTED_ARCH, 4
    ):
        raise ValueError("Checkpoint identity does not match the selected release source")
    if len(payload.get("history", [])) != 4:
        raise ValueError("Expected four completed epochs in checkpoint history")
    args.output_dir.mkdir(parents=True, exist_ok=True)
    weights = args.output_dir / "weights.pt"
    torch.save(payload["model"], weights)
    source = args.output_dir
    manifest = {
        "public_version": "1.2",
        "release_status": "research_candidate_not_operational",
        "source_checkpoint_version": payload["version"],
        "source_checkpoint_epoch": payload["epoch"],
        "architecture": payload["architecture"],
        "source_checkpoint_sha256": EXPECTED_SHA256,
        "inference_weights_sha256": sha256(weights),
        "dataset_sha256": payload["dataset_sha256"],
        "source_module_sha256": {name: sha256(source / name) for name in (
            "model.py", "baseline_model.py", "v165_base.py")},
        "data_contract": payload["data_contract"],
    }
    (args.output_dir / "manifest.json").write_text(json.dumps(manifest, indent=2) + "\n")
    print(json.dumps({key: manifest[key] for key in (
        "public_version", "architecture", "inference_weights_sha256")}, indent=2))


if __name__ == "__main__":
    main()
```

B33. training/trackformer_1_2/source/data_preparation/requirements.txt

55 bytes | SHA-256: 54918b8f29330299e18b8924e9cd93d05807e7f8cd99b2efd4805e9be94003c1

```
numpy>=1.26
torch>=2.1
xarray
zarr
gcsfs
netCDF4
scipy
```

B34. training/trackformer_1_2/source/data_preparation/v164_dataset_contract.md

4,664 bytes | SHA-256: cfd838651e632aba0d2439ba202a59895bd4df22137185974df1d97f0e2525d4

```
# v1.2.64 Combined-Archive Training

## Active Workflow

The same `ai_train.ipynb` has two cells: version information, then fetching and preflight. It uses Google Drive only, installs no packages, and does not start training. The trainer remains `v164_global_regional_train.py` for command compatibility. Its active architecture is `wp-coarse8-native-pressure1-v164`, with 53,435,692 parameters. The old 31-channel global experiment and six-window pilot are not used.

Run in the Colab terminal after setup reports READY:

```sh
cd /content/v164_colab_bundle
python v164_global_regional_train.py 20 3 1
```

Numbers mean effective batch, workers, and CUDA AMP. Physical microbatch is one; workers are capped at two. Twelve epochs use rollout horizons 2, 4, 8, 12, then 20 steps. First two epochs train fields; later epochs add route and intensity objectives. Best selection starts at epoch five with the stable full-horizon objective. Validation is never training data; test is excluded from selection.

Data Coverage

- Old archives supply 13,949 train, 1,041 validation, and 1,195 test windows with complete six-hour context/target weather coverage inside the Western Pacific domain.
- Train years 2000-2021; validation 2022-2023; test 2024-2025. Whole storms and window boundaries cannot cross splits. Tip 1979 is excluded from all three.
- Eight 2.5-degree context variables: mean sea-level pressure, 500-hPa height, and u/v winds at 850/500/200 hPa. Domain: 0-60 N, 100-180 E. These are old NCEP reanalysis fields, not high-resolution ERA5.
- New native 0.25-degree ERA5 pressure patches provide 800/100/100 detailed windows within those splits. Each patch is 121x121 over 30 degrees, fixed at its observed issue-time center, with nine history and twenty target frames. These 1,000 windows are additional fine supervision, NOT the entire old corpus.
- Static geographic channels: latitude, longitude, land fraction and elevation. Fine pressure is never synthesized by enlarging the old coarse map.
- SST, ONI and fine upper-air variables are not consumed by this configuration; they are not silently substituted or claimed present.

Causality And Outputs

Inputs are analyses at -48,-42,...,0 hours, static geography, observed issue-time center/intensity and past six-hour motion. Future analysis fields and best-track positions/intensities are labels only. Issue-time retrospective best-track estimates are a historical hindcast assumption, not proof of real-time operational availability. Normalization uses only the training years. No official forecasts or future weather are model inputs.

The model autoregressively predicts eight basin fields and a native pressure patch when native history is available. No future recentering or route-painted vortex is used. Its track head uses generated basin pressure/winds/features. Fine data outside the parent domain receives an explicit missing-coverage mask, not extrapolated fields. Central pressure is a sampled generated field plus a separately learned subgrid correction; maximum wind is a scalar head, not a predicted 10m-wind map. No exact physical conservation is claimed.

The archive-wide training processes every eligible coarse window. Only the detailed subset trains the fine branch. A case lacking native pressure history can run the coarse branch but is not represented as a genuine high-resolution pressure forecast.

Storage And Resume

Drive folder: `typhoon_predict/data/v164_reuse` contains the manifest and bundle descriptor referencing existing archived data plus `data/v164_pressure_geography` parts. No large archives are duplicated in Drive. Total payload about 2.13 GB. Four independent Drive HTTP clients fetch parts concurrently. Existing verified files are reused; the manifest is published after complete-file checksums pass.

Checkpoints are written directly to `MyDrive/typhoon_predict/training/v164_reuse_pressure/last.pt` and `best.pt`, with history and run manifest. Resume restores the last completed epoch, optimizer, scaler, and RNG; an interrupted epoch restarts. Architecture and dataset hashes must match. A stale lock after runtime death requires checking that no other training process is active before removing it. Pilot checkpoints are incompatible and are not auto-loaded.

Verification Limits

Mac checks cover all archive hashes and split counts, real coarse and fine samples, tiny-model +120h finite rollouts, backward gradients, completed-epoch checkpoint resume, and full-width two-step MPS gradients. These are software checks, not model-skill validation. CUDA/T4 execution and a complete training run have not been tested; Colab was not accessed.
```

B35. training/trackformer_1_2/source/data_preparation/v164_download_pressure_geography.py

9,256 bytes | SHA-256: b8602cd2fba131a287dc9e25781c8f037d3325fec8a23203576916c2f7d2189c

```
"""Mac-only, resumable MSLP patches and geography. Not a full trainer dataset."""
import argparse
from collections import defaultdict
from concurrent.futures import ThreadPoolExecutor
import hashlib
import json
import os
from pathlib import Path
import sqlite3
import time

import numpy as np
import v164_prepare_data as prep


def make_plan(track):
    if prep.trainer.digest(track) != prep.TRACK_SHA:
        raise ValueError('Unexpected track archive')
    rng = np.random.default_rng(164)
    with np.load(track, allow_pickle=False) as z:
        times, ids = z['base_time'], z['storm_id'].astype(str)
        lat, lon = z['base_lat'], z['base_lon'] % 360
    first = {}
    for sid, ns in zip(ids, times):
        first[sid] = min(first.get(sid, 9999), prep.year(ns))
    rows = []
    for split, lo, hi, count in [('train', 2000, 2021, 800), ('validation', 2022, 2023, 100), ('test', 2024,
2025, 100)]:
        groups = defaultdict(list)
        for i, (sid, ns) in enumerate(zip(ids, times)):
            ns = int(ns)
            if not (lo <= first[sid] <= hi and lo <= prep.year(ns-8*prep.STEP)
                    and prep.year(ns+20*prep.STEP) <= hi and ns % prep.STEP == 0
                    and np.isfinite(lat[i]) and np.isfinite(lon[i])
                    and 0 < lat[i] < 60 and 100 < lon[i] < 180):
                continue
            groups[sid].append(i)
        for group in groups.values():
            rng.shuffle(group)
        storms = sorted(groups)
        rng.shuffle(storms)
        chosen = []
        while len(chosen) < count:
            before = len(chosen)
            for sid in storms:
                if groups[sid]:
                    chosen.append(groups[sid].pop())
                    if len(chosen) == count:
                        break
            if len(chosen) == before:
                raise ValueError(f'Insufficient eligible {split} windows')
        for i in chosen:
            rows.append({'storm_id': str(ids[i]), 'issue_ns': int(times[i]), 'split': split,
                         'center_lat': float(np.round(lat[i]*4)/4), 'center_lon': float(np.round(lon[i]*4)/4),
                         'center_source': 'retrospective issue-time best-track observation; no future
positions',
                         'track_archive_row': int(i)})
    plan = {'schema': 'v164-pressure-geography-v1', 'dataset_role': 'partial_inputs_not_training_ready',
            'source': prep.SOURCE, 'track_sha256': prep.TRACK_SHA, 'rows': rows,
            'lead_hours': list(range(-48, 121, 6)), 'shape': [len(rows), 29, 121, 121],
            'units': 'hPa', 'resolution_degrees': .25, 'fixed_issue_time_crops': True,
            'forecast_inputs': 'lead_hours <= 0 only', 'targets': 'lead_hours > 0 only',
            'split_policy': 'existing 2000-2021/2022-2023/2024-2025 policy, pending separately requested
changes',
            'hindcast_not_operational_availability_claim': True}
    plan['sha256'] = hashlib.sha256(json.dumps(plan, sort_keys=True).encode()).hexdigest()
    return plan


def patch_indices(row):
    lat = row['center_lat'] + 15 - np.arange(121)*.25
    lon = row['center_lon'] - 15 + np.arange(121)*.25
    return np.rint((90-lat)*4).astype(int), np.rint(lon*4).astype(int) % 1440


def patch_hash(a):
    return hashlib.sha256(np.ascontiguousarray(a).tobytes()).hexdigest()


def run(root, track, workers, max_frames=None):
```

```
    root.mkdir(parents=True, exist_ok=True)
    fd = os.open(root/'download.lock', os.O_CREAT | os.O_EXCL | os.O_WRONLY)
    os.write(fd, str(os.getpid()).encode())
    os.close(fd)
    db = None
    try:
        plan = make_plan(track)
        path = root/'plan.json'
        if path.exists() and json.loads(path.read_text()) != plan:
            raise ValueError('Existing download plan differs; refusing overwrite')
        prep.trainer.atomic_json(path, plan)
        shape = tuple(plan['shape'])
        output = root/'pressure_hpa.npy'
        a = np.lib.format.open_memmap(output, mode='r+' if output.exists() else 'w+', dtype='float32',
shape=shape)
        if a.shape != shape or a.dtype != np.float32:
            raise ValueError('Existing array shape/dtype differs')
        indices = [patch_indices(r) for r in plan['rows']]
        requests = defaultdict(list)
        for i, row in enumerate(plan['rows']):
            for j in range(29):
                requests[row['issue_ns'] + (j-8)*prep.STEP].append((i, j))
        db = sqlite3.connect(root/'progress.sqlite')
        db.execute('CREATE TABLE IF NOT EXISTS patches (i INTEGER, j INTEGER, sha TEXT, PRIMARY KEY(i,j))')
        verified = set()
        for i, j, sha in db.execute('SELECT i,j,sha FROM patches'):
            if patch_hash(a[i,j]) == sha:
                verified.add((i,j))
        pending = [(ns, slots) for ns, slots in sorted(requests.items()) if any(s not in verified for s in
slots)]
        print(json.dumps({'windows': len(plan['rows']), 'pressure_gib': a.nbytes/1024**3,
                          'unique_times': len(requests), 'remaining_times': len(pending),
                          'reused_verified_patches': len(verified), 'workers': workers}), flush=True)
        source = prep.Era5Source()
        geography = root/'geography.npz'
        if not geography.exists():
            ns = plan['rows'][0]['issue_ns']
            land = source.read('land_sea_mask', ns)
            height = source.read('geopotential_at_surface', ns)/9.80665
            if not np.isfinite(land).all() or not np.isfinite(height).all():
                raise ValueError('Invalid geography')
            with (root/'geography.pending').open('wb') as f:
                np.savez_compressed(f, latitude=source.lat, longitude=source.lon,
                                    land_fraction=land, elevation_m=height)
            (root/'geography.pending').replace(geography)
        with np.load(geography, allow_pickle=False) as z:
            if z['land_fraction'].shape != (721,1440) or z['elevation_m'].shape != (721,1440):
                raise ValueError('Geography grid mismatch')
        prep.trainer.atomic_json(root/'geography_manifest.json', {'sha256': prep.trainer.digest(geography),
            'source': prep.SOURCE, 'size': geography.stat().st_size, 'units': {'elevation_m': 'm',
'land_fraction': '1'}})
        def fetch(item):
            ns, slots = item
            pressure = source.read('mean_sea_level_pressure', ns)/100
            if not np.isfinite(pressure).all() or pressure.min() < 800 or pressure.max() > 1120:
                raise ValueError(f'Invalid MSLP at {ns}')
            return ns, [(i,j,pressure[indices[i][0][:,None],indices[i][1][None,:]]) for i,j in slots if (i,j)
not in verified]
        started = time.monotonic()
        initial = len(requests)-len(pending)
        selected = pending if max_frames is None else pending[:max_frames]
        # Process bounded groups: completed patches are committed before requesting more.
        done = 0
        with ThreadPoolExecutor(max_workers=workers) as pool:
            for offset in range(0, len(selected), workers):
                for ns, patches in pool.map(fetch, selected[offset:offset+workers]):
                    for i,j,values in patches:
                        a[i,j] = values
                    a.flush()
                    for i,j,_ in patches:
                        db.execute('INSERT OR REPLACE INTO patches VALUES (?,?,?)', (i,j,patch_hash(a[i,j])))
                        verified.add((i,j))
                    db.commit()
                    done += 1
                    elapsed = time.monotonic()-started
                    status = {'completed_times': initial+done, 'total_times': len(requests),
                              'verified_patches': len(verified), 'total_patches': shape[0]*shape[1],
                              'elapsed_s_this_run': round(elapsed,1),
                              'eta_s': round((len(pending)-done)*elapsed/done),
                              'last_time': str(np.datetime64(ns,'ns')), 'status': 'downloading'}
                    prep.trainer.atomic_json(root/'progress.json', status)
                    print(json.dumps(status), flush=True)
        if len(verified) == shape[0]*shape[1]:
            prep.trainer.atomic_json(root/'completion.json', {'status': 'pressure_and_geography_complete',
                'training_ready': False, 'plan_sha256': plan['sha256'],
                'pressure_sha256': prep.trainer.digest(output), 'geography_sha256':
```

```
    prep.trainer.digest(geography)})
            print('Pressure and geography verified complete. Other training inputs still required.', flush=True)
    finally:
        if db is not None:
            db.close()
        (root/'download.lock').unlink(missing_ok=True)


if __name__ == '__main__':
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('--output', type=Path,
default=Path('/Volumes/D/typhoon_predict/data/v164_pressure_geography'))
    p.add_argument('--track', type=Path,
default=Path('/Volumes/D/typhoon_predict/track_build/track_windows_v13.npz'))
    p.add_argument('--workers', type=int, choices=range(1,9), default=4)
    p.add_argument('--max-frames', type=int)
    args = p.parse_args()
    run(args.output, args.track, args.workers, args.max_frames)
```

B36. training/trackformer_1_2/source/data_preparation/v164_global_regional_train.py

52,581 bytes | SHA-256: c0808a1467695d2bc2ad8483aa4f4c6de1463628221024ac559f6204effe7978

```
#!/usr/bin/env python3
"""v1.2.64 Western Pacific coarse-context/native-pressure forecaster.

Run: python v164_global_regional_train.py 10 3 1
Uses verified old coarse archives plus new native 0.25-degree pressure patches.
Predicts eight coarse fields and one fine pressure field, not 31 fine variables.
The legacy 31-channel definitions below are not the active CLI architecture.
No route-centered vortex is painted. This is not a numerical conservation solver.
"""
from __future__ import annotations

import argparse
import hashlib
import json
import math
import os
from pathlib import Path
import random
import socket
import time
from contextlib import nullcontext

import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils.checkpoint import checkpoint
from torch.utils.data import Dataset, DataLoader

VERSION = "1.2.64"
ARCHITECTURE = "global-regional-field-first-v1"
ROOT = Path(__file__).resolve().parent
LEVELS = [850, 700, 500, 300, 200]
CHANNELS = ["mslp", "surface_pressure", "u10", "v10", "t2m", "sst"] + [
    f"{v}{p}" for p in LEVELS for v in ("u", "v", "temperature", "humidity", "geopotential")]
UNITS = ["hPa", "hPa", "m/s", "m/s", "K", "K"] + [
    unit for _ in LEVELS for unit in ("m/s", "m/s", "K", "kg/kg", "m2/s2")]
HISTORY = 9
LEADS = 20
HOUR = 3_600_000_000_000


def digest(path):
    h = hashlib.sha256()
    with Path(path).open("rb") as f:
        for block in iter(lambda: f.read(8 * 1024**2), b""):
            h.update(block)
    return h.hexdigest()


def atomic_json(path, data):
    path = Path(path)
    tmp = path.with_suffix(path.suffix + ".tmp")
    tmp.write_text(json.dumps(data, indent=2, allow_nan=False) + "\n")
    tmp.replace(path)


def atomic_checkpoint(path, data):
    path = Path(path)
    tmp = path.with_suffix(".tmp")
    with tmp.open("wb") as f:
        torch.save(data, f)
        f.flush()
        os.fsync(f.fileno())
    tmp.replace(path)


def device_for(name):
    if name == "auto":
        name = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu"
    if name == "cuda" and not torch.cuda.is_available():
        raise RuntimeError("CUDA unavailable in this Python. Install a CUDA PyTorch wheel before training.")
    if name == "mps" and not torch.backends.mps.is_available():
        raise RuntimeError("MPS requested but unavailable")
    return torch.device(name)


class Block(nn.Module):
    def __init__(self, width, periodic=False):
        super().__init__()
```

```
        self.periodic = periodic
        self.depthwise = nn.Conv2d(width, width, 5, groups=width)
        self.net = nn.Sequential(nn.GroupNorm(8, width), nn.Conv2d(width, width * 4, 1),
                                 nn.GELU(), nn.Conv2d(width * 4, width, 1))
        self.scale = nn.Parameter(torch.full((1, width, 1, 1), 0.01))

    def forward(self, x):
        padded = F.pad(x, (2, 2, 0, 0), mode="circular" if self.periodic else "replicate")
        padded = F.pad(padded, (0, 0, 2, 2), mode="replicate")
        return x + self.scale * self.net(self.depthwise(padded))


class FieldOperator(nn.Module):
    def __init__(self, channels, base, periodic, attention_layers=0, tiny=False):
        super().__init__()
        widths = [base, base * 2, base * 4, base * 6]
        self.stem = nn.Conv2d(channels * 3 + 4, base, 3, padding=1)
        depths = [1, 1, 1, 1] if tiny else [2, 3, 5, 6]
        self.enc = nn.ModuleList([nn.Sequential(*[Block(w, periodic) for _ in range(n)]) for w, n in zip(widths, depths)])
        self.down = nn.ModuleList([nn.Conv2d(a, b, 3, stride=2, padding=1) for a, b in zip(widths, widths[1:])])
        self.attention = nn.ModuleList([nn.TransformerEncoderLayer(widths[-1], 8,
                                        dim_feedforward=widths[-1] * 4, batch_first=True,
                                        dropout=0.0, norm_first=True) for _ in range(attention_layers)])
        self.up = nn.ModuleList([nn.Conv2d(widths[i + 1], widths[i], 1) for i in (2, 1, 0)])
        self.dec = nn.ModuleList([nn.Sequential(Block(widths[i], periodic), Block(widths[i], periodic)) for i in (2, 1, 0)])
        self.out = nn.Conv2d(base, channels, 1)
        self.object_features = nn.Conv2d(base, 32, 1)
        self.checkpointing = True
        nn.init.normal_(self.out.weight, std=1e-3)
        nn.init.zeros_(self.out.bias)

    def run(self, module, value):
        return checkpoint(module, value, use_reentrant=False) if self.training and self.checkpointing else module(value)

    def forward(self, state, context, mask, static):
        x = self.stem(torch.cat([state, context, mask, static], 1))
        skips = []
        for i in range(4):
            x = self.run(self.enc[i], x)
            skips.append(x)
            if i < 3:
                x = self.down[i](x)
        if self.attention:
            shape = x.shape
            tokens = x.flatten(2).transpose(1, 2)
            for layer in self.attention:
                tokens = self.run(layer, tokens)
            x = tokens.transpose(1, 2).reshape(shape)
        for up, dec, skip in zip(self.up, self.dec, reversed(skips[:-1])):
            x = self.run(dec, F.interpolate(up(x), size=skip.shape[-2:], mode="bilinear", align_corners=False) + skip)
        # A learned field tendency, with no fixed per-cell hPa cap or route warp.
        return state + self.out(x), self.object_features(x)


class GlobalRegionalForecaster(nn.Module):
    def __init__(self, global_lat, global_lon, regional_lat, regional_lon, mean, std, tiny=False):
        super().__init__()
        c = len(CHANNELS)
        self.register_buffer("mean", torch.tensor(mean, dtype=torch.float32).view(1, c, 1, 1))
        self.register_buffer("std", torch.tensor(std, dtype=torch.float32).view(1, c, 1, 1))
        self.register_buffer("regional_lat", torch.tensor(regional_lat, dtype=torch.float32))
        self.register_buffer("regional_lon", torch.tensor(regional_lon, dtype=torch.float32))
        self.global_time = nn.Conv2d(c * HISTORY, c, 1)
        self.regional_time = nn.Conv2d(c * HISTORY, c, 1)
        self.global_operator = FieldOperator(c, 8 if tiny else 80, True, 1 if tiny else 3, tiny)
        self.regional_operator = FieldOperator(c, 8 if tiny else 88, False, 0, tiny)
        self.parent_fusion = nn.Conv2d(c * 2, c, 1)
        self.object_score = nn.Conv2d(32 + 3, 1, 1)
        self.intensity = nn.Sequential(nn.Linear(32 + 3, 64), nn.GELU(), nn.Linear(64, 2))
        y, x = np.meshgrid(regional_lat, regional_lon, indexing="ij")
        grid = np.stack([2 * (x - global_lon[0]) / (global_lon[-1] - global_lon[0]) - 1,
                         2 * (y - global_lat[0]) / (global_lat[-1] - global_lat[0]) - 1], -1)
        self.register_buffer("parent_grid", torch.tensor(grid, dtype=torch.float32)[None])
        self.register_buffer("yy", torch.tensor(y, dtype=torch.float32)[None])
        self.register_buffer("xx", torch.tensor(x, dtype=torch.float32)[None])

    def initial(self, inputs):
        expected = {"global_history", "regional_history", "global_mask", "regional_mask",
                    "global_static", "regional_static", "center", "motion"}
        if set(inputs) != expected:
            raise ValueError("Forward inputs must contain only the declared causal input keys")
        gh, rh = inputs["global_history"], inputs["regional_history"]
```

```
        return {"g": gh[:, -1], "r": rh[:, -1],
                "gc": self.global_time(gh.flatten(1, 2)), "rc": self.regional_time(rh.flatten(1, 2)),
                "gm": inputs["global_mask"], "rm": inputs["regional_mask"],
                "gs": inputs["global_static"], "rs": inputs["regional_static"],
                "center": inputs["center"], "motion": inputs["motion"]}

    def step(self, state):
        g, _ = self.global_operator(state["g"], state["gc"], state["gm"], state["gs"])
        parent = F.grid_sample(g, self.parent_grid.expand(len(g), -1, -1, -1), align_corners=True)
        context = self.parent_fusion(torch.cat([state["rc"], parent], 1))
        r, features = self.regional_operator(state["r"], context, state["rm"], state["rs"])
        physical = r.float() * self.std + self.mean
        pressure = physical[:, 0]
        anomaly = pressure - F.avg_pool2d(F.pad(pressure[:, None], (4, 4, 4, 4), mode="replicate"), 9,
    stride=1)[:, 0]
        idx_u, idx_v = CHANNELS.index("u850"), CHANNELS.index("v850")
        u, v = physical[:, idx_u], physical[:, idx_v]
        evidence = torch.cat([features.float(), anomaly[:, None] / 10, u[:, None] / 30, v[:, None] / 30], 1)
        score = self.object_score(evidence)[:, 0] - anomaly / 10
        lat0, lon0 = state["center"][:, 0, None, None], state["center"][:, 1, None, None]
        prior_lat = lat0 + state["motion"][:, 1, None, None] / 111.2
        prior_lon = lon0 + state["motion"][:, 0, None, None] / (111.2 * lat0.deg2rad().cos().clamp_min(.2))
        distance2 = ((self.yy - prior_lat) * 111.2).square() + ((self.xx - prior_lon) * 111.2 *
    lat0.deg2rad().cos()).square()
        score = score - distance2 / (2 * 600**2)
        # Select one spatial mode, then refine locally; never average distinct lows.
        peak = score.flatten(1).argmax(1)
        h, w = pressure.shape[-2:]
        iy = torch.arange(h, device=r.device)[None, :, None]
        ix = torch.arange(w, device=r.device)[None, None, :]
        local = ((iy - (peak // w)[:, None, None]).abs() <= 2) & ((ix - (peak % w)[:, None, None]).abs() <= 2)
        pdf = torch.softmax(score.masked_fill(~local, -1e9).flatten(1), 1).reshape_as(score)
        center = torch.stack([(pdf * self.yy).sum((1, 2)), (pdf * self.xx).sum((1, 2))], 1)
        pooled = (evidence * pdf[:, None]).sum((2, 3))
        intensity = self.intensity(pooled)
        grid_pressure = (pressure * pdf).sum((1, 2))
        central_pressure = grid_pressure - F.softplus(intensity[:, 0]) * 5
        surface_speed = torch.sqrt(physical[:, 2].square() + physical[:, 3].square() + 1e-6)
        vmax = (surface_speed * pdf).sum((1, 2)) * 1.943844 + F.softplus(intensity[:, 1]) * 5
        mean_lat = (center[:, 0] + state["center"][:, 0]) / 2
        motion = torch.stack([(center[:, 1] - state["center"][:, 1]) * 111.2 * mean_lat.deg2rad().cos(),
                              (center[:, 0] - state["center"][:, 0]) * 111.2], 1)
        next_state = dict(state, g=g, r=r, center=center, motion=motion)
        return next_state, {"global": g, "regional": r, "center": center, "pressure": central_pressure,
                            "vmax": vmax, "grid_pressure": grid_pressure, "association": pdf}

    def forward(self, inputs, steps=20):
        if not 1 <= steps <= LEADS:
            raise ValueError("steps must be 1..20")
        state = self.initial(inputs)
        outputs = []
        for _ in range(steps):
            state, out = self.step(state)
            outputs.append(out)
        return {k: torch.stack([o[k] for o in outputs], 1) for k in outputs[0]}


def require_full_dataset(m, times=None, rows=None):
    if m.get('dataset_role') != 'full_training' or m.get('synthetic', False):
        raise ValueError('FULL DATA REQUIRED: pilot, synthetic or unclassified datasets cannot be used by the
    training CLI. Prepare the full 2000-2025 dataset on the Mac first.')
    if times is not None:
        expected = np.arange(np.datetime64('2000-01-01', 'ns').astype('int64'),
                             np.datetime64('2026-01-01', 'ns').astype('int64'), 6*HOUR, dtype='int64')
        if not np.array_equal(times, expected):
            raise ValueError('Full dataset must contain every six-hour field from 2000 through 2025; partial
    coverage is not full training')
    if rows is not None:
        for split, years in [('train', range(2000, 2022)), ('validation', range(2022, 2024)), ('test',
    range(2024, 2026))]:
            covered = {int(np.datetime64(int(times[r['issue_index']]),
    'ns').astype('datetime64[Y]').astype(int))+1970
                       for r in rows if r['split'] == split}
            if covered != set(years):
                raise ValueError(f'Full dataset lacks sample coverage in {split}: {sorted(set(years)-covered)}')


def load_manifest(path, verify=True, smoke=False, require_full=False):
    path = Path(path)
    if not path.is_file():
        raise FileNotFoundError(f"V164 DATA NOT READY: {path}. Requires native global <=1-degree and regional
    <=0.25-degree multilevel fields. The v163 2.5-degree archive cannot substitute. See v164_dataset_contract.md.")
    m = json.loads(path.read_text())
    if require_full:
        require_full_dataset(m)
    if m.get("schema") != "v164-atmosphere-v1" or m.get("channels") != CHANNELS or m.get("units") != UNITS:
```

```
        raise ValueError("Dataset schema/channel order/physical units do not match v164")
    if m.get("future_weather_as_inputs") is not False or m.get("official_forecasts_as_inputs") is not False:
        raise ValueError("Dataset must explicitly disable future weather and official forecast inputs")
    if not smoke and m.get("synthetic", False):
        raise ValueError("Synthetic data is only permitted by the smoke-test API")
    for key in ("global", "regional", "global_static", "regional_static", "time", "samples"):
        item = m["files"][key]
        p = (path.parent / item["path"]).resolve()
        if not p.is_relative_to(path.parent.resolve()) or not p.is_file():
            raise ValueError(f"Missing or outside-dataset file: {key}")
        if verify and digest(p) != item["sha256"]:
            raise ValueError(f"Dataset checksum mismatch: {key}")
        item["resolved"] = str(p)
    times = np.load(m["files"]["time"]["resolved"], mmap_mode="r")
    if times.dtype != np.dtype("int64") or np.any(np.diff(times) <= 0) or np.any(times % (6 * HOUR)):
        raise ValueError("Time must be increasing int64 UTC nanoseconds aligned to six hours")
    if m["normalization"]["fit_split"] != "train" or m["normalization"]["end_year"] > 2021:
        raise ValueError("Normalization must be fitted exclusively to training years <=2021")
    mean, std = np.array(m["normalization"]["mean"]), np.array(m["normalization"]["std"])
    if mean.shape != (len(CHANNELS),) or std.shape != mean.shape or not np.isfinite(mean).all() or not
np.isfinite(std).all() or (std <= 0).any():
        raise ValueError("Invalid training normalization")
    for region, max_step in (("global", 1), ("regional", .25)):
        lat, lon = np.array(m[region + "_lat"]), np.array(m[region + "_lon"])
        if min(len(lat), len(lon)) < 2 or np.any(np.diff(lat) >= 0) or np.any(np.diff(lon) <= 0):
            raise ValueError("Coordinates must be north-to-south, west-to-east")
        if not np.allclose(np.diff(lat), lat[1]-lat[0], atol=1e-5) or not np.allclose(np.diff(lon),
lon[1]-lon[0], atol=1e-5):
            raise ValueError("This operator requires regular latitude/longitude grids")
        if not smoke and (np.max(-np.diff(lat)) > max_step + 1e-5 or np.max(np.diff(lon)) > max_step + 1e-5):
            raise ValueError(f"{region} resolution too coarse")
        if region == "global" and not smoke and (lat[0] < 89 or lat[-1] > -89 or lon[0] != 0 or lon[-1] < 359):
            raise ValueError("Global parent must cover the globe without duplicated longitude seam")
        array = np.load(m["files"][region]["resolved"], mmap_mode="r")
        static = np.load(m["files"][region + "_static"]["resolved"], mmap_mode="r")
        if array.shape != (len(times), len(CHANNELS), len(lat), len(lon)) or static.shape != (4, len(lat),
len(lon)):
            raise ValueError(f"{region} field/static shape mismatch")
        if not np.isfinite(static).all():
            raise ValueError("Non-finite static geography")
    if not (min(m["global_lat"]) <= min(m["regional_lat"]) <= max(m["regional_lat"]) <= max(m["global_lat"]) and
            min(m["global_lon"]) <= min(m["regional_lon"]) <= max(m["regional_lon"]) <= max(m["global_lon"])):
        raise ValueError("Regional grid lies outside global parent coverage")
    rows = [json.loads(line) for line in Path(m["files"]["samples"]["resolved"]).read_text().splitlines() if
line.strip()]
    membership = {}
    for row in rows:
        i = row["issue_index"]
        if i < HISTORY - 1 or i + LEADS >= len(times):
            raise ValueError("Sample lacks full causal history or target coverage")
        if np.any(np.diff(times[i-8:i+21]) != 6 * HOUR):
            raise ValueError("Sample crosses a gap in causal history or future target coverage")
        year = int(np.datetime64(int(times[i]), "ns").astype("datetime64[Y]").astype(int)) + 1970
        valid_years = {"train": (2000, 2021), "validation": (2022, 2023), "test": (2024, 2025)}
        low, high = valid_years[row["split"]]
        first_year = int(np.datetime64(int(times[i-8]), "ns").astype("datetime64[Y]").astype(int)) + 1970
        last_year = int(np.datetime64(int(times[i+20]), "ns").astype("datetime64[Y]").astype(int)) + 1970
        if not smoke and not (low <= first_year <= year <= last_year <= high):
            raise ValueError("Sample violates fixed chronological split")
        storm = row.get("storm_id")
        if storm:
            if "1979" in storm or storm.upper() == "TIP":
                raise ValueError("Tip is excluded from training, validation and test")
            if storm in membership and membership[storm] != row["split"]:
                raise ValueError("Storm crosses dataset splits")
            membership[storm] = row["split"]
            if "center" not in row or "target_centers" not in row:
                raise ValueError("Storm samples require observed center and masked target centers")
            centers = np.asarray(row["target_centers"])
            route_mask = np.asarray(row.get("route_mask", [1] * LEADS))
            if centers.shape != (LEADS, 2) or route_mask.shape != (LEADS,) or not np.isfinite(centers).all():
                raise ValueError("Invalid storm target coordinates/mask")
        for field in ("target_pressure_hpa", "target_vmax_kt", "pressure_mask", "vmax_mask"):
            values = np.asarray(row.get(field, [0] * LEADS))
            if values.shape != (LEADS,) or not np.isfinite(values).all():
                raise ValueError(f"Invalid target field: {field}")
        for field in ("route_mask", "pressure_mask", "vmax_mask"):
            values = np.asarray(row.get(field, [0] * LEADS))
            if not np.isin(values, [0, 1]).all():
                raise ValueError("Target masks must be binary")
        center = np.asarray(row.get("center", [20, 140]))
        motion = np.asarray(row.get("motion_km_per_6h", [0, 0]))
        if center.shape != (2,) or motion.shape != (2,) or not np.isfinite([center, motion]).all():
            raise ValueError("Invalid causal center or motion")
        if not (min(m["regional_lat"]) <= center[0] <= max(m["regional_lat"]) and min(m["regional_lon"]) <=
center[1] <= max(m["regional_lon"])):
```

```
            raise ValueError("Issue center outside regional domain")
    if not smoke and not any(not r.get("storm_id") for r in rows if r["split"] == "train"):
        raise ValueError("Field pretraining requires non-storm atmospheric samples")
    if require_full:
        require_full_dataset(m, times, rows)
    m["rows"] = rows
    m["identity"] = digest(path)
    return m


class AtmosphereDataset(Dataset):
    def __init__(self, manifest, split, training=False):
        self.m, self.training = manifest, training
        self.rows = [r for r in manifest["rows"] if r["split"] == split]
        self.arrays = None

    def __len__(self):
        return len(self.rows)

    def __getitem__(self, index):
        if self.arrays is None:
            self.arrays = {k: np.load(self.m["files"][k]["resolved"], mmap_mode="r") for k in ("global",
"regional", "global_static", "regional_static")}
        row = self.rows[index]
        i = row["issue_index"]
        mean = np.array(self.m["normalization"]["mean"], dtype="float32")[:, None, None]
        std = np.array(self.m["normalization"]["std"], dtype="float32")[:, None, None]
        inputs, targets = {}, {}
        drop_sst = self.training and np.random.random() < .15
        for name in ("global", "regional"):
            history = np.array(self.arrays[name][i - 8:i + 1], dtype="float32")
            available = np.isfinite(history)
            # SST can be masked; all atmospheric predictor channels are required.
            if not available[:, [k for k in range(len(CHANNELS)) if k != 5]].all():
                raise ValueError(f"Missing required atmospheric inputs at sample {i}")
            if drop_sst:
                available[:, 5] = False
            history = np.where(available, (history - mean) / std, 0)
            inputs[name + "_history"] = torch.from_numpy(history)
            inputs[name + "_mask"] = torch.from_numpy(available.all(0).astype("float32"))
            inputs[name + "_static"] = torch.from_numpy(np.array(self.arrays[name + "_static"],
dtype="float32"))
            future = np.array(self.arrays[name][i + 1:i + 21], dtype="float32")
            mask = np.isfinite(future)
            if not mask[:, [k for k in range(len(CHANNELS)) if k != 5]].all():
                raise ValueError(f"Missing required atmospheric verification fields at sample {i}")
            targets[name] = torch.from_numpy(np.where(mask, (future - mean) / std, 0))
            targets[name + "_mask"] = torch.from_numpy(mask.astype("float32"))
        inputs["center"] = torch.tensor(row.get("center", [20, 140]), dtype=torch.float32)
        inputs["motion"] = torch.tensor(row.get("motion_km_per_6h", [0, 0]), dtype=torch.float32)
        storm = bool(row.get("storm_id"))
        targets["center"] = torch.tensor(row.get("target_centers", [[0, 0]] * 20), dtype=torch.float32)
        targets["pressure"] = torch.tensor(row.get("target_pressure_hpa", [0] * 20), dtype=torch.float32)
        targets["vmax"] = torch.tensor(row.get("target_vmax_kt", [0] * 20), dtype=torch.float32)
        targets["storm_mask"] = torch.tensor(row.get("route_mask", [float(storm)] * 20), dtype=torch.float32)
        targets["pressure_mask"] = torch.tensor(row.get("pressure_mask", [0] * 20), dtype=torch.float32)
        targets["vmax_mask"] = torch.tensor(row.get("vmax_mask", [0] * 20), dtype=torch.float32)
        return inputs, targets


def masked_mean(value, mask):
    return (value * mask).sum() / mask.expand_as(value).sum().clamp_min(1)


def objective(output, target, step, manifest, joint):
    terms = {}
    for region in ("global", "regional"):
        p, t = output[region].float(), target[region][:, step]
        mask = target[region + "_mask"][:, step]
        lat = torch.tensor(manifest[region + "_lat"], device=p.device,
dtype=torch.float32).deg2rad().cos()[None, None, :, None]
        if region == "regional" and "regional_lat_weight" in target:
            lat = target["regional_lat_weight"][:, step]
        terms[region] = masked_mean(F.smooth_l1_loss(p, t, reduction="none"), mask * lat)
        dx = (p[..., 1:] - p[..., :-1]) - (t[..., 1:] - t[..., :-1])
        dy = (p[..., 1:, :] - p[..., :-1, :]) - (t[..., 1:, :] - t[..., :-1, :])
        terms[region + "_gradient"] = .5 * (masked_mean(dx.abs(), mask[..., 1:] * mask[..., :-1]) +
masked_mean(dy.abs(), mask[..., 1:, :] * mask[..., :-1, :]))
    delta = output["center"] - target["center"][:, step]
    scale = torch.stack([torch.ones_like(delta[:, 0]), target["center"][:, step, 0].deg2rad().cos()], 1) * 111.2
 / 100
    terms["route"] = masked_mean(F.smooth_l1_loss(delta * scale, torch.zeros_like(delta), reduction="none"),
target["storm_mask"][:, step, None])
    terms["pressure"] = masked_mean((output["pressure"] - target["pressure"][:, step]).abs() / 20,
target["pressure_mask"][:, step])
    terms["vmax"] = masked_mean((output["vmax"] - target["vmax"][:, step]).abs() / 30, target["vmax_mask"][:,
```

```
    step])
    value = terms["global"] + terms["regional"] + .2 * (terms["global_gradient"] + terms["regional_gradient"])
    if joint:
        value = value + .2 * terms["route"] + .1 * (terms["pressure"] + terms["vmax"])
    return value, terms


def make_model(m, tiny=False):
    return GlobalRegionalForecaster(*(m[k] for k in ("global_lat", "global_lon", "regional_lat",
    "regional_lon")),
                                    m["normalization"]["mean"], m["normalization"]["std"], tiny)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("batch", type=int, nargs="?", default=10, help="Effective examples per optimizer update;
     physical microbatch is 1")
    parser.add_argument("workers", type=int, nargs="?", default=3)
    parser.add_argument("amp", type=int, nargs="?", default=1, choices=[0, 1])
    persistent = Path("/content/drive/MyDrive/typhoon_predict") if Path("/content").is_dir() else ROOT
    default_manifest = persistent / "data/v164/manifest.json"
    parser.add_argument("--manifest", type=Path, default=Path(os.getenv("V164_MANIFEST", default_manifest)))
    parser.add_argument("--output", type=Path, default=os.getenv("V164_OUTPUT_DIR"))
    parser.add_argument("--epochs", type=int, default=12)
    parser.add_argument("--device", default="auto", choices=["auto", "cuda", "mps", "cpu"])
    parser.add_argument("--preflight", action="store_true")
    parser.add_argument("--model-count", action="store_true")
    args = parser.parse_args()
    if args.batch < 1 or args.workers < 0 or args.epochs < 1:
        parser.error("batch/epochs must be positive; workers must be nonnegative")
    if args.model_count:
        m = {"global_lat": np.linspace(90, -90, 181), "global_lon": np.arange(360),
             "regional_lat": np.linspace(60, 0, 241), "regional_lon": np.linspace(100, 180, 321),
             "normalization": {"mean": [0] * len(CHANNELS), "std": [1] * len(CHANNELS)}}
        model = make_model(m)
        print(json.dumps({"version": VERSION, "architecture": ARCHITECTURE, "parameters": sum(p.numel() for p in
     model.parameters()), "channels": len(CHANNELS)}))
        return
    m = load_manifest(args.manifest, require_full=True)
    if args.output is None:
        args.output = persistent / "training/v164"
    print(json.dumps({"version": VERSION, "dataset_role": m.get("dataset_role", "full"),
                      "pilot_only": False, "samples": len(m["rows"]), "output": str(args.output)}), flush=True)
    if args.preflight:
        for split in ("train", "validation", "test"):
            dataset = AtmosphereDataset(m, split)
            if not len(dataset):
                raise ValueError(f"Empty {split} split")
            dataset[0]
        print("PASS dataset integrity and first-sample checks. No training started.")
        return
    if Path("/content").is_dir() and not
    args.output.resolve().is_relative_to(Path("/content/drive/MyDrive/typhoon_predict")):
        raise ValueError("Colab outputs must persist in MyDrive/typhoon_predict")
    if Path("/content").is_dir() and not Path("/content/drive/MyDrive").is_dir():
        raise RuntimeError("Mount Drive before training; refusing temporary checkpoints")
    device = device_for(args.device)
    random.seed(164); np.random.seed(164); torch.manual_seed(164)
    model = make_model(m).to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=.01)
    scaler = torch.amp.GradScaler("cuda", enabled=device.type == "cuda" and bool(args.amp))
    args.output.mkdir(parents=True, exist_ok=True)
    lock = args.output / "training.lock"
    try:
        fd = os.open(lock, os.O_CREAT | os.O_EXCL | os.O_WRONLY)
    except FileExistsError as exc:
        raise RuntimeError(f"Output locked: {lock}. Verify no other runtime is training before removing a stale
    lock.") from exc
    os.write(fd, json.dumps({"pid": os.getpid(), "host": socket.gethostname()}).encode()); os.close(fd)
    try:
        train(model, optimizer, scaler, m, args, device)
    finally:
        lock.unlink(missing_ok=True)


def train(model, optimizer, scaler, m, args, device):
    last, best = args.output / "last.pt", args.output / "best.pt"
    start, history, best_value = 1, [], float("inf")
    if last.exists():
        saved = torch.load(last, map_location="cpu", weights_only=False)
        if saved["architecture"] != ARCHITECTURE or saved["dataset_sha256"] != m["identity"]:
            raise ValueError("Resume architecture/dataset mismatch")
        model.load_state_dict(saved["model"]); optimizer.load_state_dict(saved["optimizer"]);
    scaler.load_state_dict(saved["scaler"])
        start, history, best_value = saved["epoch"] + 1, saved["history"], saved["best_value"]
        random.setstate(saved["python_rng"]); np.random.set_state(saved["numpy_rng"]);
```

```
    torch.set_rng_state(saved["torch_rng"])
481         if device.type == "cuda" and saved.get("cuda_rng") is not None:
482             torch.cuda.set_rng_state_all(saved["cuda_rng"])
483     atomic_json(args.output / "manifest.json", {"version": VERSION, "architecture": ARCHITECTURE,
484                 "parameters": sum(p.numel() for p in model.parameters()), "dataset_sha256": m["identity"],
485                 "script_sha256": digest(__file__), "future_weather_as_inputs": False,
486                 "official_forecasts_as_inputs": False, "ensemble": False,
487                 "resume_policy": "last completed epoch with optimizer/scaler/RNG",
488                 "physical_microbatch": 1, "effective_batch": args.batch})
489     workers = min(args.workers, max(0, (os.cpu_count() or 1) - 1), 2)
490     for epoch in range(start, args.epochs + 1):
491         horizon = [2, 4, 8, 12, 20][min(epoch - 1, 4)]
492         joint = epoch > 2
493         epoch_start = time.time()
494         metrics = {}
495         for split in ("train", "validation"):
496             training = split == "train"
497             model.train(training)
498             dataset = AtmosphereDataset(m, split, training)
499             if not len(dataset):
500                 raise ValueError(f"Empty {split} split")
501             loader = DataLoader(dataset, batch_size=1, shuffle=training, num_workers=workers,
502                                 **({"prefetch_factor": 1} if workers else {}))
503             optimizer.zero_grad(set_to_none=True)
504             sums = []
505             for index, (inputs, targets) in enumerate(loader, 1):
506                 inputs = {k: v.to(device) for k, v in inputs.items()}
507                 steps = horizon if training else 20
508                 loss_block, total = 0, 0.0
509                 with torch.set_grad_enabled(training):
510                     state = model.initial(inputs)
511                     for step in range(steps):
512                         with torch.autocast(device_type="cuda", dtype=torch.float16) if scaler.is_enabled() else
 nullcontext():
513                             state, out = model.step(state)
514                         # Transfer one target frame at a time to bound VRAM.
515                         frame = {k: v[:, step:step+1].to(device) for k, v in targets.items()}
516                         loss, _ = objective(out, frame, 0, m, joint)
517                         if not torch.isfinite(loss):
518                             raise RuntimeError(f"Non-finite loss epoch={epoch} sample={index} step={step};
    checkpoint not advanced")
519                         total += float(loss.detach()) / steps
520                         loss_block = loss_block + loss / (steps * args.batch)
521                         if training and ((step + 1) % 2 == 0 or step + 1 == steps):
522                             scaler.scale(loss_block).backward()
523                             state = {k: v.detach() for k, v in state.items()}
524                             loss_block = 0
525                     if training and (index % args.batch == 0 or index == len(loader)):
526                         scaler.unscale_(optimizer)
527                         remainder = index % args.batch
528                         if index == len(loader) and remainder:
529                             for p in model.parameters():
530                                 if p.grad is not None:
531                                     p.grad.mul_(args.batch / remainder)
532                         norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
533                         if not torch.isfinite(norm):
534                             raise RuntimeError("Non-finite gradients; checkpoint not advanced. Retry last epoch
    with AMP=0.")
535                         scaler.step(optimizer); scaler.update(); optimizer.zero_grad(set_to_none=True)
536                 sums.append(total)
537                 if index == 1 or index % 10 == 0:
538                     print(f"{split} epoch={epoch}/{args.epochs} sample={index}/{len(loader)} horizon={steps}
    loss={total:.5f} elapsed_s={time.time()-epoch_start:.0f}", flush=True)
539             metrics[split] = float(np.mean(sums))
540         # Selection only after reaching the full horizon, on a stable joint objective.
541         promote = epoch >= 5 and metrics["validation"] < best_value
542         if promote:
543             best_value = metrics["validation"]
544         history.append({"epoch": epoch, "horizon": horizon, "joint": joint, **metrics})
545         payload = {"version": VERSION, "architecture": ARCHITECTURE, "dataset_sha256": m["identity"],
546                    "data_contract": {k: m[k] for k in ("channels", "units", "global_lat", "global_lon",
    "regional_lat", "regional_lon", "normalization")},
547                    "epoch": epoch, "model": model.state_dict(), "optimizer": optimizer.state_dict(),
548                    "scaler": scaler.state_dict(), "history": history, "best_value": best_value,
549                    "python_rng": random.getstate(), "numpy_rng": np.random.get_state(), "torch_rng":
    torch.get_rng_state(),
550                    "cuda_rng": torch.cuda.get_rng_state_all() if device.type == "cuda" else None}
551         atomic_checkpoint(last, payload)
552         if promote:
553             atomic_checkpoint(best, payload)
554         atomic_json(args.output / "history.json", history)
555         print(f"Saved completed epoch {epoch}: {last}; promoted_best={promote}", flush=True)
556     print("Training completed. Checkpoints and history are persistent. Test set was not used for selection.",
    flush=True)
557
558
```

```
# BEGIN REUSE IMPLEMENTATION
"""Embedded in the standalone v164 trainer: coarse archive plus native MSLP patches."""
REUSE_ARCH = 'wp-coarse8-native-pressure1-v164'
REUSE_CHANNELS = ['mslp', 'hgt500', 'uwnd850', 'vwnd850', 'uwnd500', 'vwnd500', 'uwnd200', 'vwnd200']


def load_reuse(path):
    path = Path(path)
    if not path.is_file():
        raise FileNotFoundError('Combined dataset missing. Run the updated Drive setup cell first: '+str(path))
    m = json.loads(path.read_text())
    if m.get('schema') != 'v164-reuse-pressure-v1' or m.get('pilot', True):
        raise ValueError('Combined non-pilot archive required; no pilot fallback')
    for name, entry in m['files'].items():
        p = path.parent/entry['path']
        print('Checking dataset file:', name, flush=True)
        if digest(p) != entry['sha256']:
            raise ValueError('Dataset checksum mismatch: '+name)
        entry['resolved'] = str(p)
    with np.load(m['files']['basin']['resolved'], allow_pickle=False) as z:
        if z['channels'].tolist() != REUSE_CHANNELS[1:]:
            raise ValueError('Coarse channel order mismatch')
        times, lat, lon = z['time'], z['lat'], z['lon']
        q = z['q']
        scale, offset = z['scale'], z['offset']
    if q.shape != (len(times),7,25,33) or not np.all(np.diff(times) == 6*HOUR):
        raise ValueError('Coarse grid/time mismatch')
    qpath = path.parent/'basin_q_verified.npy'
    # Refresh the compact worker mmap from the checksum-verified source every preflight.
    temporary = qpath.with_suffix('.pending.npy')
    np.save(temporary, q)
    os.replace(temporary, qpath)
    slp = np.load(m['files']['slp']['resolved'], mmap_mode='r')
    if slp.shape != (len(times),25,33) or not np.isfinite(slp).all():
        raise ValueError('Coarse pressure coverage invalid')
    with np.load(m['files']['track']['resolved'], allow_pickle=False) as z:
        track = {k:z[k] for k in ['base_time','storm_id','base_lat','base_lon','target','target_mask']}
        issue_intensity = z['track'][:,-1,4:6]*z['track_std'][4:6]+z['track_mean'][4:6]
    plan = json.loads(Path(m['files']['plan']['resolved']).read_text())
    if len(plan['rows']) != 1000 or plan['shape'] != [1000,29,121,121] or plan['lead_hours'] !=
list(range(-48,121,6)):
        raise ValueError('Expected the verified 1000-window native pressure package, not the six-sample pilot')
    pressure = np.load(m['files']['pressure']['resolved'], mmap_mode='r')
    if pressure.shape != (1000,29,121,121):
        raise ValueError('Native pressure shape mismatch')
    patches = {}
    for pi,row in enumerate(plan['rows']):
        ti = row['track_archive_row']
        if int(track['base_time'][ti]) != row['issue_ns'] or str(track['storm_id'][ti]) != row['storm_id']:
            raise ValueError('Native patch/track identity mismatch')
        patches[ti] = pi
    first_year = {}
    years = track['base_time'].astype('datetime64[ns]').astype('datetime64[Y]').astype(int)+1970
    for sid,y in zip(track['storm_id'],years):
        first_year[str(sid)] = min(first_year.get(str(sid),9999),int(y))
    lookup = {(str(s),int(t)):i for i,(s,t) in enumerate(zip(track['storm_id'],track['base_time']))}
    rows = []
    for i,ns in enumerate(track['base_time']):
        sid = str(track['storm_id'][i])
        y = first_year[sid]
        split = 'train' if 2000 <= y <= 2021 else 'validation' if 2022 <= y <= 2023 else 'test' if 2024 <= y <=
2025 else None
        if split is None:
            continue
        low,high = {'train':(2000,2021),'validation':(2022,2023),'test':(2024,2025)}[split]
        bounds = np.array([ns-48*HOUR,ns+120*HOUR],dtype='int64').astype('datetime64[ns]').astype('datetime64[Y]
').astype(int)+1970
        ai = int(np.searchsorted(times,ns))
        center = [float(track['base_lat'][i]),float(track['base_lon'][i])%360]
        if bounds[0] < low or bounds[1] > high or ai < 8 or ai+20 >= len(times) or times[ai] != ns:
            continue
        if not np.isfinite(center).all() or not (0 < center[0] < 60 and 100 < center[1] < 180):
            continue
        previous = lookup.get((sid,int(ns-6*HOUR)))
        motion = [0.,0.]
        if previous is not None:
            old = [float(track['base_lat'][previous]),float(track['base_lon'][previous])%360]
            if np.isfinite(old).all():
                motion = [(center[1]-old[1])*111.2*np.cos(np.deg2rad(center[0])),(center[0]-old[0])*111.2]
        future = [lookup.get((sid,int(ns+j*6*HOUR))) for j in range(1,21)]
        target_centers = [[float(track['base_lat'][j]),float(track['base_lon'][j])%360] if j is not None else
[0.,0.] for j in future]
        rm = [float(j is not None and np.isfinite(target_centers[k]).all()) for k,j in enumerate(future)]
        target_centers = np.nan_to_num(target_centers).tolist()
        wind,p = track['target'][i,:,2],track['target'][i,:,3]
        wm = track['target_mask'][i,:,2].astype(bool)&np.isfinite(wind)&(wind>=0)&(wind<250)
```

```
        pm = track['target_mask'][i,:,3].astype(bool)&np.isfinite(p)&(p>800)&(p<1100)
        pi = patches.get(i,-1)
        if pi >= 0 and plan['rows'][pi]['split'] != split:
            raise ValueError('Patch split mismatch')
        issue = issue_intensity[i]
        iv = [bool(np.isfinite(issue[0]) and 0<=issue[0]<250),bool(np.isfinite(issue[1]) and 800<issue[1]<1100)]
        rows.append({'split':split,'storm_id':sid,'atlas':ai,'patch':pi,'center':center,'motion':motion,
                     'issue_intensity':[float(issue[k]) if iv[k] else 0. for k in range(2)],'issue_mask':iv,
                     'target_centers':target_centers,'route_mask':rm,
                     'pressure':np.where(pm,p,0).tolist(),'pressure_mask':pm.tolist(),
                     'vmax':np.where(wm,wind,0).tolist(),'vmax_mask':wm.tolist()})
    counts = {s:sum(r['split']==s for r in rows) for s in ['train','validation','test']}
    patch_counts = {s:sum(r['split']==s and r['patch']>=0 for r in rows) for s in counts}
    if counts['train'] < 10000 or patch_counts != {'train':800,'validation':100,'test':100}:
        raise ValueError(f'Incomplete combined coverage: {counts}, native patches {patch_counts}')
    train_times =
np.flatnonzero((times.astype('datetime64[ns]').astype('datetime64[Y]').astype(int)+1970>=2000)&
                               (times.astype('datetime64[ns]').astype('datetime64[Y]').astype(int)+1970<=2021))
    sums,squares = np.zeros(8),np.zeros(8)
    n = 0
    for start in range(0,len(train_times),256):
        ii = train_times[start:start+256]
        b = np.concatenate([np.asarray(slp[ii],dtype='float32')[:,None],q[ii].astype('float32')*scale[None,:,Non
e,None]+offset[None,:,None,None]],1).astype('float64')
        if not np.isfinite(b).all():
            raise ValueError('Nonfinite coarse training fields')
        sums += b.sum((0,2,3)); squares += (b*b).sum((0,2,3)); n += len(ii)*25*33
    mean = sums/n
    std = np.maximum(np.sqrt(np.maximum(squares/n-mean*mean,0)),1e-4)
    m.update(rows=rows, plan=plan, qpath=str(qpath), scale=scale.tolist(), offset=offset.tolist(),
             channels=REUSE_CHANNELS,units=['hPa','m','m/s','m/s','m/s','m/s','m/s','m/s'],
             global_lat=lat.tolist(),global_lon=lon.tolist(),regional_lat=list(np.linspace(15,-15,121)),
             regional_lon=list(np.linspace(-15,15,121)),normalization={'mean':mean.tolist(),'std':std.tolist(),'
fit_split':'train','end_year':2021},
             identity=digest(path),counts=counts,patch_counts=patch_counts)
    print(json.dumps({'all_archive_windows':counts,'native_pressure_windows':patch_counts,'architecture':REUSE_A
RCH}),flush=True)
    return m


class ReuseDataset(Dataset):
    def __init__(self,m,split,training=False):
        self.m = m
        self.rows = [r for r in m['rows'] if r['split']==split]
        self.arrays = None

    def __len__(self):
        return len(self.rows)

    def __getitem__(self,i):
        m,row = self.m,self.rows[i]
        if self.arrays is None:
            self.arrays = {k:np.load(m['files'][k]['resolved'],mmap_mode='r') for k in ['slp','pressure']}
            self.arrays['q'] = np.load(m['qpath'],mmap_mode='r')
            with np.load(m['files']['geography']['resolved'],allow_pickle=False) as geo:
                self.arrays['land'] = geo['land_fraction']
                self.arrays['height'] = geo['elevation_m']/8000
        ai = row['atlas']
        ii = slice(ai-8,ai+21)
        coarse = np.concatenate([np.array(self.arrays['slp'][ii],dtype='float32')[:,None],
              np.array(self.arrays['q'][ii],dtype='float32')*np.array(m['scale'],dtype='float32')[None,:,None,No
ne]+np.array(m['offset'],dtype='float32')[None,:,None,None]],1)
        mean,std =
np.array(m['normalization']['mean'],dtype='float32'),np.array(m['normalization']['std'],dtype='float32')
        coarse = (coarse-mean[None,:,None,None])/std[None,:,None,None]
        pi = row['patch']
        anchor = m['plan']['rows'][pi] if pi>=0 else
{'center_lat':round(row['center'][0]*4)/4,'center_lon':round(row['center'][1]*4)/4}
        lat = anchor['center_lat']+np.linspace(15,-15,121,dtype='float32')
        lon = anchor['center_lon']+np.linspace(-15,15,121,dtype='float32')
        yy,xx = np.meshgrid(lat,lon,indexing='ij')
        y = np.rint((90-yy)*4).astype(int); x = np.rint(xx*4).astype(int)%1440
        rs = np.stack([yy/90,xx/180-1,self.arrays['land'][y,x],self.arrays['height'][y,x]]).astype('float32')
        cy,cx = np.meshgrid(m['global_lat'],m['global_lon'],indexing='ij')
        iy = np.rint((90-cy)*4).astype(int); ix = np.rint(cx*4).astype(int)%1440
        gs =
np.stack([cy/90,cx/180-1,self.arrays['land'][iy,ix],self.arrays['height'][iy,ix]]).astype('float32')
        detail = np.zeros((29,1,121,121),dtype='float32')
        if pi>=0:
            detail[:,0] = (np.array(self.arrays['pressure'][pi],dtype='float32')-mean[0])/std[0]
            if not np.isfinite(detail).all():
                raise ValueError('Native pressure contains nonfinite fields')
        inputs = {'global_history':coarse[:9], 'regional_history':detail[:9],
                  'global_static':gs,'regional_static':rs,'detail_available':np.array([pi>=0],dtype='float32'),

'center':np.array(row['center'],dtype='float32'),'motion':np.array(row['motion'],dtype='float32'),
```

```
                  'issue_intensity':np.array(row['issue_intensity'],dtype='float32'),'issue_mask':np.array(row['issue_mask'],dtype='float32')}
        targets = {'global':coarse[9:],'global_mask':np.ones_like(coarse[9:]),
'regional_lat_weight':np.broadcast_to(np.cos(np.deg2rad(lat))[None,None,:,None],(20,1,121,1)).astype('float32'),
                   'regional':detail[9:],'regional_mask':np.full_like(detail[9:],float(pi>=0)),
                   'center':np.array(row['target_centers'],dtype='float32'),
                   'storm_mask':np.array(row['route_mask'],dtype='float32')}
        for k in ['pressure','pressure_mask','vmax','vmax_mask']:
            targets[k] = np.array(row[k],dtype='float32')
        return {k:torch.from_numpy(v.copy()) for k,v in inputs.items()},{k:torch.from_numpy(v.copy()) for k,v in targets.items()}


class ReuseForecaster(nn.Module):
    def __init__(self,m,tiny=False):
        super().__init__()
        self.global_time = nn.Conv2d(8*9,8,1)
        self.regional_time = nn.Conv2d(9,1,1)
        self.global_operator = FieldOperator(8,8 if tiny else 80,False,1 if tiny else 3,tiny)
        self.regional_operator = FieldOperator(1,8 if tiny else 88,False,0,tiny)
        self.parent_fusion = nn.Conv2d(1+8+1,1,1)
        self.object_score = nn.Conv2d(32+3,1,1)
        self.intensity = nn.Sequential(nn.Linear(32+3+4,64),nn.GELU(),nn.Linear(64,2))
self.register_buffer('mean',torch.tensor(m['normalization']['mean'],dtype=torch.float32)[None,:,None,None])
self.register_buffer('std',torch.tensor(m['normalization']['std'],dtype=torch.float32)[None,:,None,None])
        yy,xx = np.meshgrid(m['global_lat'],m['global_lon'],indexing='ij')
        self.register_buffer('yy',torch.tensor(yy,dtype=torch.float32)[None])
        self.register_buffer('xx',torch.tensor(xx,dtype=torch.float32)[None])

    def initial(self,inputs):
        required = {'global_history','regional_history','global_static','regional_static','detail_available','center','motion','issue_intensity','issue_mask'}
        if set(inputs)!=required:
            raise ValueError('Only declared causal inputs are allowed')
        gh,rh = inputs['global_history'],inputs['regional_history']
        rs = inputs['regional_static']
        yy,xx = rs[:,0]*90,(rs[:,1]+1)*180
        grid = torch.stack([(xx-100)/40-1,1-yy/30],-1)
        return {'g':gh[:,-1],'r':rh[:,-1],'gc':self.global_time(gh.flatten(1,2)),
                'rc':self.regional_time(rh.flatten(1,2)),'gs':inputs['global_static'],'rs':rs,
                'grid':grid,'available':inputs['detail_available'],'center':inputs['center'],
                'motion':inputs['motion'],'issue':inputs['issue_intensity'],'issue_mask':inputs['issue_mask']}

    def step(self,s):
        g,features = self.global_operator(s['g'],s['gc'],torch.ones_like(s['g']),s['gs'])
        valid = (s['grid'].abs()<=1).all(-1)[:,None].to(g.dtype)
        parent = F.grid_sample(g,s['grid'],align_corners=True,padding_mode='zeros')*valid
        r = s['r']
        if bool(s['available'].any()):
            context = self.parent_fusion(torch.cat([s['rc'],parent,valid],1))
            r,_ = self.regional_operator(r,context,torch.ones_like(r),s['rs'])
        physical = g.float()*self.std+self.mean
        p = physical[:,0]
        anomaly = p-F.avg_pool2d(F.pad(p[:,None],(2,2,2,2),mode='replicate'),5,1)[:,0]
        evidence = torch.cat([features.float(),anomaly[:,None]/10,physical[:,2:4]/30],1)
        lat0,lon0 = s['center'][:,0,None,None],s['center'][:,1,None,None]
        plat = lat0+s['motion'][:,1,None,None]/111.2
        plon = lon0+s['motion'][:,0,None,None]/(111.2*lat0.deg2rad().cos().clamp_min(.2))
        d2 = ((self.yy-plat)*111.2).square()+((self.xx-plon)*111.2*lat0.deg2rad().cos()).square()
        score = self.object_score(evidence)[:,0]-anomaly/10-d2/(2*600**2)
        peak = score.flatten(1).argmax(1)
        iy,ix = torch.arange(25,device=g.device)[None,:,None],torch.arange(33,device=g.device)[None,None,:]
        local = ((iy-(peak//33)[:,None,None]).abs()<=1)&((ix-(peak%33)[:,None,None]).abs()<=1)
        pdf = torch.softmax(score.masked_fill(~local,-1e9).flatten(1),1).reshape_as(score)
        center = torch.stack([(pdf*self.yy).sum((1,2)),(pdf*self.xx).sum((1,2))],1)
        pooled = (evidence*pdf[:,None]).sum((2,3))
        scalar = torch.cat([s['issue']/torch.tensor([100.,1000.],device=g.device),s['issue_mask']],1)
        intensity = self.intensity(torch.cat([pooled,scalar],1))
        grid_p = (p*pdf).sum((1,2))
        dy,dx = s['rs'][:,0]*90,(s['rs'][:,1]+1)*180
        sample = torch.stack([2*(center[:,1]-dx[:,0,0])/30-1,2*(dy[:,0,0]-center[:,0])/30-1],-1)[:,None,None]
        inside = (sample.abs()<=1).all((1,2,3)) & (s['available'][:,0]>0)
        detailed_p = F.grid_sample(r.float()*self.std[:,:1]+self.mean[:,:1],sample,align_corners=True)[:,0,0,0]
        grid_p = torch.where(inside,detailed_p,grid_p)
        pressure = grid_p-F.softplus(intensity[:,0])*5
        vmax = s['issue'][:,0]*s['issue_mask'][:,0]+intensity[:,1]*10
        meanlat = (center[:,0]+s['center'][:,0])/2
        motion = torch.stack([(center[:,1]-s['center'][:,1])*111.2*meanlat.deg2rad().cos(),(center[:,0]-s['center'][:,0])*111.2],1)
        return dict(s,g=g,r=r,center=center,motion=motion),{'global':g,'regional':r,'center':center,'pressure':pressure,'vmax':vmax,'grid_pressure':grid_p}

    def forward(self,inputs,steps=20):
```

```
        s = self.initial(inputs)
        out = []
        for _ in range(steps):
            s,o = self.step(s); out.append(o)
        return {k:torch.stack([o[k] for o in out],1) for k in out[0]}


def reuse_main():
    global ARCHITECTURE, AtmosphereDataset
    ARCHITECTURE,AtmosphereDataset = REUSE_ARCH,ReuseDataset
    p = argparse.ArgumentParser(description='v1.2.64 coarse weather + native pressure training; no pilot fallback')
    p.add_argument('batch',type=int,nargs='?',default=10)
    p.add_argument('workers',type=int,nargs='?',default=3)
    p.add_argument('amp',type=int,nargs='?',default=1,choices=[0,1])
    p.add_argument('--manifest',type=Path,default=ROOT/'data/v164_reuse/manifest.json')
    persistent = Path('/content/drive/MyDrive/typhoon_predict') if Path('/content').is_dir() else ROOT
    p.add_argument('--output',type=Path,default=persistent/'training/v164_reuse_pressure')
    p.add_argument('--epochs',type=int,default=12)
    p.add_argument('--device',default='auto',choices=['auto','cpu','mps','cuda'])
    p.add_argument('--preflight',action='store_true')
    args = p.parse_args()
    if args.batch<1 or args.workers<0 or args.epochs<1:
        p.error('Invalid batch/workers/epochs')
    m = load_reuse(args.manifest)
    for split in ['train','validation','test']:
        d = ReuseDataset(m,split)
        d[0]
        d[next(i for i,r in enumerate(d.rows) if r['patch']>=0)]
    if args.preflight:
        print('PASS combined non-pilot archives and native pressure data. No training started.',flush=True)
        return
    if Path('/content').is_dir() and not args.output.resolve().is_relative_to(persistent.resolve()):
        raise ValueError('Colab checkpoints must persist in typhoon_predict on Drive')
    device = device_for(args.device)
    random.seed(164); np.random.seed(164); torch.manual_seed(164)
    model = ReuseForecaster(m).to(device)
    print(json.dumps({'version':VERSION,'architecture':REUSE_ARCH,'parameters':sum(x.numel() for x in model.parameters()),'device':str(device),'windows':m['counts'],'native_windows':m['patch_counts'],'output':str(args.output)}),flush=True)
    optimizer = torch.optim.AdamW(model.parameters(),lr=1e-4,weight_decay=.01)
    scaler = torch.amp.GradScaler('cuda',enabled=device.type=='cuda' and bool(args.amp))
    args.output.mkdir(parents=True,exist_ok=True)
    lock = args.output/'training.lock'
    fd = os.open(lock,os.O_CREAT|os.O_EXCL|os.O_WRONLY)
    os.write(fd,json.dumps({'pid':os.getpid(),'host':socket.gethostname()}).encode()); os.close(fd)
    try:
        train(model,optimizer,scaler,m,args,device)
    finally:
        lock.unlink(missing_ok=True)

if __name__ == "__main__":
    reuse_main()
```

B37. training/trackformer_1_2/source/data_preparation/v164_prepare_data.py

19,312 bytes | SHA-256: c1746e0c0f985890a0f02061a2d84b521cc7b29042c92972aa1ebab3354e4c95

```
"""Prepare a resumable, explicitly labelled v164 data pilot from public ERA5.

This downloads native fields. It does not upsample the old coarse archive,
start training, or put verification labels into model inputs.
"""
import argparse
import hashlib
import json
import os
from pathlib import Path
import shutil
import time

import numpy as np
import v164_global_regional_train as trainer

SOURCE = 'https://storage.googleapis.com/gcp-public-data-arco-era5/ar/full_37-1h-0p25deg-chunk-1.zarr-v3'
TRACK_SHA = 'cb8d7291aa8140124ca522feb448a1d44124b018efc4ba1d54824b60a4f73451'
SURFACE = ['mean_sea_level_pressure', 'surface_pressure', '10m_u_component_of_wind',
           '10m_v_component_of_wind', '2m_temperature', 'sea_surface_temperature']
UPPER = ['u_component_of_wind', 'v_component_of_wind', 'temperature', 'specific_humidity', 'geopotential']
STEP = 6 * trainer.HOUR


def year(ns):
    return int(np.datetime64(int(ns), 'ns').astype('datetime64[Y]').astype(int)) + 1970


def build_plan(track_path, counts=(3, 1, 1), oversample=1):
    if trainer.digest(track_path) != TRACK_SHA:
        raise ValueError('Shared track archive checksum mismatch')
    with np.load(track_path, allow_pickle=False) as z:
        tracks = {k: z[k] for k in ('storm_id', 'base_time', 'base_lat', 'base_lon', 'n_leads', 'target',
'target_mask')}
        issue_intensity = z['track'][:, -1, 4:6]*z['track_std'][4:6] + z['track_mean'][4:6]
    sid, times = tracks['storm_id'], tracks['base_time']
    first_year = {}
    for storm, ns in zip(sid, times):
        first_year[str(storm)] = min(first_year.get(str(storm), 9999), year(ns))
    lookup = {(str(s), int(t)): i for i, (s, t) in enumerate(zip(sid, times))}
    rows = []
    rng = np.random.default_rng(164)
    for split, bounds, count in zip(('train', 'validation', 'test'), ((2000, 2021), (2022, 2023), (2024, 2025)),
 counts):
        candidates = {}
        for i, (storm, ns) in enumerate(zip(sid, times)):
            storm, ns = str(storm), int(ns)
            lat, lon = float(tracks['base_lat'][i]), float(tracks['base_lon'][i]) % 360
            if not (bounds[0] <= first_year[storm] <= bounds[1] and bounds[0] <= year(ns-8*STEP)
                    and year(ns+20*STEP) <= bounds[1] and ns % STEP == 0 and tracks['n_leads'][i] >= 20
                    and issue_intensity[i, 0] >= 90 and 850 < issue_intensity[i, 1] <= 960
                    and 0 < lat < 40 and 100 < lon < 180 and '1979' not in storm and storm.upper() != 'TIP'):
                continue
            future = [lookup.get((storm, ns + lead*STEP)) for lead in range(1, 21)]
            if sum(j is not None for j in future) < 16:
                continue
            candidates.setdefault(storm, []).append((i, future))
        storms = sorted(candidates)
        if len(storms) < count:
            raise ValueError(f'Not enough eligible {split} storms: {len(storms)} < {count}')
        for storm in rng.choice(storms, size=min(len(storms), count*oversample), replace=False):
            choices = candidates[storm]
            i, future = min(choices, key=lambda item: float(issue_intensity[item[0], 1]))
            p, vmax = tracks['target'][i, :, 3], tracks['target'][i, :, 2]
            pm = tracks['target_mask'][i, :, 3] & np.isfinite(p) & (p > 800) & (p < 1100)
            vm = tracks['target_mask'][i, :, 2] & np.isfinite(vmax) & (vmax >= 0) & (vmax < 250)
            rows.append({'issue_ns': int(times[i]), 'split': split, 'storm_id': str(storm),
                         'verification_issue_center': [float(tracks['base_lat'][i]),
float(tracks['base_lon'][i]) % 360],
                         'target_centers': [[float(tracks['base_lat'][j]), float(tracks['base_lon'][j]) % 360]
                                            if j is not None else [0., 0.] for j in future],
                         'route_mask': [int(j is not None) for j in future],
                         'target_pressure_hpa': np.where(pm, p, 0).tolist(), 'pressure_mask':
pm.astype(int).tolist(),
                         'target_vmax_kt': np.where(vm, vmax, 0).tolist(), 'vmax_mask':
vm.astype(int).tolist()})
    # An unconditional atmospheric sample, not a claim that no cyclone exists.
    rows.append({'issue_ns': int(np.datetime64('2021-01-15T00', 'ns').astype('int64')), 'split': 'train',
                 'storm_id': None, 'sample_kind': 'unconditional_atmosphere'})
    stamps = sorted({row['issue_ns'] + i*STEP for row in rows for i in range(-8, 21)})
```

```
    indices = {t: i for i, t in enumerate(stamps)}
    for row in rows:
        row['issue_index'] = indices[row['issue_ns']]
    result = {'schema': 'v164-preparation-plan-v1', 'dataset_role': 'pipeline_pilot_not_full_training',
              'source': SOURCE, 'track_sha256': TRACK_SHA, 'rows': rows, 'time': stamps,
              'estimated_field_bytes': len(stamps)*31*(181*360+241*321)*4,
              'network_warning': 'ARCO upper-air chunks contain all 37 levels and the globe; transfer can greatly exceed saved size.',
              'sampling': 'mature intense issue-time storms for association testing; not representative benchmark sampling',
              'counts': list(counts)}
    result['plan_sha256'] = hashlib.sha256(json.dumps(result, sort_keys=True).encode()).hexdigest()
    return result


def resolve_associations(plan, source):
    selected, rejected = [], []
    for split, required in zip(('train', 'validation', 'test'), plan['counts']):
        accepted = 0
        for row in plan['rows']:
            if row['split'] != split or not row.get('storm_id'):
                continue
            print('Checking issue-time analysis association:', split, row['storm_id'], flush=True)
            pressure = source.domains(source.read('mean_sea_level_pressure', row['issue_ns'])/100)[1]
            center = analysis_center(pressure[None], source.rl, source.ro)
            truth = np.asarray(row['verification_issue_center'])
            distance = float(np.linalg.norm((center-truth)*[111.2, 111.2*np.cos(np.deg2rad(truth[0]))]))
            if distance > 400:
                rejected.append({'storm_id': row['storm_id'], 'issue_ns': row['issue_ns'], 'distance_km': distance})
                continue
            selected.append(dict(row))
            accepted += 1
            if accepted == required:
                break
        if accepted < required:
            raise ValueError(f'Insufficient unambiguous issue-time associations for {split}; no dataset was published')
    selected.extend(dict(r) for r in plan['rows'] if not r.get('storm_id'))
    times = sorted({r['issue_ns']+i*STEP for r in selected for i in range(-8, 21)})
    lookup = {t: i for i, t in enumerate(times)}
    for row in selected:
        row['issue_index'] = lookup[row['issue_ns']]
    result = dict(plan, rows=selected, time=times, rejected_issue_associations=rejected,
                  estimated_field_bytes=len(times)*31*(181*360+241*321)*4)
    result.pop('plan_sha256', None)
    result['plan_sha256'] = hashlib.sha256(json.dumps(result, sort_keys=True).encode()).hexdigest()
    return result


class Era5Source:
    def __init__(self):
        import xarray as xr
        self.ds = xr.open_zarr(SOURCE, consolidated=True, chunks=None)
        missing = set(SURFACE + UPPER + ['land_sea_mask', 'geopotential_at_surface']) - set(self.ds)
        if missing:
            raise ValueError(f'ERA5 missing variables: {sorted(missing)}')
        if not set(trainer.LEVELS).issubset(set(self.ds.level.values.tolist())):
            raise ValueError('ERA5 does not supply all required pressure levels')
        self.lat, self.lon = self.ds.latitude.values, self.ds.longitude.values
        if not np.allclose(self.lat, np.linspace(90, -90, 721)) or not np.allclose(self.lon, np.arange(1440)*.25):
            raise ValueError('Unexpected ERA5 native grid; no implicit regridding')
        self.gl, self.go = self.lat[::4], self.lon[::4]
        self.ys = np.flatnonzero((self.lat >= 0) & (self.lat <= 60))
        self.xs = np.flatnonzero((self.lon >= 100) & (self.lon <= 180))
        self.rl, self.ro = self.lat[self.ys], self.lon[self.xs]

    def read(self, name, ns, levels=None):
        date = np.datetime64(int(ns), 'ns')
        stop = np.datetime64(self.ds.attrs['valid_time_stop'])
        if date > stop or date < np.datetime64(self.ds.attrs['valid_time_start']):
            raise ValueError(f'{date} outside final ERA5 availability; no ERA5T substitution')
        a = self.ds[name].sel(time=date)
        if levels is not None:
            a = a.sel(level=list(levels)).transpose('level', 'latitude', 'longitude')
        else:
            a = a.transpose('latitude', 'longitude')
        for attempt in range(3):
            try:
                return np.asarray(a.values, dtype='float32')
            except (OSError, TimeoutError, ConnectionError):
                if attempt == 2:
                    raise
                time.sleep(2**attempt)

```

```
    def domains(self, a):
        return a[..., ::4, ::4], a[..., self.ys[:, None], self.xs[None, :]]

    def frame(self, ns):
        out = {name: np.empty((31, len(lat), len(lon)), dtype='float32')
               for name, lat, lon in [('global', self.gl, self.go), ('regional', self.rl, self.ro)]}
        for i, name in enumerate(SURFACE):
            print('  reading', name, flush=True)
            a = self.read(name, ns)
            if i < 2:
                a = a / 100.0
            if i == 5:
                a = np.where((a >= 260) & (a <= 330), a, np.nan)
            for domain, values in zip(out, self.domains(a)):
                out[domain][i] = values
        for j, name in enumerate(UPPER):
            print('  reading', name, 'levels', trainer.LEVELS, flush=True)
            a = self.read(name, ns, trainer.LEVELS)
            for domain, values in zip(out, self.domains(a)):
                for level in range(5):
                    out[domain][6+level*5+j] = values[level]
        for name, values in out.items():
            validate_frame(values, name)
        return out

    def static(self, ns):
        land = np.clip(self.read('land_sea_mask', ns), 0, 1)
        altitude = self.read('geopotential_at_surface', ns)/9.80665
        result = {}
        for i, (name, lat, lon) in enumerate([('global', self.gl, self.go), ('regional', self.rl, self.ro)]):
            yy, xx = np.meshgrid(lat, lon, indexing='ij')
            result[name] = np.stack([yy/90, xx/180-1, self.domains(land)[i],
self.domains(altitude)[i]/8000]).astype('float32')
        return result


def validate_frame(a, name):
    if not np.isfinite(a[[i for i in range(31) if i != 5]]).all():
        raise ValueError(f'Nonfinite {name} atmospheric field; refusing zero-fill')
    if not (800 < a[0].min() <= a[0].max() < 1120):
        raise ValueError(f'{name} MSLP range/unit failure')
    if not (150 < a[4].min() <= a[4].max() < 350):
        raise ValueError(f'{name} temperature range/unit failure')


def analysis_center(field, lat, lon, previous=None):
    pressure = field[0].copy()
    yy, xx = np.meshgrid(lat, lon, indexing='ij')
    allowed = (yy > 0) & (yy < 40)
    if previous is not None:
        distance = ((yy-previous[0])*111.2)**2 + ((xx-previous[1])*111.2*np.cos(np.deg2rad(previous[0])))**2
        allowed &= distance < 600**2
    index = np.argmin(np.where(allowed, pressure, np.inf))
    return np.array([yy.flat[index], xx.flat[index]], dtype='float32')


def materialize(plan, root, source, max_gb=4, smoke=False):
    root = Path(root)
    root.mkdir(parents=True, exist_ok=True)
    plan_path, progress_path = root/'preparation_plan.json', root/'preparation_progress.json'
    if plan_path.exists() and json.loads(plan_path.read_text())['plan_sha256'] != plan['plan_sha256']:
        raise ValueError('Existing preparation uses a different plan; choose a separate dataset directory')
    needed = plan['estimated_field_bytes']
    if needed > max_gb*1e9:
        raise ValueError(f'Plan needs {needed/1e9:.2f} GB; exceeds explicit {max_gb} GB limit')
    existing = sum(p.stat().st_size for p in root.glob('*.npy'))
    if shutil.disk_usage(root).free < max(0, needed-existing) + (128 if smoke else 512)*1024**2:
        raise OSError('Insufficient storage for data and safety margin')
    trainer.atomic_json(plan_path, plan)
    progress = json.loads(progress_path.read_text()) if progress_path.exists() else {'frames': {}}
    if (root/'manifest.json').exists():
        trainer.load_manifest(root/'manifest.json', smoke=smoke)
        print('Prepared dataset already verified.', flush=True)
        return
    arrays = {}
    for name, lat, lon in [('global', source.gl, source.go), ('regional', source.rl, source.ro)]:
        p = root/(name+'.npy')
        shape = (len(plan['time']), 31, len(lat), len(lon))
        arrays[name] = np.lib.format.open_memmap(p, mode='r+' if p.exists() else 'w+', dtype='float32',
shape=shape)
        if arrays[name].shape != shape:
            raise ValueError('Partial data shape mismatch')
    for i, ns in enumerate(plan['time']):
        key = str(ns)
        def frame_digest():
            h = hashlib.sha256()
```

```
            for a in arrays.values():
                h.update(np.ascontiguousarray(a[i]).tobytes())
            return h.hexdigest()
        if key in progress['frames'] and frame_digest() == progress['frames'][key]:
            continue
        print(f'ERA5 frame {i+1}/{len(plan["time"])}: {np.datetime64(ns, "ns")} (31 channels)', flush=True)
        frame = source.frame(ns)
        for name, a in arrays.items():
            a[i] = frame[name]
            a.flush()
        progress['frames'][key] = frame_digest()
        progress.update(completed=len(progress['frames']), total=len(plan['time']),
last_completed_utc=str(np.datetime64(ns, 'ns')))
        trainer.atomic_json(progress_path, progress)
    np.save(root/'time.npy', np.asarray(plan['time'], dtype='int64'))
    for name, a in source.static(plan['time'][0]).items():
        np.save(root/(name+'_static.npy'), a)
    sums, squares, counts = np.zeros(31), np.zeros(31), np.zeros(31)
    for i, ns in enumerate(plan['time']):
        if not 2000 <= year(ns) <= 2021:
            continue
        for a in arrays.values():
            values = np.asarray(a[i], dtype='float64').reshape(31, -1)
            sums += np.nansum(values, axis=1)
            squares += np.nansum(values**2, axis=1)
            counts += np.isfinite(values).sum(axis=1)
    if (counts == 0).any():
        raise ValueError('Missing training-only normalization data')
    mean = sums/counts
    std = np.maximum(np.sqrt(np.maximum(squares/counts-mean**2, 0)), 1e-6)
    rows = []
    for original in plan['rows']:
        row = dict(original)
        i = row['issue_index']
        # Association is derived solely from issue/past ERA5, never best-track coordinates.
        center = analysis_center(arrays['regional'][i], source.rl, source.ro)
        past = analysis_center(arrays['regional'][i-1], source.rl, source.ro, center)
        row['center'] = center.tolist()
        row['motion_km_per_6h'] = [float((center[1]-past[1])*111.2*np.cos(np.deg2rad(center[0]))),
float((center[0]-past[0])*111.2)]
        truth = row.pop('verification_issue_center', None)
        if truth is not None:
            dist = np.linalg.norm((center-np.array(truth))*[111.2, 111.2*np.cos(np.deg2rad(truth[0]))])
            row['issue_association_error_km'] = float(dist)
            if dist > 400:
                row['route_mask'] = row['pressure_mask'] = row['vmax_mask'] = [0]*20
        rows.append(row)
    (root/'samples.jsonl').write_text(''.join(json.dumps(row)+'\n' for row in rows))
    m = {'schema': 'v164-atmosphere-v1', 'synthetic': bool(smoke), 'dataset_role': plan['dataset_role'],
         'channels': trainer.CHANNELS, 'units': trainer.UNITS,
         'global_lat': source.gl.tolist(), 'global_lon': source.go.tolist(),
         'regional_lat': source.rl.tolist(), 'regional_lon': source.ro.tolist(),
         'normalization': {'fit_split': 'train', 'end_year': 2021, 'mean': mean.tolist(), 'std': std.tolist()},
         'future_weather_as_inputs': False, 'official_forecasts_as_inputs': False,
         'inputs_are_analysis_valid_time_hindcasts_not_operational_release_time': True,
         'source': SOURCE, 'track_archive_sha256': plan['track_sha256'], 'files': {}}
    for name in ('global', 'regional', 'global_static', 'regional_static', 'time', 'samples'):
        p = root/(name+('.jsonl' if name == 'samples' else '.npy'))
        m['files'][name] = {'path': p.name, 'sha256': trainer.digest(p)}
    for split in ('train', 'validation', 'test'):
        if not any(row['split'] == split and any(row.get('route_mask', [])) for row in rows):
            raise ValueError(f'No valid analysis-associated storm labels in {split}. Pilot is insufficient; do
not train it.')
    # Validate a temporary manifest before publishing the ready marker.
    candidate = root/'manifest.pending.json'
    trainer.atomic_json(candidate, m)
    trainer.load_manifest(candidate, smoke=smoke)
    candidate.replace(root/'manifest.json')
    print('PILOT DATA READY. This is not the full training corpus or evidence of forecast skill.', flush=True)


def main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument('--track', type=Path, required=True)
    p.add_argument('--output', type=Path, required=True)
    p.add_argument('--prepare', action='store_true')
    p.add_argument('--max-gb', type=float, default=4)
    p.add_argument('--cases', type=int, nargs=3, default=[3, 1, 1], metavar=('TRAIN', 'VAL', 'TEST'))
    args = p.parse_args()
    if min(args.cases) < 1:
        p.error('Each split needs at least one case')
    plan = build_plan(args.track, args.cases, oversample=4 if args.prepare else 1)
    if args.prepare:
        args.output.mkdir(parents=True, exist_ok=True)
        lock = args.output/'preparation.lock'
        fd = os.open(lock, os.O_CREAT | os.O_EXCL | os.O_WRONLY)
```

```
        os.close(fd)
        try:
            source = Era5Source()
            previous = args.output/'preparation_plan.json'
            if previous.exists():
                plan = json.loads(previous.read_text())
                if plan['track_sha256'] != TRACK_SHA or plan['counts'] != args.cases or plan['source'] != SOURCE:
                    raise ValueError('Existing preparation configuration differs; choose a new output directory')
                saved_hash = plan['plan_sha256']
                payload = {k: v for k, v in plan.items() if k != 'plan_sha256'}
                if hashlib.sha256(json.dumps(payload, sort_keys=True).encode()).hexdigest() != saved_hash:
                    raise ValueError('Existing preparation plan checksum mismatch')
            else:
                plan = resolve_associations(plan, source)
            print(json.dumps({k: v for k, v in plan.items() if k not in ('rows', 'time')}, indent=2), flush=True)
            materialize(plan, args.output, source, args.max_gb)
        finally:
            lock.unlink(missing_ok=True)
    else:
        print(json.dumps({k: v for k, v in plan.items() if k not in ('rows', 'time')}, indent=2), flush=True)
        print('Plan only. Add --prepare to download. No training is started.', flush=True)


if __name__ == '__main__':
    main()
```

B38. training/trackformer_1_2/source/data_preparation/v164_prepare_mac.py

1,309 bytes | SHA-256: 7e7313f865369d42f67645196b4af0e1b2b84216303f7b2b61e577d3734df79a

```
"""Mac data preparation with three concurrent source-variable reads.

Uses the same plan, fields, validation and resume records as the serial preparer.
This utility is not called by the Drive-only Colab notebook.
"""
from concurrent.futures import ThreadPoolExecutor

import v164_prepare_data as prep


class ConcurrentEra5Source(prep.Era5Source):
    _pending = None

    def read(self, name, ns, levels=None):
        key = (name, int(ns), tuple(levels) if levels is not None else None)
        if self._pending is not None and key in self._pending:
            return self._pending.pop(key).result()
        return super().read(name, ns, levels)

    def frame(self, ns):
        jobs = [(name, None) for name in prep.SURFACE]
        jobs += [(name, tuple(prep.trainer.LEVELS)) for name in prep.UPPER]
        with ThreadPoolExecutor(max_workers=3) as pool:
            self._pending = {(name, int(ns), levels): pool.submit(super().read, name, ns, levels)
                             for name, levels in jobs}
            try:
                return super().frame(ns)
            finally:
                for future in self._pending.values():
                    future.cancel()
                self._pending = None


if __name__ == '__main__':
    prep.Era5Source = ConcurrentEra5Source
    prep.main()
```

B39. training/trackformer_1_2/source/data_preparation/v164_reuse_impl.py

18,414 bytes | SHA-256: 2da0078d4a74faa93f8cf874e42707b01f286406f0ceaa4db1814aab0eceb7d1

```
"""Embedded in the standalone v164 trainer: coarse archive plus native MSLP patches."""
REUSE_ARCH = 'wp-coarse8-native-pressure1-v164'
REUSE_CHANNELS = ['mslp', 'hgt500', 'uwnd850', 'vwnd850', 'uwnd500', 'vwnd500', 'uwnd200', 'vwnd200']


def load_reuse(path):
    path = Path(path)
    if not path.is_file():
        raise FileNotFoundError('Combined dataset missing. Run the updated Drive setup cell first: '+str(path))
    m = json.loads(path.read_text())
    if m.get('schema') != 'v164-reuse-pressure-v1' or m.get('pilot', True):
        raise ValueError('Combined non-pilot archive required; no pilot fallback')
    for name, entry in m['files'].items():
        p = path.parent/entry['path']
        print('Checking dataset file:', name, flush=True)
        if digest(p) != entry['sha256']:
            raise ValueError('Dataset checksum mismatch: '+name)
        entry['resolved'] = str(p)
    with np.load(m['files']['basin']['resolved'], allow_pickle=False) as z:
        if z['channels'].tolist() != REUSE_CHANNELS[1:]:
            raise ValueError('Coarse channel order mismatch')
        times, lat, lon = z['time'], z['lat'], z['lon']
        q = z['q']
        scale, offset = z['scale'], z['offset']
    if q.shape != (len(times),7,25,33) or not np.all(np.diff(times) == 6*HOUR):
        raise ValueError('Coarse grid/time mismatch')
    qpath = path.parent/'basin_q_verified.npy'
    # Refresh the compact worker mmap from the checksum-verified source every preflight.
    temporary = qpath.with_suffix('.pending.npy')
    np.save(temporary, q)
    os.replace(temporary, qpath)
    slp = np.load(m['files']['slp']['resolved'], mmap_mode='r')
    if slp.shape != (len(times),25,33) or not np.isfinite(slp).all():
        raise ValueError('Coarse pressure coverage invalid')
    with np.load(m['files']['track']['resolved'], allow_pickle=False) as z:
        track = {k:z[k] for k in ['base_time','storm_id','base_lat','base_lon','target','target_mask']}
        issue_intensity = z['track'][:,-1,4:6]*z['track_std'][4:6]+z['track_mean'][4:6]
    plan = json.loads(Path(m['files']['plan']['resolved']).read_text())
    if len(plan['rows']) != 1000 or plan['shape'] != [1000,29,121,121] or plan['lead_hours'] !=
list(range(-48,121,6)):
        raise ValueError('Expected the verified 1000-window native pressure package, not the six-sample pilot')
    pressure = np.load(m['files']['pressure']['resolved'], mmap_mode='r')
    if pressure.shape != (1000,29,121,121):
        raise ValueError('Native pressure shape mismatch')
    patches = {}
    for pi,row in enumerate(plan['rows']):
        ti = row['track_archive_row']
        if int(track['base_time'][ti]) != row['issue_ns'] or str(track['storm_id'][ti]) != row['storm_id']:
            raise ValueError('Native patch/track identity mismatch')
        patches[ti] = pi
    first_year = {}
    years = track['base_time'].astype('datetime64[ns]').astype('datetime64[Y]').astype(int)+1970
    for sid,y in zip(track['storm_id'],years):
        first_year[str(sid)] = min(first_year.get(str(sid),9999),int(y))
    lookup = {(str(s),int(t)):i for i,(s,t) in enumerate(zip(track['storm_id'],track['base_time']))}
    rows = []
    for i,ns in enumerate(track['base_time']):
        sid = str(track['storm_id'][i])
        y = first_year[sid]
        split = 'train' if 2000 <= y <= 2021 else 'validation' if 2022 <= y <= 2023 else 'test' if 2024 <= y <=
2025 else None
        if split is None:
            continue
        low,high = {'train':(2000,2021),'validation':(2022,2023),'test':(2024,2025)}[split]
        bounds = np.array([ns-48*HOUR,ns+120*HOUR],dtype='int64').astype('datetime64[ns]').astype('datetime64[Y]
').astype(int)+1970
        ai = int(np.searchsorted(times,ns))
        center = [float(track['base_lat'][i]),float(track['base_lon'][i])%360]
        if bounds[0] < low or bounds[1] > high or ai < 8 or ai+20 >= len(times) or times[ai] != ns:
            continue
        if not np.isfinite(center).all() or not (0 < center[0] < 60 and 100 < center[1] < 180):
            continue
        previous = lookup.get((sid,int(ns-6*HOUR)))
        motion = [0.,0.]
        if previous is not None:
            old = [float(track['base_lat'][previous]),float(track['base_lon'][previous])%360]
            if np.isfinite(old).all():
                motion = [(center[1]-old[1])*111.2*np.cos(np.deg2rad(center[0])),(center[0]-old[0])*111.2]
        future = [lookup.get((sid,int(ns+j*6*HOUR))) for j in range(1,21)]
        target_centers = [[float(track['base_lat'][j]),float(track['base_lon'][j])%360] if j is not None else
```

```
    [0.,0.] for j in future]
         rm = [float(j is not None and np.isfinite(target_centers[k]).all()) for k,j in enumerate(future)]
         target_centers = np.nan_to_num(target_centers).tolist()
         wind,p = track['target'][i,:,2],track['target'][i,:,3]
         wm = track['target_mask'][i,:,2].astype(bool)&np.isfinite(wind)&(wind>=0)&(wind<250)
         pm = track['target_mask'][i,:,3].astype(bool)&np.isfinite(p)&(p>800)&(p<1100)
         pi = patches.get(i,-1)
         if pi >= 0 and plan['rows'][pi]['split'] != split:
             raise ValueError('Patch split mismatch')
         issue = issue_intensity[i]
         iv = [bool(np.isfinite(issue[0]) and 0<=issue[0]<250),bool(np.isfinite(issue[1]) and 800<issue[1]<1100)]
         rows.append({'split':split,'storm_id':sid,'atlas':ai,'patch':pi,'center':center,'motion':motion,
                      'issue_intensity':[float(issue[k]) if iv[k] else 0. for k in range(2)],'issue_mask':iv,
                      'target_centers':target_centers,'route_mask':rm,
                      'pressure':np.where(pm,p,0).tolist(),'pressure_mask':pm.tolist(),
                      'vmax':np.where(wm,wind,0).tolist(),'vmax_mask':wm.tolist()})
     counts = {s:sum(r['split']==s for r in rows) for s in ['train','validation','test']}
     patch_counts = {s:sum(r['split']==s and r['patch']>=0 for r in rows) for s in counts}
     if counts['train'] < 10000 or patch_counts != {'train':800,'validation':100,'test':100}:
         raise ValueError(f'Incomplete combined coverage: {counts}, native patches {patch_counts}')
     train_times =
np.flatnonzero((times.astype('datetime64[ns]').astype('datetime64[Y]').astype(int)+1970>=2000)&
                                (times.astype('datetime64[ns]').astype('datetime64[Y]').astype(int)+1970<=2021))
     sums,squares = np.zeros(8),np.zeros(8)
     n = 0
     for start in range(0,len(train_times),256):
         ii = train_times[start:start+256]
         b = np.concatenate([np.asarray(slp[ii],dtype='float32')[:,None],q[ii].astype('float32')*scale[None,:,Non
e,None]+offset[None,:,None,None]],1).astype('float64')
         if not np.isfinite(b).all():
             raise ValueError('Nonfinite coarse training fields')
         sums += b.sum((0,2,3)); squares += (b*b).sum((0,2,3)); n += len(ii)*25*33
     mean = sums/n
     std = np.maximum(np.sqrt(np.maximum(squares/n-mean*mean,0)),1e-4)
     m.update(rows=rows, plan=plan, qpath=str(qpath), scale=scale.tolist(), offset=offset.tolist(),
              channels=REUSE_CHANNELS,units=['hPa','m','m/s','m/s','m/s','m/s','m/s','m/s'],
              global_lat=lat.tolist(),global_lon=lon.tolist(),regional_lat=list(np.linspace(15,-15,121)),
              regional_lon=list(np.linspace(-15,15,121)),normalization={'mean':mean.tolist(),'std':std.tolist(),'
fit_split':'train','end_year':2021},
              identity=digest(path),counts=counts,patch_counts=patch_counts)
     print(json.dumps({'all_archive_windows':counts,'native_pressure_windows':patch_counts,'architecture':REUSE_A
RCH}),flush=True)
     return m


class ReuseDataset(Dataset):
     def __init__(self,m,split,training=False):
         self.m = m
         self.rows = [r for r in m['rows'] if r['split']==split]
         self.arrays = None

     def __len__(self):
         return len(self.rows)

     def __getitem__(self,i):
         m,row = self.m,self.rows[i]
         if self.arrays is None:
             self.arrays = {k:np.load(m['files'][k]['resolved'],mmap_mode='r') for k in ['slp','pressure']}
             self.arrays['q'] = np.load(m['qpath'],mmap_mode='r')
             with np.load(m['files']['geography']['resolved'],allow_pickle=False) as geo:
                 self.arrays['land'] = geo['land_fraction']
                 self.arrays['height'] = geo['elevation_m']/8000
         ai = row['atlas']
         ii = slice(ai-8,ai+21)
         coarse = np.concatenate([np.array(self.arrays['slp'][ii],dtype='float32')[:,None],
               np.array(self.arrays['q'][ii],dtype='float32')*np.array(m['scale'],dtype='float32')[None,:,None,No
ne]+np.array(m['offset'],dtype='float32')[None,:,None,None]],1)
         mean,std =
np.array(m['normalization']['mean'],dtype='float32'),np.array(m['normalization']['std'],dtype='float32')
         coarse = (coarse-mean[None,:,None,None])/std[None,:,None,None]
         pi = row['patch']
         anchor = m['plan']['rows'][pi] if pi>=0 else
{'center_lat':round(row['center'][0]*4)/4,'center_lon':round(row['center'][1]*4)/4}
         lat = anchor['center_lat']+np.linspace(15,-15,121,dtype='float32')
         lon = anchor['center_lon']+np.linspace(-15,15,121,dtype='float32')
         yy,xx = np.meshgrid(lat,lon,indexing='ij')
         y = np.rint((90-yy)*4).astype(int); x = np.rint(xx*4).astype(int)%1440
         rs = np.stack([yy/90,xx/180-1,self.arrays['land'][y,x],self.arrays['height'][y,x]]).astype('float32')
         cy,cx = np.meshgrid(m['global_lat'],m['global_lon'],indexing='ij')
         iy = np.rint((90-cy)*4).astype(int); ix = np.rint(cx*4).astype(int)%1440
         gs =
np.stack([cy/90,cx/180-1,self.arrays['land'][iy,ix],self.arrays['height'][iy,ix]]).astype('float32')
         detail = np.zeros((29,1,121,121),dtype='float32')
         if pi>=0:
             detail[:,0] = (np.array(self.arrays['pressure'][pi],dtype='float32')-mean[0])/std[0]
             if not np.isfinite(detail).all():
```

```
                raise ValueError('Native pressure contains nonfinite fields')
        inputs = {'global_history':coarse[:9], 'regional_history':detail[:9],
                  'global_static':gs,'regional_static':rs,'detail_available':np.array([pi>=0],dtype='float32'),
'center':np.array(row['center'],dtype='float32'),'motion':np.array(row['motion'],dtype='float32'),
                  'issue_intensity':np.array(row['issue_intensity'],dtype='float32'),'issue_mask':np.array(row['issue_mask'],dtype='float32')}
        targets = {'global':coarse[9:],'global_mask':np.ones_like(coarse[9:]),
'regional_lat_weight':np.broadcast_to(np.cos(np.deg2rad(lat))[None,None,:,None],(20,1,121,1)).astype('float32'),
                   'regional':detail[9:],'regional_mask':np.full_like(detail[9:],float(pi>=0)),
                   'center':np.array(row['target_centers'],dtype='float32'),
                   'storm_mask':np.array(row['route_mask'],dtype='float32')}
        for k in ['pressure','pressure_mask','vmax','vmax_mask']:
            targets[k] = np.array(row[k],dtype='float32')
        return {k:torch.from_numpy(v.copy()) for k,v in inputs.items()},{k:torch.from_numpy(v.copy()) for k,v in targets.items()}


class ReuseForecaster(nn.Module):
    def __init__(self,m,tiny=False):
        super().__init__()
        self.global_time = nn.Conv2d(8*9,8,1)
        self.regional_time = nn.Conv2d(9,1,1)
        self.global_operator = FieldOperator(8,8 if tiny else 80,False,1 if tiny else 3,tiny)
        self.regional_operator = FieldOperator(1,8 if tiny else 88,False,0,tiny)
        self.parent_fusion = nn.Conv2d(1+8+1,1,1)
        self.object_score = nn.Conv2d(32+3,1,1)
        self.intensity = nn.Sequential(nn.Linear(32+3+4,64),nn.GELU(),nn.Linear(64,2))
self.register_buffer('mean',torch.tensor(m['normalization']['mean'],dtype=torch.float32)[None,:,None,None])
self.register_buffer('std',torch.tensor(m['normalization']['std'],dtype=torch.float32)[None,:,None,None])
        yy,xx = np.meshgrid(m['global_lat'],m['global_lon'],indexing='ij')
        self.register_buffer('yy',torch.tensor(yy,dtype=torch.float32)[None])
        self.register_buffer('xx',torch.tensor(xx,dtype=torch.float32)[None])

    def initial(self,inputs):
        required = {'global_history','regional_history','global_static','regional_static','detail_available','center','motion','issue_intensity','issue_mask'}
        if set(inputs)!=required:
            raise ValueError('Only declared causal inputs are allowed')
        gh,rh = inputs['global_history'],inputs['regional_history']
        rs = inputs['regional_static']
        yy,xx = rs[:,0]*90,(rs[:,1]+1)*180
        grid = torch.stack([(xx-100)/40-1,1-yy/30],-1)
        return {'g':gh[:,-1],'r':rh[:,-1],'gc':self.global_time(gh.flatten(1,2)),
                'rc':self.regional_time(rh.flatten(1,2)),'gs':inputs['global_static'],'rs':rs,
                'grid':grid,'available':inputs['detail_available'],'center':inputs['center'],
                'motion':inputs['motion'],'issue':inputs['issue_intensity'],'issue_mask':inputs['issue_mask']}

    def step(self,s):
        g,features = self.global_operator(s['g'],s['gc'],torch.ones_like(s['g']),s['gs'])
        valid = (s['grid'].abs()<=1).all(-1)[:,None].to(g.dtype)
        parent = F.grid_sample(g,s['grid'],align_corners=True,padding_mode='zeros')*valid
        r = s['r']
        if bool(s['available'].any()):
            context = self.parent_fusion(torch.cat([s['rc'],parent,valid],1))
            r,_ = self.regional_operator(r,context,torch.ones_like(r),s['rs'])
        physical = g.float()*self.std+self.mean
        p = physical[:,0]
        anomaly = p-F.avg_pool2d(F.pad(p[:,None],(2,2,2,2),mode='replicate'),5,1)[:,0]
        evidence = torch.cat([features.float(),anomaly[:,None]/10,physical[:,2:4]/30],1)
        lat0,lon0 = s['center'][:,0,None,None],s['center'][:,1,None,None]
        plat = lat0+s['motion'][:,1,None,None]/111.2
        plon = lon0+s['motion'][:,0,None,None]/(111.2*lat0.deg2rad().cos().clamp_min(.2))
        d2 = ((self.yy-plat)*111.2).square()+((self.xx-plon)*111.2*lat0.deg2rad().cos()).square()
        score = self.object_score(evidence)[:,0]-anomaly/10-d2/(2*600**2)
        peak = score.flatten(1).argmax(1)
        iy,ix = torch.arange(25,device=g.device)[None,:,None],torch.arange(33,device=g.device)[None,None,:]
        local = ((iy-(peak//33)[:,None,None]).abs()<=1)&((ix-(peak%33)[:,None,None]).abs()<=1)
        pdf = torch.softmax(score.masked_fill(~local,-1e9).flatten(1),1).reshape_as(score)
        center = torch.stack([(pdf*self.yy).sum((1,2)),(pdf*self.xx).sum((1,2))],1)
        pooled = (evidence*pdf[:,None]).sum((2,3))
        scalar = torch.cat([s['issue']/torch.tensor([100.,1000.],device=g.device),s['issue_mask']],1)
        intensity = self.intensity(torch.cat([pooled,scalar],1))
        grid_p = (p*pdf).sum((1,2))
        dy,dx = s['rs'][:,0]*90,(s['rs'][:,1]+1)*180
        sample = torch.stack([2*(center[:,1]-dx[:,0,0])/30-1,2*(dy[:,0,0]-center[:,0])/30-1],-1)[:,None,None]
        inside = (sample.abs()<=1).all((1,2,3)) & (s['available'][:,0]>0)
        detailed_p = F.grid_sample(r.float()*self.std[:,:1]+self.mean[:,:1],sample,align_corners=True)[:,0,0,0]
        grid_p = torch.where(inside,detailed_p,grid_p)
        pressure = grid_p-F.softplus(intensity[:,0])*5
        vmax = s['issue'][:,0]*s['issue_mask'][:,0]+intensity[:,1]*10
        meanlat = (center[:,0]+s['center'][:,0])/2
        motion = torch.stack([(center[:,1]-s['center'][:,1])*111.2*meanlat.deg2rad().cos(),(center[:,0]-s['cente
```

```
r'][:,0])*111.2],1)
        return dict(s,g=g,r=r,center=center,motion=motion),{'global':g,'regional':r,'center':center,'pressure':pressure,'vmax':vmax,'grid_pressure':grid_p}

    def forward(self,inputs,steps=20):
        s = self.initial(inputs)
        out = []
        for _ in range(steps):
            s,o = self.step(s); out.append(o)
        return {k:torch.stack([o[k] for o in out],1) for k in out[0]}


def reuse_main():
    global ARCHITECTURE, AtmosphereDataset
    ARCHITECTURE,AtmosphereDataset = REUSE_ARCH,ReuseDataset
    p = argparse.ArgumentParser(description='v1.2.64 coarse weather + native pressure training; no pilot fallback')
    p.add_argument('batch',type=int,nargs='?',default=10)
    p.add_argument('workers',type=int,nargs='?',default=3)
    p.add_argument('amp',type=int,nargs='?',default=1,choices=[0,1])
    p.add_argument('--manifest',type=Path,default=ROOT/'data/v164_reuse/manifest.json')
    persistent = Path('/content/drive/MyDrive/typhoon_predict') if Path('/content').is_dir() else ROOT
    p.add_argument('--output',type=Path,default=persistent/'training/v164_reuse_pressure')
    p.add_argument('--epochs',type=int,default=12)
    p.add_argument('--device',default='auto',choices=['auto','cpu','mps','cuda'])
    p.add_argument('--preflight',action='store_true')
    args = p.parse_args()
    if args.batch<1 or args.workers<0 or args.epochs<1:
        p.error('Invalid batch/workers/epochs')
    m = load_reuse(args.manifest)
    for split in ['train','validation','test']:
        d = ReuseDataset(m,split)
        d[0]
        d[next(i for i,r in enumerate(d.rows) if r['patch']>=0)]
    if args.preflight:
        print('PASS combined non-pilot archives and native pressure data. No training started.',flush=True)
        return
    if Path('/content').is_dir() and not args.output.resolve().is_relative_to(persistent.resolve()):
        raise ValueError('Colab checkpoints must persist in typhoon_predict on Drive')
    device = device_for(args.device)
    random.seed(164); np.random.seed(164); torch.manual_seed(164)
    model = ReuseForecaster(m).to(device)
    print(json.dumps({'version':VERSION,'architecture':REUSE_ARCH,'parameters':sum(x.numel() for x in model.parameters()),'device':str(device),'windows':m['counts'],'native_windows':m['patch_counts'],'output':str(args.output)}),flush=True)
    optimizer = torch.optim.AdamW(model.parameters(),lr=1e-4,weight_decay=.01)
    scaler = torch.amp.GradScaler('cuda',enabled=device.type=='cuda' and bool(args.amp))
    args.output.mkdir(parents=True,exist_ok=True)
    lock = args.output/'training.lock'
    fd = os.open(lock,os.O_CREAT|os.O_EXCL|os.O_WRONLY)
    os.write(fd,json.dumps({'pid':os.getpid(),'host':socket.gethostname()}).encode()); os.close(fd)
    try:
        train(model,optimizer,scaler,m,args,device)
    finally:
        lock.unlink(missing_ok=True)
```

B40. training/trackformer_1_2/source/data_preparation/v164_structure_research.md

17,119 bytes | SHA-256: a4b40e50b7d0d1433a162438f14ed110541702c4cec2a386959857293ea78141

```
# TrackFormer v1.2.64: field-first atmosphere and cyclone redesign

Date: 2026-09-13. Status: investigated proposal, NOT a trained or validated v164 model.
No Colab interaction and no training were performed. Diagnostics ran on this Mac's MPS GPU.

## Decision

Change the state representation, atmospheric training coverage, and field/track contract before scaling the current architecture. Do not merely widen v163, remove its tracker, or add more geometric route losses.

The recommended v164 prototype is a global-to-regional, multi-variable atmospheric predictor with a storm-object decoder tied to its generated fields. It must genuinely forecast pressure, not move or paint a pressure template onto a separately predicted route. A stochastic residual extension should follow a verified deterministic baseline, not obscure an input bug.

## What the Tip investigation actually established

Case: 1979-10-12 00 UTC, nine analyses from t-48 through t0, 20 six-hour forecast steps. Checkpoint metadata says epoch 7. The model has **27,179,604 parameters**, despite the `20m` configuration/file label.

Evidence files:

- `diagnostics/v164_tip_audit/report.json`: actual baseline and two controlled inference ablations.
- `diagnostics/v164_tip_audit/baseline.npz`: unmodified checkpoint tensor outputs.
- `diagnostics/v164_tip_audit/grib_verification.json`: issue-time channel checks against original NOAA CFSR GRIB.
- `audit_tip_v163_for_v164.py`: repeatable local diagnostic, with labels excluded from forward inputs.

### Confirmed data/display problems

1. The demo plotted six-hour truth movements as if they were cumulative displacements. All 20 corrected coordinates now match the archived time-tagged realized route within 0.0001 degrees. The true +120 location is 21.5 N, 128.1 E, not approximately 17 N, 138 E. This was a renderer defect, not proof that the training labels were wrong: training already accumulates the step targets.
2. The forecast renderer used predicted latitude to convert eastward displacement, whereas model field sampling uses an issue-latitude tangent plane. The renderer now uses the model's transform. The training step-coordinate versus fixed-grid geometry remains an architectural consistency issue to remove in v164 using one tested spherical coordinate API.
3. The displayed v1.1 and v1.2.26 legacy routes match exactly at every lead. The v1.2.26 renderer imports a legacy v62 artifact, not a freshly identified v1.2.26 checkpoint. They are not two independently verified model replays. A provenance warning was added; this audit does not certify their independent skill or modify the global benchmark.
4. Tip has no SST patch in this demo, so it receives zeros. The training loader filters on `sst_ok`; it does not train this exact missing-data condition. This is a confirmed train/inference contract gap. Its causal contribution requires a matched-source SST replay, not an invented warm ocean patch.

### Checks that passed

- All forward inputs are finite. Weather ranges are plausible: pressure about 972-1031 hPa over history; height about 5116-5922 gpm; winds in m/s. There is no demonstrated Pa/hPa or height/geopotential factor error.
- The nine weather timestamps stop exactly at the issue time.
- All eight issue-time map channels were checked against the original GRIB analysis. Pressure differs by at most 0.000059 hPa after float conversion; height and wind differences are zero on the sampled native grid. GRIB time is the requested issue and lead is zero. Older snapshots were timestamp-checked, not all independently re-decoded here.
- The issue-pressure mask is zero. The grid-pressure fallback therefore follows the training behavior. Missing pressure is not evidence of an incorrectly supplied 870-hPa observation.
- ONI is intentionally masked in both training and this replay until release-dated vintages exist. An empty ONI dictionary is not an accidental missing feature for this checkpoint.

### Measured model failure

These are forecast-structure diagnostics, not future-field accuracy scores. Core minimum means the lowest grid value within 500 km of the predicted center; the ring is 500-1000 km. That region contains only about 9-13 native grid points in the core.

| Lead | Whole-grid minimum, hPa | Core minimum, hPa | Ring mean minus core minimum, hPa |
| --- | ---: | ---: | ---: |
| 0 | 980.425 | 980.425 | 24.993 |
| 24 | 983.234 | 983.234 | 20.906 |
| 48 | 987.474 | 987.474 | 16.002 |
| 72 | 992.260 | 994.568 | 9.411 |
| 120 | 999.023 | 1008.427 | 2.562 |

The generated route ends near 34.30 N, 139.66 E. The scalar central-pressure head says 992.23 hPa, but the field near that route has no comparably deep resolved low. Some difference between a subgrid scalar and coarse reanalysis is legitimate; the combination of lost core contrast, weakening gradients, and diverging route/field location is the issue.

After correcting the displayed truth and projection, descriptive Tip mean geodesic route error is **677.7 km**,
```

with mean heading error **54.6 degrees**. The stored DeepMind route has 140.1 km mean error on those leads, but uses a different analysis pipeline and this historical case is not established as held out for its pretrained weights. Do not call that a controlled generalization benchmark.

Controlled MPS ablations (same checkpoint/inputs, local-distance approximation for this diagnostic):

Variant	Mean route error, km	Interpretation
Original model	678.1	Baseline
Learned acceleration forced to zero	694.9	Does not repair the failure
Field-tracker correction disabled	1106.4	Tracker helps on this case

The baseline mean tracker correction is about 21.3 km per step. These interventions do NOT isolate every possible cause, but disprove the easy claim that deleting the tracker or acceleration is the demonstrated solution. They are diagnostics, not new tuned models.

What remains unproved

- A future, matched-source Tip atmospheric verification sequence was not evaluated here; no whole-map Tip MAE claim is made. Some real weakening is expected, so simply requiring a deep low forever would be wrong.
- CFSR versus the training analysis source is a distribution shift. Finite/range checks do not eliminate it; compare source-matched replays and normalized distributions on many storms.
- This one storm cannot prove overfitting, an optimal parameter count, or guaranteed improvements from any proposed architecture.

Why another v163-style network is unlikely to address the main issue

The actual code uses a 25 x 33, 2.5-degree grid with eight variables: MSLP, height at 500 hPa, and winds at 850/500/200 hPa. A multi-scale encoder cannot recover missing fine-scale or thermodynamic information from that grid. The 0.5-degree source already has a relatively weak resolved low (minimum 976.435 hPa); sampling to 2.5 degrees weakens it further to 980.425 hPa. These are grid values, not observed cyclone central pressure.

It predicts bounded pointwise state increments on a fixed regional grid without a global boundary forecast. The two pressure update branches permit at most about 3.52 hPa per six-hour step at a cell (0.50 times the checkpoint pressure standard deviation). This is a potential translation/gradient limitation, not proof that saturation is the observed cause.

It lacks a prognostic humidity/temperature column and evolving ocean coupling. The current physics loss matches derivatives, vorticity, divergence and shear against targets; it is not a numerical conservation-law solver. Increasing that weight cannot supply missing thermodynamics or guarantee physical pressure evolution.

The scalar intensity head, route proposal and field tracker can disagree. Numerous route losses coexist with comparatively small field objectives. Their numerical weights alone do not establish gradient dominance: measure shared-layer gradient norms and conflict before reweighting.

Training uses storm windows that have 120-hour targets and valid SST. That removes many partial lifecycle samples and does not teach general atmospheric evolution from every available analysis time. Parameter scaling leaves these problems intact.

Proposed architecture and data contract

1. Atmospheric state, not pressure-only images

Use a real global 1-degree parent state and a genuine 0.25-degree western-Pacific regional state with a surrounding halo. Start with MSLP, surface pressure, 10-m winds, 2-m temperature, SST, and u/v/T/q/geopotential at 850, 700, 500, 300 and 200 hPa. Include land fraction, orography, solar geometry, availability and age channels. Add ocean heat-content/mixed-layer information only when a causal, consistently covered archive exists.

Use physical units and one canonical regridding/normalization implementation for training, replay and benchmarking. A larger interpolated copy of the current 2.5-degree labels is NOT a high-resolution dataset. Reanalysis does not fully resolve the eye even at 0.25 degrees; retain a separately identified observation-scale intensity output.

Train from 2000 onward on all eligible atmospheric timestamps, including non-storm periods, with predeclared storm-disjoint/year splits. Older years can supplement training only when sources and masks are consistent, and Tip remains excluded. No future reanalysis, best-track values or operational forecast product may enter a rollout input. A global parent generated internally from t0 is a prediction, not a future observation.

2. Global-to-regional atmospheric dynamics backbone

Use a spherical multi-resolution graph or spherical neural-operator backbone for the global state, with vertically coupled tokens. It forecasts evolving ridges, troughs and steering winds, not just a global pooled context vector. A regional windowed-attention/ConvNeXt operator consumes the parent forecast, its own prior predicted fields, and lateral boundary state at each step. Decode all atmospheric variables jointly.

Start with supervised six-hour state evolution and multi-step training. A differentiable transport operator can be tested for prognostic quantities with a learned physical-process residual, but do not treat MSLP as a passive scalar or claim conservation from a simple image warp. If implementing a full dynamics solver, use an established tested solver rather than an unverified hand-written approximation.

Do not require a moving cyclone crop for the first prototype: a fixed regional field avoids using true future centers to choose inputs and avoids route-driven template placement. Predicted-center local attention is allowed, but cannot move the output field to force agreement.

3. One coupled storm-object and field representation

```
101
102 Maintain a persistent storm-identity token initialized from the observed issue-time storm. Cross-attend to
    generated pressure, vorticity, wind and warm-core features. Forecast a spatial association distribution over the
     generated field, retaining multiple candidate lows where necessary. The final route is a subgrid center
    associated with a generated meteorological object, not an independent trajectory plus a pasted low.
103
104 Use a motion prior only as a soft association aid. Avoid a global minimum tracker, and avoid an expectation
    between two distinct lows. Model cyclone existence, weakening, transition and association confidence explicitly;
     do not force a tropical vortex to survive after genuine dissipation.
105
106 The pressure field remains the atmospheric decoder's raw output. Grid-scale pressure, observation-scale central
    pressure, and maximum surface wind have different contracts. Train the subgrid intensity decoder against
    available observations and calibrate its relation to the resolved vortex. Do not force central pressure to equal
     a coarse cell or claim a head offset creates a resolved eye.
107
108 ### 4. Conditional probabilistic extension, after the deterministic baseline
109
110 Add a spatially coherent stochastic residual to the atmospheric evolution, shared across variables and time,
    using a conditional flow or FGN-style formulation. Each ensemble member must contain its own full field, storm
    association, track and intensity. The purpose is to represent uncertain storm positions without averaging
    several locations into a diffuse low.
111
112 Keep the deterministic baseline for ablation. Score members/distributions properly; never choose the
    best-looking member against future truth. In the demo distinguish a fixed-seed member from the ensemble mean. A
    smooth ensemble-mean field is not itself proof of a bad probabilistic forecast.
113
114 ### 5. Training stages and sizes
115
116 1. Pretrain atmospheric next-state prediction on non-storm and storm times, then train genuinely free-running
    48/120-hour rollouts with predicted lateral boundaries.
117 2. Add the storm-object and intensity decoders and joint training. Use field accuracy/structure and storm
    association losses with logged gradient scales, not ten overlapping route terms by default.
118 3. Add stochastic evolution only if deterministic members lose structure under uncertainty, and evaluate
    calibration as well as sharpness.
119
120 Initial trainable target: **40-60M total**, for example a roughly 25-30M global core plus 15-25M regional/object
     components. These are architecture budgets, not measured parameter counts or confirmed T4 fits. Test the same
    architecture at approximately 25M, 50M and 100M under matched data/optimization before attributing gains to
    size. A large global dataset can require substantially more training than previous storm-only runs.
121
122 Practical alternative: freeze an existing pretrained global weather backbone and train only the regional/object
    refinement. This may be the fastest skillful system, but label it a hybrid and disclose its pretrained-data
    coverage, license and full parameter count. Do not present its accuracy as a standalone TrackFormer win over
    that same parent. No parent product or remote service should be silently introduced.
123
124 ## Promotion gates before calling v164 better
125
126 - Shared input tests: exact timestamps, field units, channel order, coordinate round trip, longitude wrapping,
    missingness, normalization and source checksums. Delete future-label tensors from the forward interface and
    verify predictions are unchanged when verification labels are shuffled.
127 - Missing SST: either fetch a verified causal patch or use a masked encoder trained with modality dropout. No
    unmasked all-zero fallback on an always-present training feature.
128 - Genuine field skill: matched-grid area-weighted MSLP MAE/RMSE plus anomaly and gradient error, spectral
    diagnostics, cyclone-centered pressure contrast and radial profiles, low-center displacement, and co-location
    with vorticity/winds. Evaluate core and background separately.
129 - Route: per-step direction, turning/recurvature timing, cross-track and along-track error, and full-path mean
    distance. Do not substitute a single terminal-distance score or optimize only appearance.
130 - Intensity: separate observed central-pressure/wind errors from gridded MSLP error; keep wind averaging
    conventions explicit.
131 - Multi-storm free rollouts: tropical, recurving, landfalling and dissipating cases; report failure rate and
    confidence. A persistence baseline must be included.
132 - Frozen validation selection, storm-clustered bootstrap uncertainty, and a new untouched test cohort. Tip is
    now a development diagnostic due repeated inspection; it cannot remain an untouched test merely because it is
    excluded from gradient training. The repeatedly used 270 cases likewise require an independent final test.
133 - Gates must not be tuned to force the DeepMind ranking. Compare the same initialization/verification sources
    and disclose pretrained train/test overlap where known.
134
135 ## Research basis and limits
136
137 - [GraphCast](https://arxiv.org/abs/2212.12794): useful basis for multi-scale global atmospheric forecasting
    rather than a storm-only latent backbone.
138 - [WeatherNext Cyclones, official
    explanation](https://deepmind.google/blog/weathernext-ai-model-achieves-breakthrough-in-forecasting-cyclones/):
    combines global atmospheric prediction with cyclone observations and coherent ensembles. Its reported Mini
    performance also shows that resolution alone is not a sufficient explanation for our failure. Adopting the
    design principles does not reproduce its skill or training corpus.
139 - [StormCast](https://arxiv.org/abs/2408.10958): supports conditioning regional probabilistic evolution on
    synoptic variables and evaluating realistic spectra. Its short-range kilometer-scale experiments are not proof
    of 120-hour western-Pacific cyclone skill.
140 - [NeuralGCM](https://www.nature.com/articles/s41586-024-07744-y): supports investigating tested hybrid dynamics
     and learned physics, not adding an arbitrary pressure-balance penalty and calling it physically guaranteed.
141
142 Bottom line: the defensible v164 change is stronger atmospheric evolution plus a genuinely coupled storm
    representation, with verified data and evaluation contracts. Tip supports investigating this direction; it does
    not prove that more parameters or a particular new block will win.
```